

么我们先从构建 1 位宽 ALU 开始。

A.5.1 1 位 ALU

逻辑操作是最简单的，因为可以直接映射为图 A-2-1 中的硬件元件。

AND 和 OR 对应的 1 位逻辑单元如图 A-5-1 所示。右边的多路选择器选择是进行 a AND b 操作还是 a OR b 操作，取决于 Operation 的值为 0 还是 1。为了与数据信号线进行区分，多路选择器的控制信号用灰色表示。需要注意的是，我们重新命名了多路选择器的控制线和输出线，以便反映它们在 ALU 中的功能。

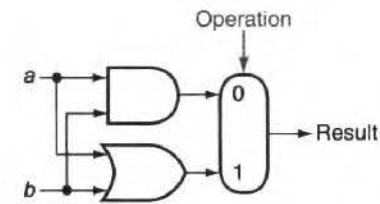


图 A-5-1 与和或的 1 位逻辑单元

下一个需要增加的功能是加法。加法器必须包含两个输入操作数，并输出一位和。同时，需要另外一个输出来传输进位，称为 CarryOut。因为来自相邻加法器的进位是作为输入处理的，因此加法器需要第三个输入。这个输入称为 CarryIn。图 A-5-2 显示了一位加法器的输入和输出。因为我们知道加法应该做什么，因此可以根据输入来指定这个“黑匣子”的输出，如图 A-5-3 所示。

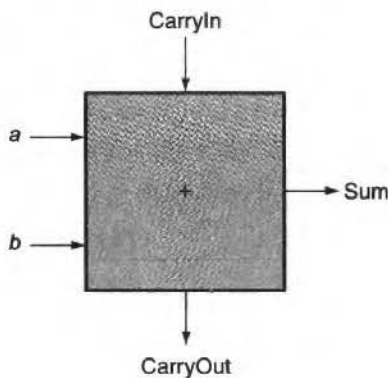


图 A-5-2 1 位加法器。该加法器称为全加法器，也称为 (3, 2) 加法器，因为有 3 个输入和 2 个输出。如果一个加法器只有 a 和 b 两个输入，则称为 (2, 2) 加法器或半加法器

输入			输出		描述
a	b	CarryIn	CarryOut	Sum	
0	0	0	0	0	$0 + 0 + 0 = 00_{\text{two}}$
0	0	1	0	1	$0 + 0 + 1 = 01_{\text{two}}$
0	1	0	0	1	$0 + 1 + 0 = 01_{\text{two}}$
0	1	1	1	0	$0 + 1 + 1 = 10_{\text{two}}$
1	0	0	0	1	$1 + 0 + 0 = 01_{\text{two}}$
1	0	1	1	0	$1 + 0 + 1 = 10_{\text{two}}$
1	1	0	1	0	$1 + 1 + 0 = 10_{\text{two}}$
1	1	1	1	1	$1 + 1 + 1 = 11_{\text{two}}$

图 A-5-3 1 位加法器的输入输出定义

我们可以用逻辑方程式来表示输出函数 CarryOut 和 Sum，这些方程式又可以通过逻辑门来实现。以 CarryOut 为例。图 A-5-4 显示了当 CarryOut 为 1 时，对应输入的值。

我们可以将该真值表转化为逻辑方程式：

$$\text{CarryOut} = (b \cdot \text{CarryIn}) + (a \cdot \text{CarryIn}) + (a \cdot b) + (a \cdot b \cdot \text{CarryIn})$$

输入		
<i>a</i>	<i>b</i>	CarryIn
0	1	1
1	0	1
1	1	0
1	1	1

图 A-5-4 当 CarryOut 为 1 时的输入值

如果 $a \cdot b \cdot \text{CarryIn}$ 为真，则剩余的三个乘积项也必然为真，因此我们可以对应真值表的第 4 行将最后一项省略掉，从而将方程式简化为：

$$\text{CarryOut} = (b \cdot \text{CarryIn}) + (a \cdot \text{CarryIn}) + (a \cdot b)$$

图 A-5-5 显示了加法器黑盒内部的硬件，其中 CarryOut 由三个与门和一个或门组成。三个与门分别对应上式中括号内的乘积项，或门用来得到三个乘积项的和。

当有一个输入为 1 或三个输入都为 1 时，Sum 设置为 1。Sum 对应的布尔等式较为复杂 ($\bar{a}$ 表示 NoT a)，如下所示：

$$\text{Sum} = (a \cdot \bar{b} \cdot \text{CarryIn}) + (\bar{a} \cdot b \cdot \text{CarryIn}) + (\bar{a} \cdot \bar{b} \cdot \text{CarryIn}) + (a \cdot b \cdot \text{CarryIn})$$

如何画出加法器中 Sum 对应的逻辑，留给读者作为练习。

图 A-5-6 所示的是用加法器和之前的部件组成的 1 位 ALU。有时设计人员也希望 ALU 能完成更多的简单操作，比如生成 0。增加操作最简单的方法是扩大由操作线控制的多路选择器，例如，将 0 直接连到扩展的多路选择器的新输入端。

A-26
}
A-28

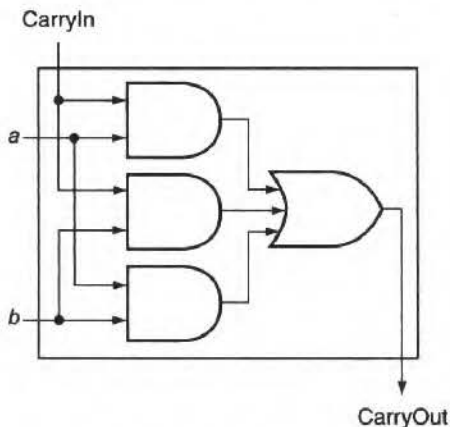


图 A-5-5 加法器中产生 CarryOut 信号的硬件。加法器硬件的剩余部分即上式中输出和 (Sum) 的硬件逻辑

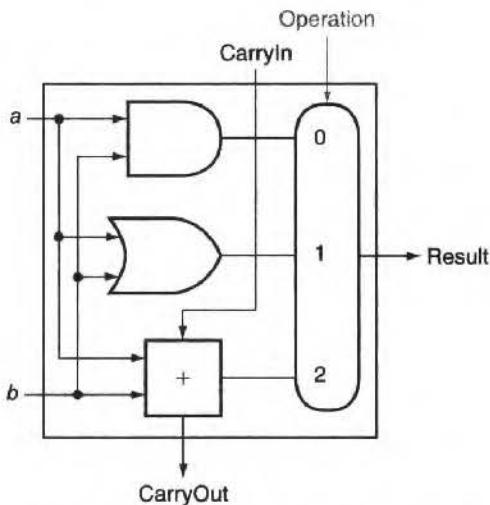


图 A-5-6 完成“与”“或”和“加法”运算的一个 1 位 ALU (见图 A-5-5)

A.5.2 64 位 ALU

既然已经实现了 1 位的 ALU，那么 64 位 ALU 就可以通过将相邻的“黑盒子”连接构成。用 x_i 表示 x 的第 i 位，图 A-5-7 所示的是一个 64 位的 ALU。犹如一块石头也能使涟漪辐射到平静的湖岸一样，最低有效位的一个进位 (Result0) 能通过所有的加法器传输，使得最高有效位 (Result63) 产生进位[⊖]。因此，将 1 位加法器进位直接相连的加法器称为行波进

⊖ 原书为 Result31，应为笔误。——译者注

位加法器。从 A.6 节开始我们将看到一种更快连接 1 位加法器的方法。

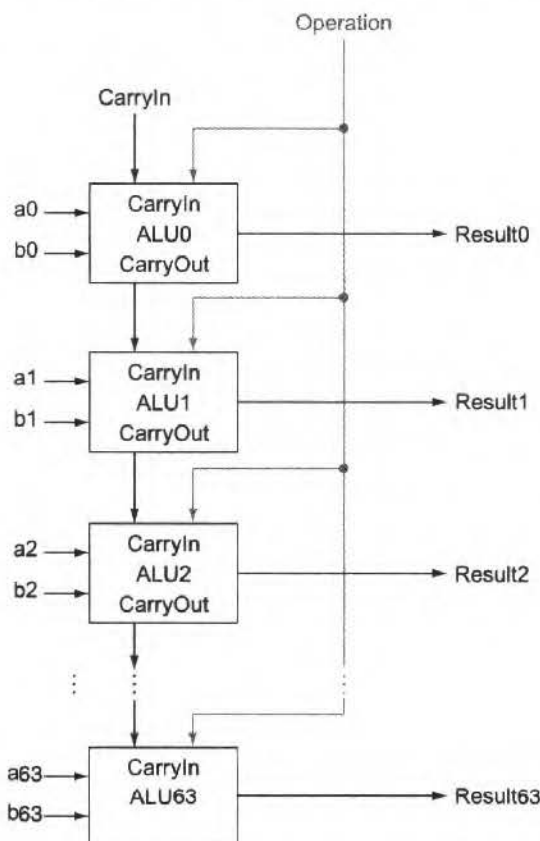


图 A-5-7 由 64 个 1 位 ALU 构成的 64 位 ALU。低有效位的 CarryOut 信号连接到高有效位的 CarryIn 信号上，这种组织方式称为行波进位

减法和加上操作数的负数是等价的，因此可以用加法器执行减法。对一个二进制补码数求相负数的快速方法是，将这个数按位取反，然后加 1。为了反转每一位，我们只需增加一个 2:1 多路选择器，用以在 b 和 $\bar{b}$ 之间进行选择，如图 A-5-8 所示。

假设将 64 个 1 位的 ALU 连接到一起，如图 A-5-7 所示。所添加的多路选择器根据 Binvert 信号选择 b 或其按位取反数，但是这仅是对二进制补码数求负数的一个步骤。注意，最低位仍然有一个 CarryIn 信号，即使这对加法并不是必要的。如果我们用 1 代替 0 来设置 CarryIn 信号，将会发生什么？加法器会计算 $a + b + 1$ 。通过将 b 取反，就能得到我们想要的结果：

$$a + \bar{b} + 1 = a + (\bar{b} + 1) = a + (-b) = a - b$$

二进制补码加法器设计的简单性有助于解释为什么二进制补码会成为整数计算机算术运算的通用标准。

LEGV8 的 ALU 还需要或非 (NOR) 功能。我们可以通过重复使用 ALU 内部已经有的硬件（如同我们在减法的实现中所做的一样）来实现这种功能，而不需单独增加一个或非门。基本的原理来自于以下有关或非的事实：

$$(\overline{a + b}) = \bar{a} \cdot \bar{b}$$

即 a 或 b 的非和非 a 与非 b 是等价的，这也被称为德·摩根定律，练习题中将对此进行更加深入的探究。

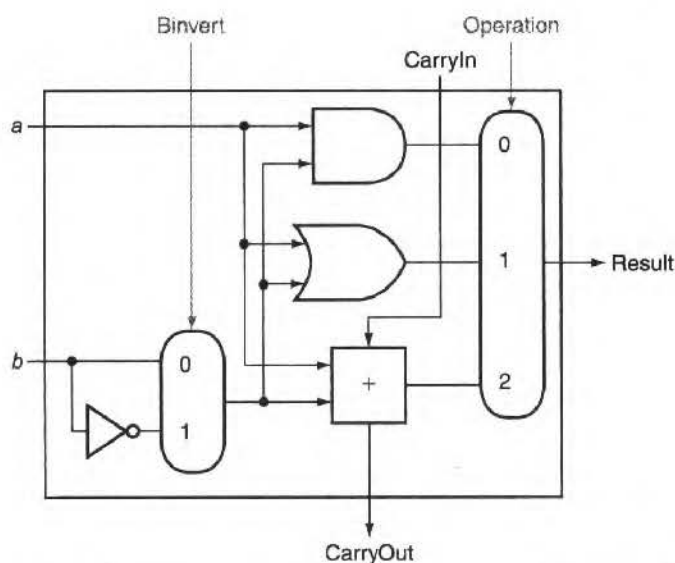


图 A-5-8 对 a 和 b 或 a 和 $\bar{b}$ 执行与、或以及加法的 1 位 ALU。通过选择 $\bar{b}$ ($\text{Binvert} = 1$)，并将最低有效位上的 CarryIn 设置为 1，我们得到 b 的补码减法，而不是 b 到 a 的加法

ALU 上已经有了与和非 b ，因此只需要再增加非 a ，图 A-5-9 所示的是改变后的结构。

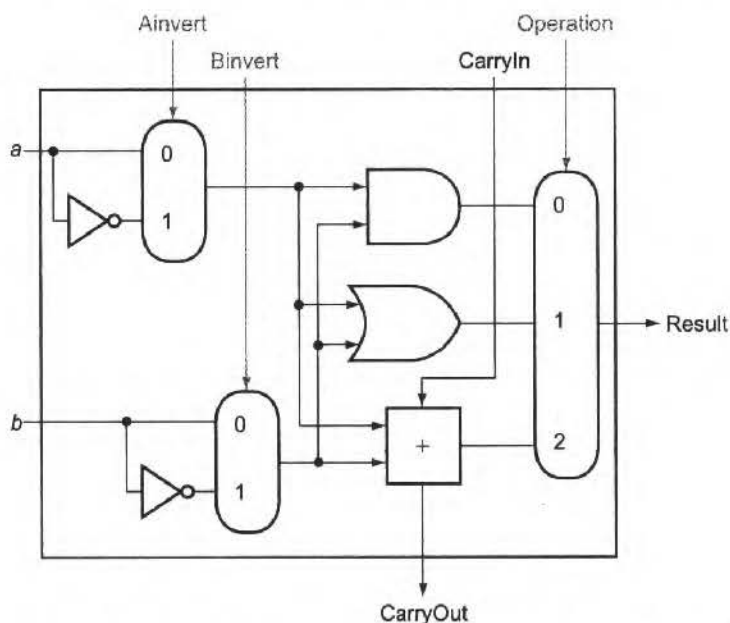


图 A-5-9 对 a 和 b 或者 $\bar{a}$ 和 $\bar{b}$ 进行与、或以及加法的 1 位 ALU。通过选择 $\bar{a}$ ($\text{Ainvert}=1$) 和 $\bar{b}$ ($\text{Binvert} = 1$)，能得到 a 或非 b 而不是 a 与 b

A.5.3 实现 LEGv8 的 64 位 ALU

加、减、与、或四个操作几乎在每一台计算机的 ALU 部件里面都能找到，并且大多数的 LEGv8 指令都能由 ALU 实现。但 ALU 的设计并未完成。

还有一条需要支持的指令是比较为 0 分支指令 CBZ。回顾一下，ALU 只传输寄存器输入值的操作。要使得 ALU 执行 CBZ，我们首先需要扩展图 A-5-8 中的三输入多路选择器，为 CBZ 结果增加一个输入。新的输入称为 Pass，并仅用于 CBZ。

图 A-5-10 上面给出的是一个新的扩展了多路选择器的 1 位 ALU。图 A-5-11 给出了 64 位 ALU。

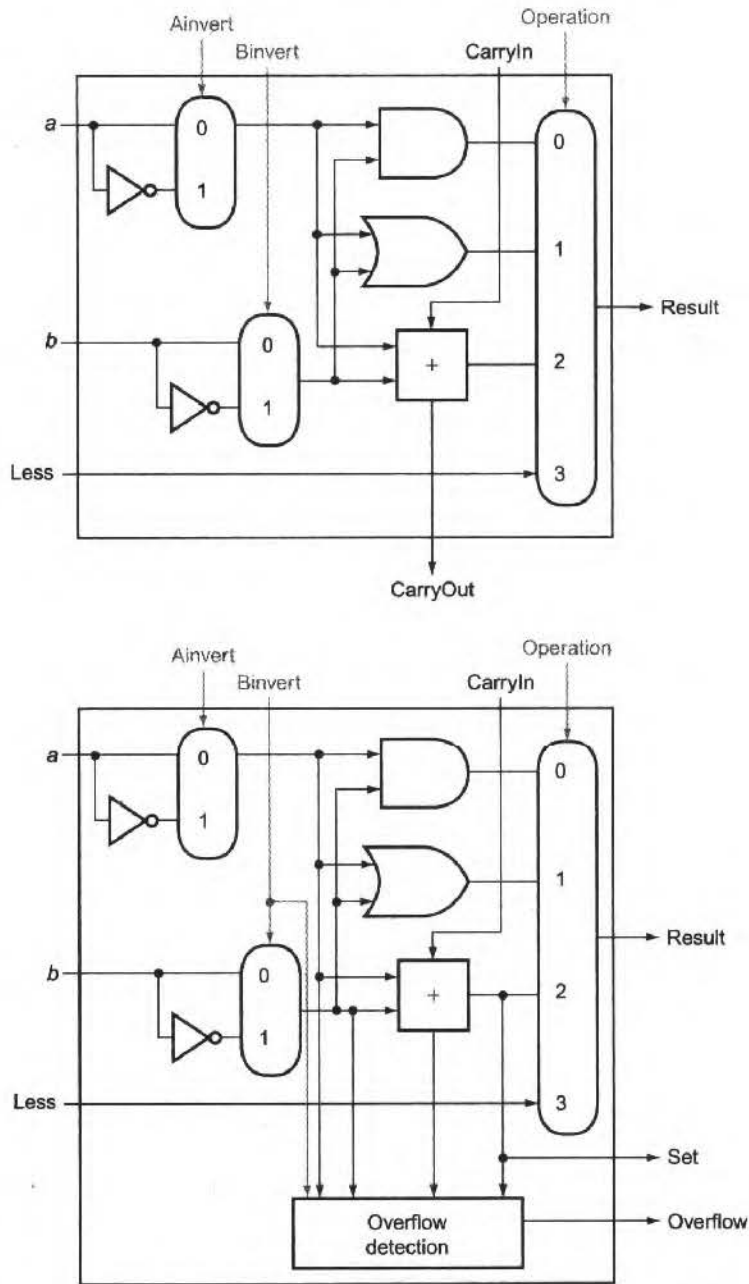


图 A-5-10 顶部为一个对 *a* 和 *b* 或者 $\bar{b}$ 执行与、或以及加法的 1 位 ALU，底部为最高有效位的 1 位 ALU。顶部的图上有直接输入，用以执行 *slt* 操作（见图 A-5-11）；底部图中的加法器有一个用于指示小于比较的直接输出，称为 *Set*（见练习题 A.24，了解如何用较少的输入计算溢出）

为了进一步实现适用于 LEV8 指令集的 ALU，我们必须支持条件分支指令。这些指令根据两个寄存器相等或不相等进行分支。通过 ALU 检测是否相等最简单的方法是执行 $a - b$ 的操作，并测试结果是否为 0，由于

$$(a - b = 0) \Rightarrow a = b$$

因此，如果我们增加检测结果是否为 0 的硬件，就能测试两个寄存器是否相等。最简单的方式是将所有输出进行或操作并将结果取反（送入反相器），即：

$$\text{Zero} = \overline{(\text{Result63} + \text{Result62} + \dots + \text{Result2} + \text{Result1} + \text{Result0})}$$

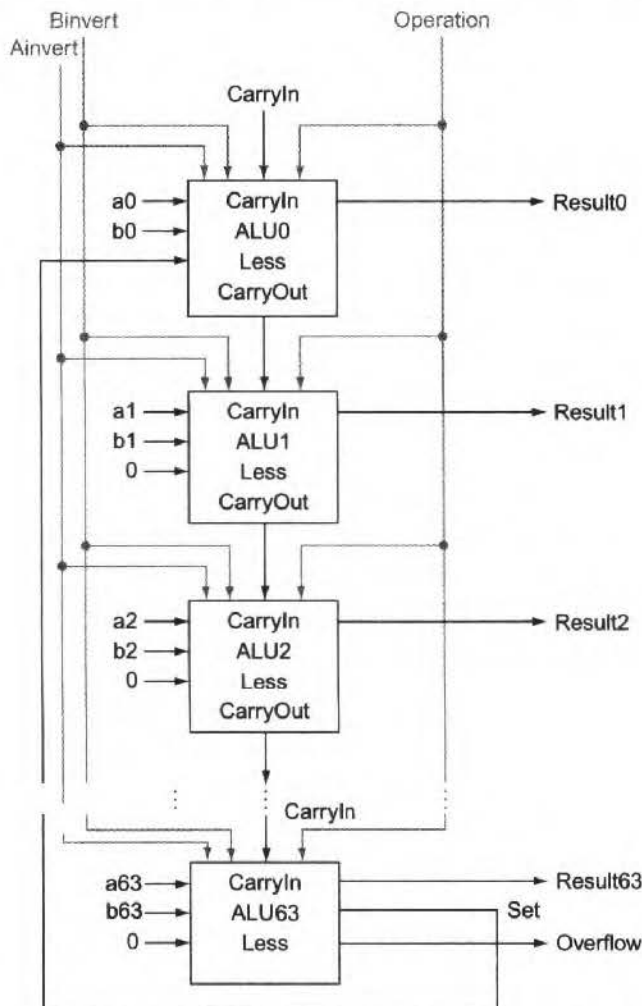


图 A-5-11 一个复制 63 个图 A-5-10 顶部结构和一个图 A-5-10 底部结构组成的 64 位 ALU。除了最低有效位，输入 Less 都连接到 0，最低有效位连接至最高有效位的 Set 输出。如果 ALU 执行 $a - b$ ，并且我们选择图 A-5-10 中多路选择器的输入 3，那么如果 $a < b$ ， $\text{Result} = 0 \cdots 001$ ，其他情况 $\text{Result} = 0 \cdots 000$

图 A-5-12 给出了修改后的 64 位 ALU。考虑将 1 位 Ainvert 线、1 位 Binvert 线以及 2 位的 Operation 线作为 4 位的 ALU 控制线结合在一起，用以指示进行加、减、与、或，或者 slt 指令中的哪一种。图 A-5-13 给出了 ALU 控制线以及相应的 ALU 操作。

最终，我们看到了 64 位 ALU 的内部结构，我们用通用符号表示完整的 ALU，如图 A-5-14 所示。

A.5.4 用 Verilog 定义 LEGv8 ALU

图 A-5-15 展示了如何用 Verilog 定义 LEGv8 ALU（组合逻辑），这样的定义可能会通过实例化标准单元库中的加法器来进行编译。为了完整性，图 A-5-16 中展示了 LEGv8 的 ALU 控制器（在第 4 章使用过），在这个控制器上我们建立了一个 Verilog 版本的 LEGv8 数据通路。

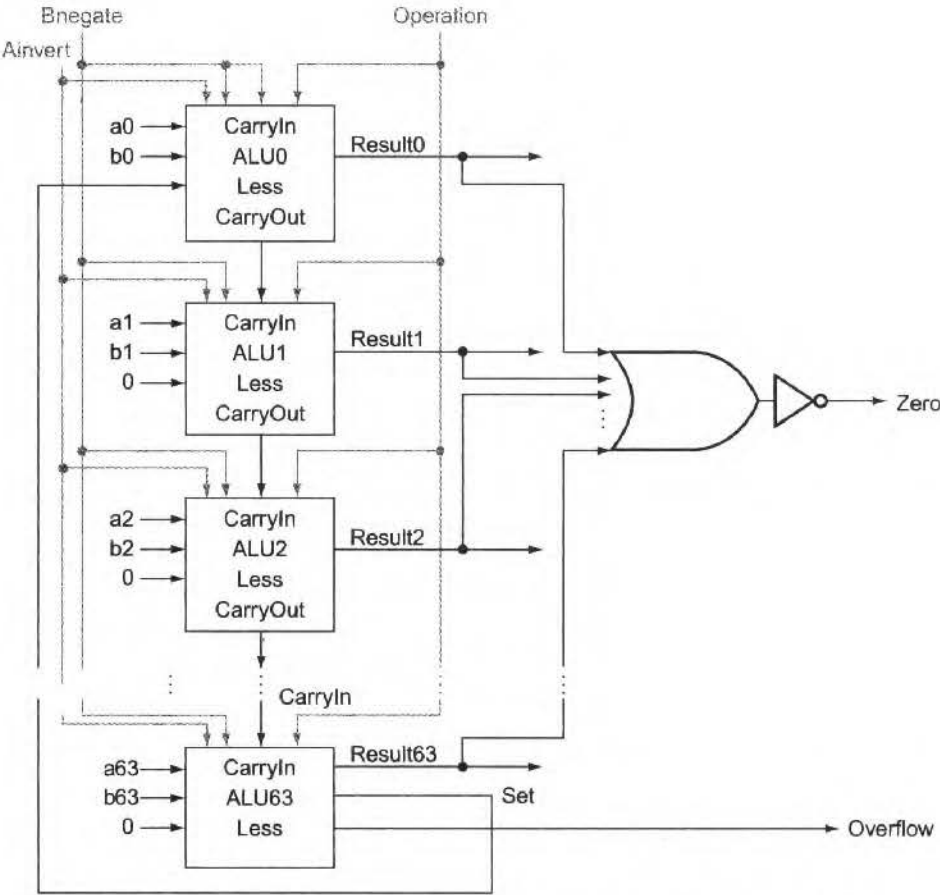


图 A-5-12 最终的 64 位 ALU。在图 A-5-11 的结构上增加了一个 0 检测器

ALU控制线	功能
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

图 A-5-13 三个 ALU 控制线、Bnegate 和 Operation 的值以及对应的 ALU 操作

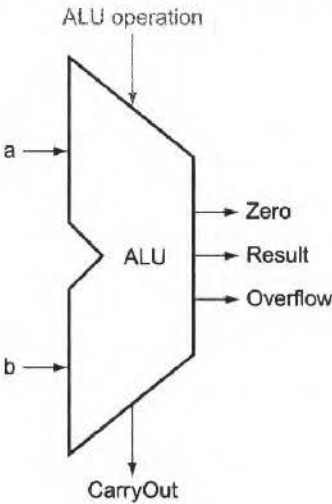


图 A-5-14 ALU 常用表示符号。该符号也用来表示加法器，因此通常需要标记出 ALU 或 Adder

```

module MIPSALU (ALUctl, A, B, ALUOut, Zero);
  input [3:0] ALUctl;
  input [63:0] A,B;
  output reg [63:0] ALUOut;
  output Zero;
  assign Zero = (ALUOut==0); //Zero is true if ALUOut is 0
  always @(ALUctl, A, B) begin //reevaluate if these change
    case (ALUctl)
      0: ALUOut <= A & B;
      1: ALUOut <= A | B;
      2: ALUOut <= A + B;
      6: ALUOut <= A - B;
      7: ALUOut <= A < B ? 1 : 0;
      12: ALUOut <= ~(A | B); // result is nor
      default: ALUOut <= 0;
    endcase
  end
endmodule

```

图 A-5-15 LEGv8 ALU 的 Verilog 行为级定义

A-34
}
A-36

```

module ALUControl (ALUOp, FuncCode, ALUctl);
  input [1:0] ALUOp;
  input [5:0] FuncCode;
  output [3:0] reg ALUctl;
  always case (FuncCode)
    32: ALUOp<=2; // add
    34: ALUOp<=6; //subtract
    36: ALUOp<=0; // and
    37: ALUOp<=1; // or
    39: ALUOp<=12; // nor
    42: ALUOp<=7; // slt
    default: ALUOp<=15; // should not happen
  endcase
endmodule

```

图 A-5-16 LEGv8 ALU 控制：简单的组合控制逻辑

下一个问题是，ALU 将两个 64 位的操作数相加能有多快？我们能决定输入 a 和 b ，但是输入 CarryIn 取决于相邻的 1 位加法器的操作。如果我们跟踪有依赖关系的进位链，将最高有效位连接到最低有效位上，那么和的最高有效位必须等到所有 64 个 1 位加法器顺序完成计算后才能得到。这种顺序链的反应太慢，以至于不能在时间关键的硬件电路中使用。下一节将探究如何加快加法的速度。这个论题对于理解附录的其余部分并不重要，可以跳过。

小测验 假设想增加 NOT (a AND b) 操作，称为与非 (NAND)，应如何修改 ALU 以支持该操作？

1. 没有改变。你可以用当前的 ALU 快速计算出 NAND，因为 $\overline{(a \cdot b)} = \bar{a} + \bar{b}$ ，而且已经有 NOT a 、NOT b 以及或门。
2. 必须扩展多路选择器以增加额外的输入，然后增加新的逻辑电路来计算 NAND。

A.6 快速加法：超前进位

加快加法的关键在于更快地确定高阶位的进位。有多种方案来预测进位，使得最坏情况是加法器中所有位数 $\log_2$ 的函数。由于进位经过的逻辑门较少，所以这些预测信号执行得比较快，但是需要更多的门来进行预测。

A-37

理解快速进位的关键在于：无论输入何时改变，硬件都是并行执行的，这一点与软件不同。

A.6.1 使用“无限”硬件的快速进位

正如前面提到的，任何一个方程式都能用两级逻辑表示。因为外部输入只有两个操作数和加法器的最低位的 CarryIn，在理论上我们可以在两级逻辑层上计算出到加法器所有剩余位的 CarryIn 值。

例如，加法器 bit 2 的 CarryIn 实际是 bit 1 的 CarryOut，因此公式为：

$$\text{CarryIn}_2 = (b_1 \cdot \text{CarryIn}_1) + (a_1 \cdot \text{CarryIn}_1) + (a_1 \cdot b_1)$$

类似地，CarryIn₁ 可以定义为

$$\text{CarryIn}_1 = (b_0 \cdot \text{CarryIn}_0) + (a_0 \cdot \text{CarryIn}_0) + (a_0 \cdot b_0)$$

用 c_i 代替 CarryIn_i，上式重写为

$$c_2 = (b_1 \cdot c_1) + (a_1 \cdot c_1) + (a_1 \cdot b_1)$$

$$c_1 = (b_0 \cdot c_0) + (a_0 \cdot c_0) + (a_0 \cdot b_0)$$

将表达式 c_1 带入第一个公式，可得：

$$\begin{aligned} c_2 = & (a_1 \cdot a_0 \cdot b_0) + (a_1 \cdot a_0 \cdot c_0) + (a_1 \cdot b_0 \cdot c_0) \\ & + (b_1 \cdot a_0 \cdot b_0) + (b_1 \cdot a_0 \cdot c_0) + (b_1 \cdot b_0 \cdot c_0) + (a_1 \cdot b_1) \end{aligned}$$

可以想象一下，对于加法器中的更高位，方程式会如何扩大？它将随着位数的增加而快速增加。这一复杂性影响了快速进位的硬件开销，因此这一简单方案对于宽加法器而言非常昂贵。

A.6.2 采用第一级抽象的快速进位：进位传输和进位生成

大多数的快速进位方法都会限制方程式的复杂性以简化硬件，同时与行波进位相比速度得到了大幅度提高。其中一种机制称为超前进位加法器。在第 1 章中已经介绍了，计算机系统通过使用抽象层次来处理复杂性。超前进位加法器依赖于其实现过程中的抽象级别。

首先考虑原始方程：

$$c_{i+1} = (b_i \cdot c_i) + (a_i \cdot c_i) + (a_i \cdot b_i) = (a_i \cdot b_i) + (a_i + b_i) \cdot c_i$$

如果用这个公式重写 c_2 的方程，我们将会看到一些重复的部分：

$$c_2 = (a_1 \cdot b_1) + (a_1 + b_1) \cdot ((a_0 \cdot b_0) + (a_0 + b_0) \cdot c_0)$$

注意到 $(a_i \cdot b_i)$ 和 $(a_i + b_i)$ 在上面的公式中重复出现，这两个重要函数通常称为进位生成 (g_i) 和进位传输 (p_i)：

$$g_i = a_i \cdot b_i$$

$$p_i = a_i + b_i$$

用它们来定义 c_{i+1} ，可得：

$$c_{i+1} = g_i + p_i \cdot c_i$$

为了理解信号是从哪里得到的，假设 $g_i = 1$ ，即

$$c_{i+1} = g_i + p_i \cdot c_i = 1 + p_i \cdot c_i = 1$$

也就是说，加法器生成一个独立于 CarryIn (c_i) 值的 CarryOut (c_{i+1})。现在假设 $g_i = 0$ ， $p_i = 1$ ，则

$$c_{i+1} = g_i + p_i \cdot c_i = 0 + 1 \cdot c_i = c_i$$

也就是说，加法器将 CarryIn 传输到 CarryOut。将以上二者放在一起可得，无论 $g_i = 1$ 或者 $p_i = 1$ 且 CarryIn_i = 1，CarryIn_{i+1} = 1。

作为比喻，想象一排多米诺骨牌。假设两张牌之间没有间隙，那么推倒远处的一张牌可以将最后一张多米诺骨牌推倒。类似地，一个进位可以通过远处的一个生成因子而使其为真，只要它们之间所有的传输因子均为真。

根据传输因子和生成因子的定义，我们将其作为第一级抽象，从而能更加方便地描述 CarryIn 信号。下面所示的是 4 位的情况：

$$c1 = g0 + (p0 \cdot c0)$$

$$c2 = g1 + (p1 \cdot g0) + (p1 \cdot p0 \cdot c0)$$

$$c3 = g2 + (p2 \cdot g1) + (p2 \cdot p1 \cdot g0) + (p2 \cdot p1 \cdot p0 \cdot c0)$$

$$c4 = g3 + (p3 \cdot g2) + (p3 \cdot p2 \cdot g1) + (p3 \cdot p2 \cdot p1 \cdot g0) + (p3 \cdot p2 \cdot p1 \cdot p0 \cdot c0) \quad \text{A-39}$$

这些方程式只代表一般情况：如果之前的加法器生成了一个进位，并且所有中间加法器都传输进位，那么 $CarryIn_i = 1$ 。图 A-6-1 采用管道来解释先行（超前）进位。

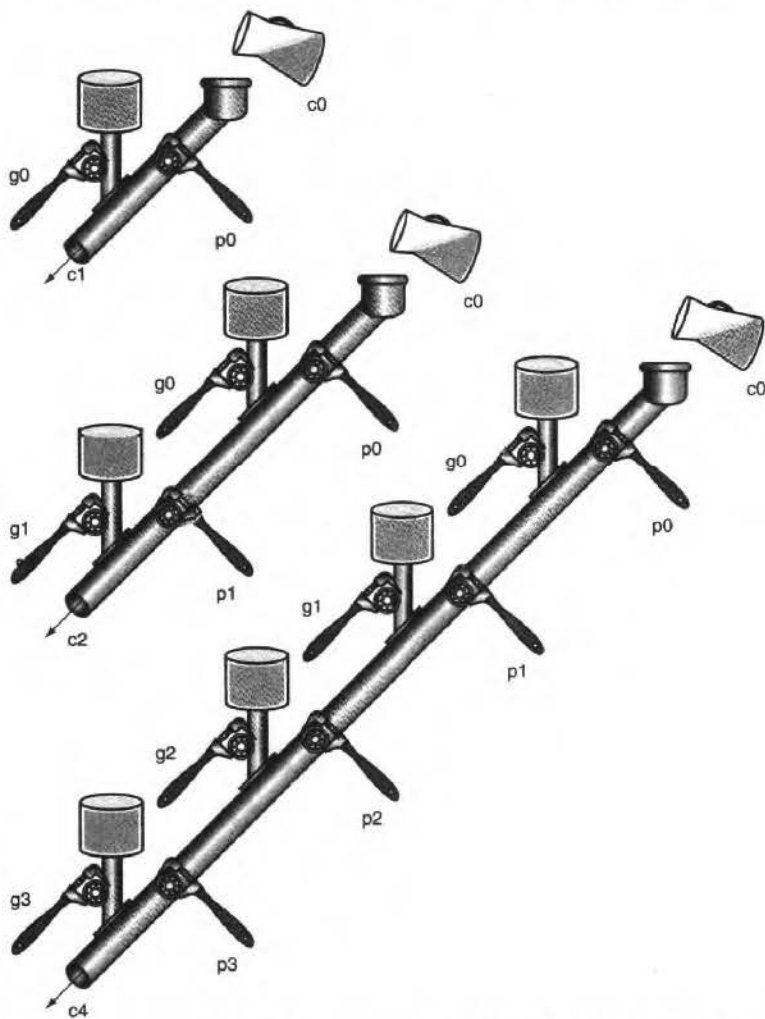


图 A-6-1 以管道和阀门类比 1 位、2 位、4 位超前进位。扳手用于打开和关闭阀门。水用灰色部分表示。如果最近的进位生成因子的值 (g_i) 处于打开状态，或者第 i 个进位传输因子 (p_i) 也是打开的，并且上游有水（来自之前生成的或从后面传过来的水），那么管道的输出 (c_{i+1}) 将会变满。没有任何生成因子， $CarryIn(c_0)$ 也能产生一个进位输出，但是需要所有传输因子的帮助

即便是这种简化的形式也会导致大的方程式，因此即使是一个 16 位的加法器也有相当可观的逻辑。下面我们试着移动两个抽象层次。

A.6.3 采用第二级抽象的快速进位

首先，我们考虑 4 位的加法器，其超前进位逻辑作为一个单独的块。如果将 4 位加法器以

行波进位方式相连构成一个 16 位的加法器，那么加法将比原来更快，并且只增加了少数的硬件。

为了执行得更快，需要将超前进位放置在更高层上。为了实现 4 位加法器的超前进位，需要将传输因子和生成因子信号也置于较高的层次。下面是 4 位加法器的块：

$$P0 = p3 \cdot p2 \cdot p1 \cdot p0$$

$$P1 = p7 \cdot p6 \cdot p5 \cdot p4$$

$$P2 = p11 \cdot p10 \cdot p9 \cdot p8$$

$$P3 = p15 \cdot p14 \cdot p13 \cdot p12$$

因此，当且仅当组中每一位都将传输一个进位时，用于 4 位抽象的“超级”传输信号 (P_i) 为真。

对于“超级”生成信号 (G_i)，我们只关心 4 位的组中最高有效位是否有一个进位。如果最高有效位生成因子为真，那么这些情况是显而易见的。如果较早的一个生成因子为真，并且所有中间传输因子（包括最高有效位的）也为真，以上情况也是会出现的。

$$G0 = g3 + (p3 \cdot g2) + (p3 \cdot p2 \cdot g1) + (p3 \cdot p2 \cdot p1 \cdot g0)$$

$$G1 = g7 + (p7 \cdot g6) + (p7 \cdot p6 \cdot g5) + (p7 \cdot p6 \cdot p5 \cdot g4)$$

$$G2 = g11 + (p11 \cdot g10) + (p11 \cdot p10 \cdot g9) + (p11 \cdot p10 \cdot p9 \cdot g8)$$

$$G3 = g15 + (p15 \cdot g14) + (p15 \cdot p14 \cdot g13) + (p15 \cdot p14 \cdot p13 \cdot g12)$$

图 A-6-2 更新了管道类比，以显示 $P0$ 和 $G0$ 。

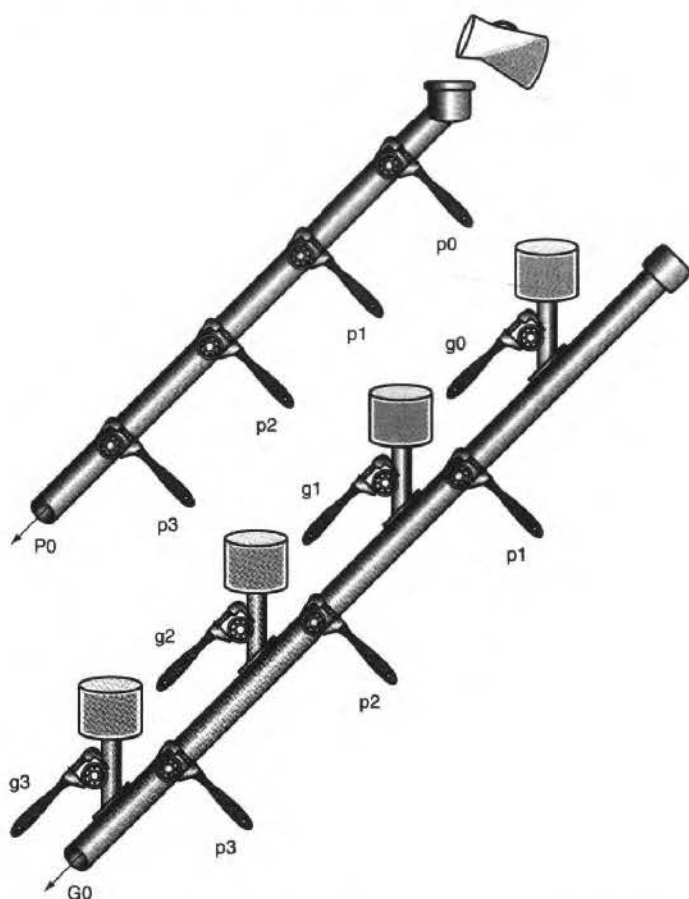


图 A-6-2 下一级超前进位信号 $P0$ 和 $G0$ 的管道类比。仅当所有 4 个进位传输因子 (p_i) 都打开时 $P0$ 打开； $G0$ 里有水流，仅当至少有一个进位生成因子 (g_i) 打开，并且从该生成因子开始下游所有的进位传输因子都是打开的

对 16 位加法器的每一个 4 位组的进位在较高的层次上抽象为 $C1$ 、 $C2$ 、 $C3$ 、 $C4$ (如图 A-6-3), 类似于 A.6.2 节 4 位加法器的 $c1$ 、 $c2$ 、 $c3$ 、 $c4$:

$$C1 = G0 + (P0 \cdot c0)$$

$$C2 = G1 + (P1 \cdot G0) + (P1 \cdot P0 \cdot c0)$$

$$C3 = G2 + (P2 \cdot G1) + (P2 \cdot P1 \cdot G0) + (P2 \cdot P1 \cdot P0 \cdot c0)$$

$$C4 = G3 + (P3 \cdot G2) + (P3 \cdot P2 \cdot G1) + (P3 \cdot P2 \cdot P1 \cdot G0) + (P3 \cdot P2 \cdot P1 \cdot P0 \cdot c0)$$

图 A-6-3 中 4 位的加法器连接到这样一个超前进位单元。练习题中会探究这些进位方案间速度的差异、多位传输信号和生成信号的不同表示, 并讨论 64 位加法器的设计。

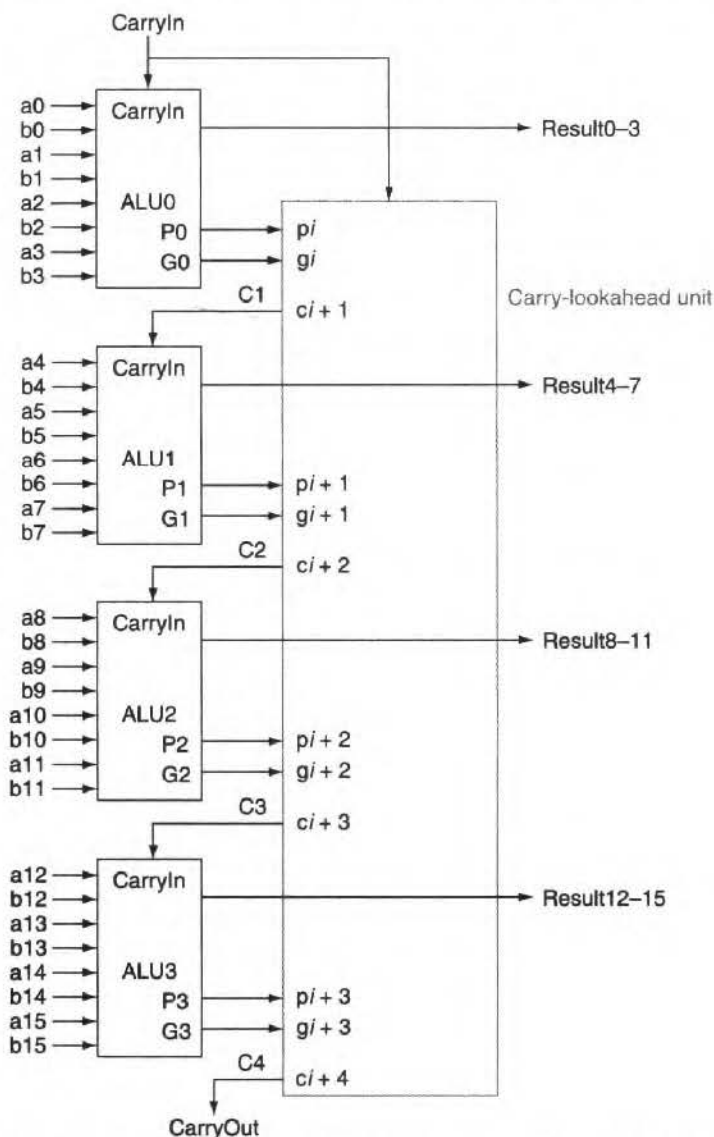


图 A-6-3 4 个 4 位 ALU 使用超前进位形成 16 位加法器。注意, 进位均来自超前进位单元, 而不是 4 位的 ALU

例 题 | 进位传输因子和进位生成因子

确定下列两个 16 位数的 gi 、 pi 、 Pi 以及 Gi 值:

a: 0001 1010 0011 0011_{two}
b: 1110 0101 1110 1011_{two}

同样, CarryOut15 (C4) 的值是多少?

答案 将各位对准, 以便更容易观察进位生成因子 $g_i (a_i \cdot b_i)$ 和进位传输因子 $p_i (a_i + b_i)$ 的值:

a:	0001	1010	0011	0011
b:	1110	0101	1110	1011
g_i :	0000	0000	0010	0011
p_i :	1111	1111	1111	1011

从左到右各位依次标记为 15 ~ 0。“超级”进位传输因子 (P3、P2、P1、P0) 是低级进位传输因子的简单相与。

$$P3 = 1 \cdot 1 \cdot 1 \cdot 1 \cdot 1$$

$$P2 = 1 \cdot 1 \cdot 1 \cdot 1 \cdot 1$$

$$P1 = 1 \cdot 1 \cdot 1 \cdot 1 \cdot 1$$

$$P0 = 1 \cdot 0 \cdot 1 \cdot 1 \cdot 0$$

“超级”进位生成因子较复杂一些, 用下式表示:

$$\begin{aligned} G0 &= g3 + (p3 \cdot g2) + (p3 \cdot p2 \cdot g1) + (p3 \cdot p2 \cdot p1 \cdot g0) \\ &= 0 + (1 \cdot 0) + (1 \cdot 0 \cdot 1) + (1 \cdot 0 \cdot 1 \cdot 1) = 0 + 0 + 0 + 0 = 0 \end{aligned}$$

$$\begin{aligned} G1 &= g7 + (p7 \cdot g6) + (p7 \cdot p6 \cdot g5) + (p7 \cdot p6 \cdot p5 \cdot g4) \\ &= 0 + (1 \cdot 0) + (1 \cdot 1 \cdot 1) + (1 \cdot 1 \cdot 1 \cdot 0) = 0 + 0 + 1 + 0 = 1 \end{aligned}$$

$$\begin{aligned} G2 &= g11 + (p11 \cdot g10) + (p11 \cdot p10 \cdot g9) + (p11 \cdot p10 \cdot p9 \cdot g8) \\ &= 0 + (1 \cdot 0) + (1 \cdot 1 \cdot 0) + (1 \cdot 1 \cdot 1 \cdot 0) = 0 + 0 + 0 + 0 = 0 \end{aligned}$$

$$\begin{aligned} G3 &= g15 + (p15 \cdot g14) + (p15 \cdot p14 \cdot g13) + (p15 \cdot p14 \cdot p13 \cdot g12) \\ &= 0 + (1 \cdot 0) + (1 \cdot 1 \cdot 0) + (1 \cdot 1 \cdot 1 \cdot 0) = 0 + 0 + 0 + 0 = 0 \end{aligned}$$

最后, CarryOut15 为:

$$\begin{aligned} C4 &= G3 + (P3 \cdot G2) + (P3 \cdot P2 \cdot G1) + (P3 \cdot P2 \cdot P1 \cdot G0) + (P3 \cdot P2 \cdot P1 \cdot P0 \cdot c0) \\ &= 0 + (1 \cdot 0) + (1 \cdot 1 \cdot 1) + (1 \cdot 1 \cdot 1 \cdot 0) + (1 \cdot 1 \cdot 1 \cdot 0 \cdot 0) \\ &= 0 + 0 + 1 + 0 + 0 = 1 \end{aligned}$$

因此, 这两个 16 位的数相加之后会有一个进位输出。 —————

超前进位能快速进位的原因是, 当时钟周期开始时所有的逻辑单元同时开始计算, 并且一旦每个门的输出停止变化, 结果就不会改变。通过利用更少的门发送进位信号这种快捷方式, 门的输出将很快停止变化, 因此加法器花费的时间就变少了。

为了更好地理解超前进位的优点, 我们可以计算其与行波进位加法器之间的相对性能。

例题 行波进位加法器和超前进位加法器速度的比较 —————

一种为逻辑建立时间模型的简单方法是, 假设每个与门或者或门的延迟是相同的。通过简单计算逻辑路径上门的数量来估计时间。比较两个 16 位加法器路径上门延迟的数量, 一个用行波进位, 另一个用两级超前进位。

答案 A.5 节中图 A-5-5 所示的每个进位输出信号每位需要两个门延迟。从最低有效位上的进位输入到最高有效位上的进位输出之间的门延迟为 $32 \times 2 = 64$ 。

对于超前进位, 最高有效位的进位输出正是例子中定义的 C4。用 P_i 和 G_i 两级逻辑 (几个 AND 组成的 OR 式) 定义了 C4。 P_i 在一级逻辑 (AND) 中用 p_i 定义, G_i 在两级逻辑中用 p_i 和 g_i 定义。因此, 对该下一级抽象最差的情况是两级逻辑。 p_i 和 g_i 都是用 a_i 和 b_i 定义的一级逻辑。

辑。假设这些方程中，每个逻辑级都是一个门延迟，那么最坏的情况是 $2+2+1=5$ 个门延迟。

因此，采用这种简单的硬件速度估算方法，对于从进位输入到进位输出的路径，16 位超前进位加法器的速度是行波进位加法器的 6 倍。

A.6.4 小结

超前进位提供了比 32 个 1 位加法器构成的行波进位更快的路径。这个快速通路由两个信号保证，即进位生成因子和进位传输因子。前者在不考虑进位输入的情况下计算进位，而后者传输进位。超前进位加法器又是一个很好的例子，说明了抽象思想对于计算机设计时解决复杂化问题的重要性。

精解 我们基本已经描述了核心 LEGv8 指令集的全部操作，除了一个算术逻辑操作之外：图 A-5-14 中的 ALU 省略了对移位指令的支持。可以将 ALU 的多路复用器加宽，以包括左移 1 位或右移 1 位。但硬件设计人员设计了一种电路，称为桶形移位器 (barrel shifter)，它可以从 1 移到 63 位，而消耗的时间不多于将两个 64 位的数相加的时间，所以移位操作通常在 ALU 外部完成。

精解 A.5 节中，全加器输出 Sum 的逻辑方程可以用一个比与门和或门能力更强的门来表示，即异或门。如果两个操作数不同，异或门输出为真，即

$$x \neq y \Rightarrow 1 \quad \text{且} \quad x == y \Rightarrow 0$$

在一些技术中，异或门比两级的与门和或门的执行效率更高。用 $\oplus$ 来表示异或运算，则等式可以重新表达为：

$$\text{Sum} = a \oplus b \oplus \text{CarryIn}$$

同样，我们用这种传统的门级表示方法来表示 ALU 电路。当今的计算机都是用 CMOS 晶体管（基本是开关）设计的。CMOS ALU 以及桶形移位器利用了开关的优点，并且使用的多路选择器比文中的设计少得多，但设计原理是相似的。

精解 当有两个以上级别时，用小写和大写字母来区分进位生成和进位传输因子的层次结果。另一种表示方法是使用 $gi.j$ 和 $pi.j$ 代表从 i 位到 j 位的进位生成信号和进位传输信号。因此，位 1 生成 $g1..1$ ，位 4 到位 1 生成 $g4..1$ ，位 16 到位 1 生成 $g16..1$ 。

小测验 使用门延迟对硬件执行速度进行简单评估，8 位的行波进位加法和 64 位的超前进位加法的相对性能如何？

1. 64 位超前进位加法器快 3 倍：8 位加法器有 16 个门延迟，64 位则有 7 个门延迟。
2. 速度大约相等，因为 64 位加法在 16 位加法器中需要更多的逻辑层次。
3. 8 位加法比 64 位的快，即使有超前进位。

A.7 时钟

在我们讨论存储元件和时序逻辑之前，有必要简要地讨论一下时钟。本节讨论这一主题，内容同 4.2 节中的讨论类似。更多的关于时钟和时序方法的细节在 A.11 节讨论。

时序逻辑需要时钟来决定包含状态的存储元件何时被更新。时钟是一个具有固定周期时间的不停运行的信号，时钟频率是周期时间的倒数。如图 A-7-1 所示，时钟周期时间或者说时钟周期被分为两部分，即高电平时钟和低电平时钟。本书中，我们只使用边沿触发时钟 (edge-triggered

边沿触发时钟：一种时钟机制，所有状态改变都发生在时钟边沿。

A-45

A-46

clocking)。这意味着所有的状态改变都将发生在时钟边沿。我们之所以使用基于边沿触发的时钟策略，是因为该策略更易于解释。从工艺学的角度来看，该策略可能是也可能不是时钟同步方法（clocking methodology）的最佳选择。

时钟同步方法：一种根据时钟来决定数据何时有效和稳定的方法。

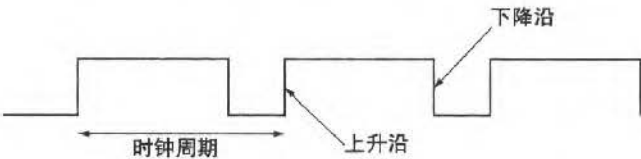


图 A-7-1 时钟信号在高电平和低电平之间振荡。时钟周期是一个完整周期的时间。在边沿触发的设计中，有效的上升沿或下降沿都可以造成状态的改变

在一种边沿触发的方法中，时钟的上升沿或下降沿都可以是有效的，并可以造成状态的改变。在下一节中我们会看到，边沿触发设计中的**状态单元**（state element），其内容仅在有效的时钟沿改变。选择哪一个时钟边沿作为有效边沿受实现策略的影响，且不会影响逻辑设计中所涉及的概念。

状态单元：一种存储元件

时钟边沿可以作为采样信号，导致状态单元的输入值被采样且存储在状态单元中。使用边沿触发器意味着采样过程实际上是瞬时的，可以消除信号在很小的时间差内被采样可能导致的问题。

时钟系统即**同步系统**（synchronous system），最主要的限制是被写入状态单元的信号在有效时钟边沿必须是有效的。信号稳定（不改变）时才是有效信号，并且在输入不变时，信号值不会改变。因为组合电路没有反馈，因此如果组合逻辑的输入没有改变，那么组合逻辑的输出最终会变为有效。

同步系统：使用时钟的存储系统，只有当时钟表明信号值是稳定状态时，数据信号才可被读取。

A-47

图 A-7-2 显示了一个同步时序逻辑设计中状态单元和组合逻辑单元间的关系。状态单元的输出只在时钟边沿之后改变，并为组合逻辑块提供有效的输入。为了保证在有效时钟边沿写入状态单元的数据是有效的，时钟周期必须足够长，才能保证组合逻辑块中的所有信号都达到稳定，并且时钟边沿采样这些数据，并将其存储在状态单元中。这个约束设置了时钟周期长度的下界，即时钟周期必须足够长，以便所有状态单元的输入有效。

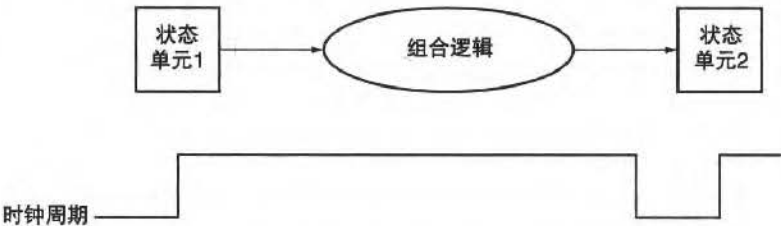


图 A-7-2 组合逻辑块的输入来自状态单元，同时其输出被写入状态单元。时钟边沿决定了状态单元的内容何时被更新

在附录的其他部分和第 4 章，我们通常忽略时钟信号，因为我们假设所有的状态单元都会在同一时钟边沿更新。一些状态单元会在每个时钟边沿被写入，而其他一些仅仅在确定的条件下被写入（如某个寄存器被更新）。在这种情况下，我们会使用一个显式的写信号来控制这个状态单元。写信号必须经过时钟门控，从而只有在写信号有效时，状态单元才能在时

钟边沿上进行更新。我们将在下一节学习和使用这一机制。

边沿触发机制的另一优势是可以将一个状态单元同时作为同一组合逻辑块的输入和输出，如图 A-7-3 所示。实际上，在这种情况下必须要防止竞争，同时要保证时钟周期足够长。这一问题将在 A.11 节讨论。

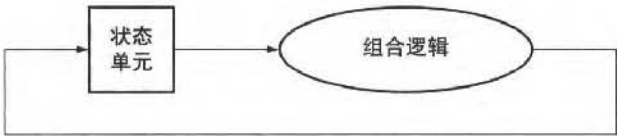


图 A-7-3 边沿触发策略允许一个状态单元在同一个时钟周期内被读写，不会引起导致不确定数据值的竞争。时钟周期必须足够长，从而使有效时钟沿到来时输入数据值是稳定的

现在我们已经讨论了时钟是如何用来更新状态单元的，可以进一步讨论如何构建状态单元。

精解 有时，设计者发现，让少数状态单元在和大多数状态单元相反的时钟沿进行状态变化是非常有用的。但是在使用这种方法时需要十分小心，因为会影响到状态单元的输入和输出。为什么设计者还要这么做呢？考虑这样的情况，一部分作为状态单元输入或者输出的组合逻辑十分小，以至于它们可以在半个周期内完成，而不是通常的完整时钟周期。因此状态单元可以在半个周期的时钟边沿被写入，因为输入和输出在半个时钟周期之后都是有效的。这种技术经常被用在寄存器文件中，在寄存器文件中，简单的寄存器文件读写通常可以在半个时钟周期内完成。第 4 章使用这种策略来减少流水线开销。

寄存器文件：包含一系列寄存器的状态单元，通过寄存器号进行读写。

A-48

A.8 存储元件：触发器、锁存器和寄存器

在本节及下一节中，我们将讨论存储元件的基本原理，从触发器、锁存器开始，再介绍寄存器文件，最后介绍存储器。所有的存储元件都存储着一些状态：存储元件的输出不仅取决于当前的输入，而且与当前存储的数据值有关。因此所有包含存储元件的逻辑块都包含有状态信息，属于时序逻辑。

最简单的存储元件类型是无时钟的，即这些元件都没有任何时钟输入。虽然本书中我们只使用时钟控制的存储元件，但因为无时钟的锁存器是最简单的存储元件，所以我们将先讨论这种元件。图 A-8-1 为一个 S-R 锁存器（set-reset 锁存器），该锁存器由一对或非门构成（或门加输出反相）。输出信号 Q 和 $\bar{Q}$ 表示存储的数据及其反相数据。当 S 和 R 都为无效时，交叉耦合的或非门就作为一个反相器，存储先前的 Q 和 $\bar{Q}$ 的值。

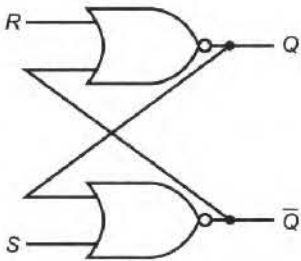


图 A-8-1 一对交叉耦合的或非门可以存储一个内部值。存储在输出 Q 上的值通过反转来获得 $\bar{Q}$ ，然后再反相 $\bar{Q}$ 以获得 Q 。如果 R 或 $\bar{Q}$ 有效，那么 Q 无效，反之亦然

A-49

例如，如果输出 Q 为真，那么下面的反相器将产生一个值为假的输出（即 $\bar{Q}$ ），这个输出又成为上面反相器的输入，上面的反相器产生一个值为真的输出，即 Q ，之后一直循环下去。

如果 S 有效, 那么输出 Q 为有效, $\overline{Q}$ 为无效; 如果 R 有效, 则输出 $\overline{Q}$ 有效, Q 无效。如果 S 和 R 都有效, 则 Q 和 $\overline{Q}$ 最后的值将被存储在交叉耦合结构内。同时将 S 和 R 置为有效, 可能会导致错误的操作: 这取决于 S 和 R 是如何被拉高的^①, 锁存器可能会不停地摆动, 也可能处于亚稳态 (这部分将在 A.11 节中详细介绍)。

这种交叉耦合结构是复杂存储元件 (存储数据信号) 的基础。这些存储元件包含额外的门用来存储信号, 并使得该状态仅与时钟一起更新。下一节将讲述如何构建这些存储元件。

A.8.1 触发器和锁存器

触发器 (flip-flop) 和锁存器 (latch) 是最简单的存储元件。在触发器和锁存器中, 输出信号的值都与存储元件中存储的状态一致。而且, 与上面提到的 S-R 锁存器不同, 从现在开始, 我们使用的所有触发器和锁存器都是带时钟的, 这意味着这些存储元件将包含时钟输入信号, 并且状态改变由时钟触发。触发器与锁存器间的差别在于, 在哪个时间点上时钟会导致状态实际改变。在一个包含时钟的锁存器中, 当输入变化并且时钟信号有效时, 状态发生变化。而在触发器中, 状态仅在时钟边沿改变。在本书中, 我们使用边沿触发的时钟方法, 即存储状态只在时钟边沿发生变化, 因此我们只使用触发器。触发器通常由锁存器构成, 因此我们先介绍简单的带时钟的锁存器, 然后讨论由这些锁存器构成的触发器的一些操作。

触发器: 一种存储元件, 其输出与内部存储的状态一致, 并且内部状态只在时钟边沿发生变化。

锁存器: 一种存储元件, 其输出与内部存储的状态一致, 只要输入发生变化并且时钟有效, 存储状态就会随之发生变化。

D 触发器: 包含一个输入数据的触发器, 这类触发器只在时钟信号的边沿, 才将输入信号存储到内部元件中。

对计算机应用程序来说, 触发器和锁存器的功能都是存储信号。**D 锁存器或 D 触发器 (flip-flop)** 将输入的数据信号存储在内部元件中。尽管有很多类型的锁存器和触发器, 但 D 触发器是我们所需的唯一的基本构件。D 锁存器包含两个输入和两个输出。两个输入中一个是要存储的数据 (D), 一个是时钟信号 (C), 时钟信号用来指示锁存器何时读取输入 D 的值并进行存储。输出信号就是内部状态 Q 及其反相信号 $\overline{Q}$ 的值。当输入时钟 C 有效时, 称为锁存器打开状态, 此时输出信号 Q 的值变为输入信号 D 的值。当时钟输入 C 无效时, 锁存器处于关闭状态, 此时输出 Q 的值等于锁存器最后一次打开时所存储的数据值。

A-50

图 A-8-2 显示了如何在交叉耦合的或非门上增加两个额外的门来构造 D 锁存器。由于当锁存器处于打开状态时, 输出 Q 的值随输入 D 的改变而变化, 因此这种结构有时也称为透明锁存器。图 A-8-3 显示了 D 锁存器是如何工作的, 图中假设输出 Q 初始为假, 并且 D 先改变。

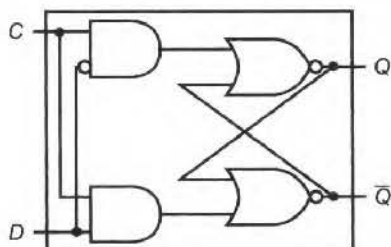


图 A-8-2 用或非门实现的 D 锁存器。如果其他的输入为 0, 则或非门作为反相器使用。因此, 交叉耦合的或非门存储状态值, 直到时钟输入 C 变为有效。此时, 输入 D 将替代 Q 并被存储。时钟信号 C 由有效变为无效时, 输入信号 D 必须保证稳定

① 原书为“拉低”, 应为笔误。——译者注

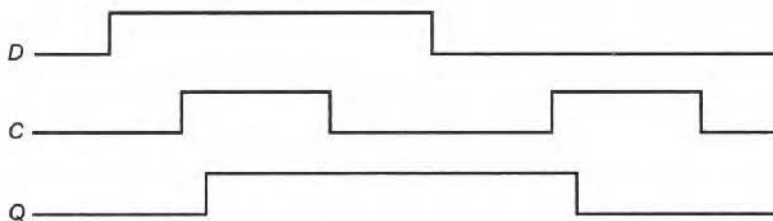


图 A-8-3 D 锁存器的操作，假设输出信号初始为无效。当时钟 C 有效时，锁存器被打开，输出信号 Q 的值立即变为输入 D 的值

正如前面提到的那样，我们使用触发器作为基本构造单元，而不是使用锁存器。触发器不是透明的：其输出只在时钟边沿发生变化。触发器可以设计成在时钟上升沿或下降沿触发，在本书的设计中我们可以使用任意一种类型。图 A-8-4 显示了如何用一对 D 锁存器来构造下降沿触发的 D 触发器。在 D 触发器中，输出在时钟边沿存储。图 A-8-5 显示了这个 D 触发器是如何操作的。

A-51

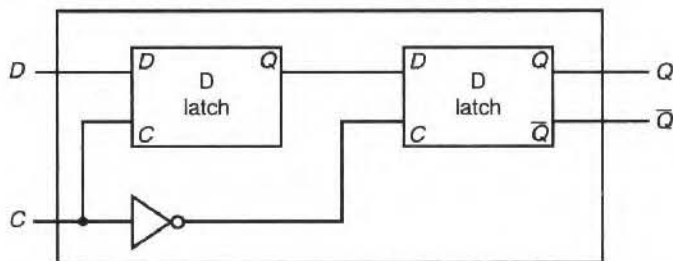


图 A-8-4 下降沿触发的 D 触发器。当输入时钟 C 有效时，第一个锁存器（称为主锁存器）打开，接受输入数据 D 。当输入时钟 C 变低时，主锁存器关闭，但第二个锁存器（称为从锁存器）打开，并从主锁存器的输出获取输入值

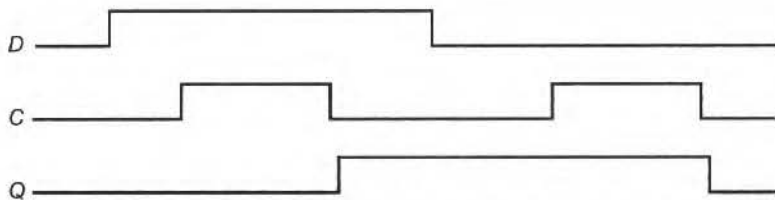


图 A-8-5 下降沿触发的 D 触发器的操作，假设其输出初始为无效。当时钟输入 C 从有效变为无效时，输出 Q 存储输入信号 D 的值。与图 A-8-3 中的时钟控制的 D 锁存器做比较。在带时钟的锁存器中，只要时钟 C 为高电平，存储的数据和输出 Q 就发生变化，与仅在 C 翻转时才发生变化相反

下面是一个上升沿触发的 D 触发器的 Verilog 代码，假设 C 为时钟输入， D 为数据输入。

```
module DFF(clock,D,Q,Qbar);
    input clock, D;
    output reg Q; // Q is a reg since it is assigned in an
always block
    output Qbar;
    assign Qbar=~ Q; // Qbar is always just the inverse
of Q
    always @(posedge clock) // perform actions whenever the
clock rises
        Q=D;
endmodule
```

由于输入 D 在时钟边沿被采样，因此在时钟边沿前后一段时间内， D 必须保持有效。

A-52

在时钟边沿前，输入信号必须保持有效的最短时间，称为**建立时间**（setup time）；在时钟边沿后，输入信号必须保持有效的最短时间，称为**保持时间**（hold time）。因此，任何触发器（或任何由触发器构造的结构）的输入必须在一个时间窗口内保持有效，这个时间窗口开始于时钟边沿前的 t_{setup} 时间，结束于时钟边沿后的 t_{hold} 时间，如图 A-8-6 所示。A.11 节中将更详细地介绍时钟和时序约束，包括触发器的传输延时。

建立时间：在时钟发生跳变前，输入信号必须保持有效的最短时间。

保持时间：在时钟发生跳变后，输入信号需要保持有效的最短时间。

我们可以使用 D 触发器的阵列来构建一个寄存器。寄存器可以存储多位数据，比如一个字节或一个字。在第 4 章中，我们在数据通路中使用了寄存器。

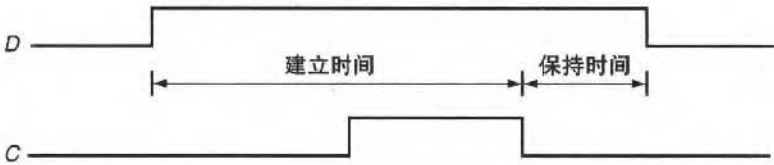


图 A-8-6 下降沿触发的 D 触发器的建立时间和保持时间。输入信号在时钟跳变前后的一段时间内保持有效。时钟跳变前，输入信号需要保持有效的最短时间称为建立时间；时钟跳变后，输入信号需要保持有效的最短时间称为保持时间。如果违反了最小建立时间和最小保持时间的要求，那么触发器的输出可能变得不可预测，如 A.11 节中将要描述的。保持时间要么为 0，要么是一个很小的值，因此不需要担心建立时间

A.8.2 寄存器文件

寄存器文件是数据通路中一个重要的核心结构。寄存器文件包含一组可读写的寄存器，通过寄存器号访问。通过一个由 D 触发器构成的寄存器阵列，并对每一个输入或输出端口添加译码器，就可以实现寄存器文件。由于读寄存器不会改变任何状态，因此我们只需提供寄存器号作为输入，输出为该寄存器号对应寄存器中的数据。对于写操作，我们需要三个输入：寄存器号、要写入的数据和一个控制写入的时钟。第 4 章中，我们使用了一个包含两个读端口和一个写端口的寄存器文件。该寄存器文件如图 A-8-7 所示。其中读端口可以通过一对多路选择器来实现，每一个多路选择器的位宽与寄存器文件中每个寄存器的位宽相等。图 A-8-8 给出了一个 64 位宽寄存器文件两个读端口的实现方法。

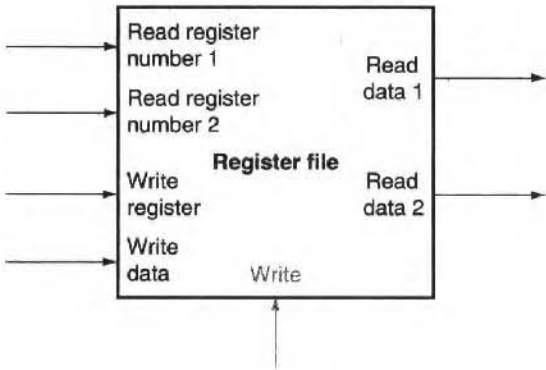


图 A-8-7 一个包含两个读端口和一个写端口的寄存器文件，该寄存器文件包含 5 个输入和 2 个输出。写控制信号（Write）用灰色表示

实现寄存器写端口稍复杂一点，因为我们只能更改指定寄存器的内容。为了达到这个目的，可以使用译码器来生成一个信号，用该信号来决定要对哪个寄存器进行写操作。图 A-8-9 显示了如何实现寄存器文件的写端口。需要注意的是，触发器的状态只在时钟边沿发生变化。

在第 4 章中，我们明确地为寄存器文件连接了写信号，并假设图 A-8-9 中的时钟是隐含连接的。

如果在一个时钟周期中，对寄存器同时进行读和写，将会发生什么？因为写寄存器文件出现在时钟边沿，因此在读操作时，寄存器是有效的，正如图 A-7-2 中所示。读出的数据将是上一个时钟周期写入的数据。如果我们想要读出当前正在写入的数据，则需要在寄存器文件内部或外部添加额外的逻辑。第 4 章广泛使用了这类寄存器。

A-53

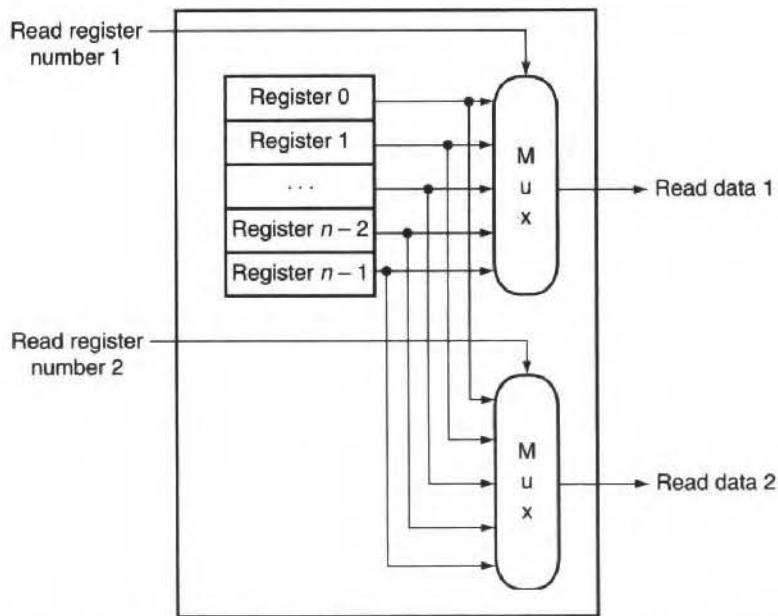


图 A-8-8 寄存器文件（含 n 个寄存器）的两个读端口，可以通过一对 n 选 1 多路选择器（每个 64 位宽）来实现。寄存器数据读信号用来作为多路选择器的选择信号。图 A-8-9 显示了如何实现写端口

A-54

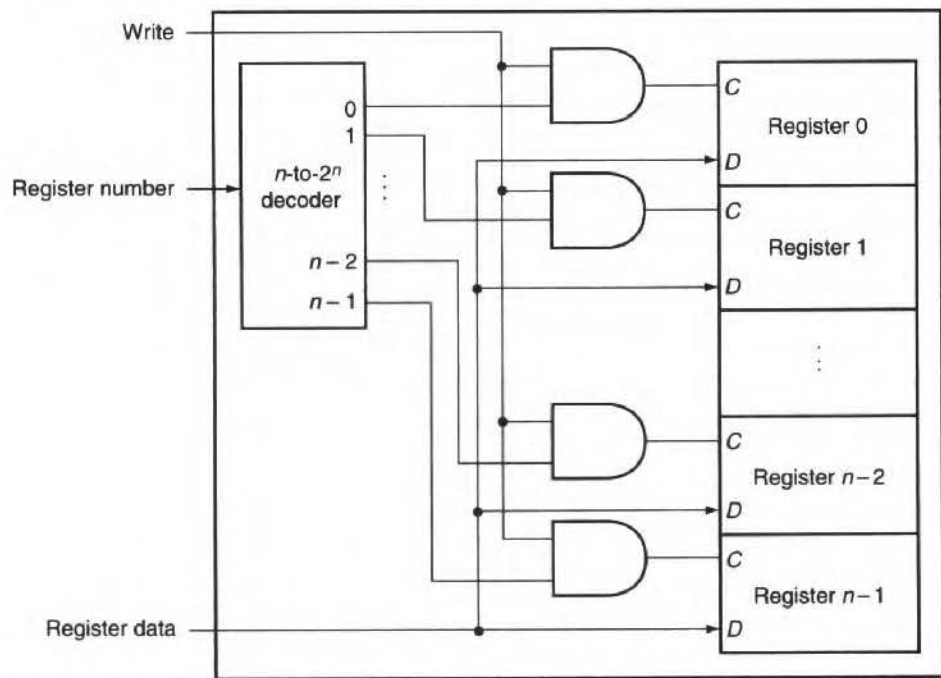


图 A-8-9 寄存器文件的写端口通过一个译码器来实现，译码器与写控制信号一起生成信号 C 并输入到寄存器中。所有三个输入（寄存器号、数据、写控制信号）都存在建立时间和保持时间的约束，以保证正确的数据被写到寄存器文件中

A.8.3 用 Verilog 描述时序逻辑

用 Verilog 来描述时序逻辑时，我们必须理解如何生成时钟，如何描述何时将数据写入寄存器中，及如何指定时序控制。我们先来描述时钟。时钟并不是 Verilog 中预定义的对象，我们可以在一条语句前使用符号 `#n` 来生成时钟，这将导致该语句在 n 个模拟时间单位之后被执行。在大多数 Verilog 模拟器中，也可以产生一个时钟作为外部输入，允许用户在模拟过程中，指定需要模拟器运行的时钟周期数。

图 A-8-10 中的代码实现了一个简单的时钟，该时钟的高电平和低电平都保持一个模拟时间单元，之后翻转状态。我们使用了延迟和阻塞赋值语句来实现时钟。

```
reg clock; // clock is a register
always
#1 clock = 1; #1 clock = 0;
```

图 A-8-10 时钟定义

A-55

接下来，我们定义边沿触发寄存器的操作。在 Verilog 中，这是通过使用 `always` 块的敏感信号列表实现的，并分别使用 `posedge` 或 `negedge` 来指定是上升沿触发还是下降沿触发。因此，下面的 Verilog 代码中，在时钟上升沿，寄存器 A 被写入值 b。

通过本章内容及第 4 章的 Verilog 部分，我们将给出一个上升沿触发的设计。图 A-8-11 显示了一个 LEGv8 寄存器文件的 Verilog 代码，代码中包含了两次读操作和一次写操作，其中只有写操作是受时钟控制的。

```
reg [63:0] A;
wire [63:0] b;

always @(posedge clock) A <= b;

module registerfile (Read1,Read2,WriteReg,WriteData,RegWrite,
Data1,Data2,clock);

input [5:0] Read1,Read2,WriteReg; // the register numbers
to read or write
input [63:0] WriteData; // data to write
input RegWrite, // the write control
clock; // the clock to trigger write
output [63:0] Data1, Data2; // the register values read
reg [63:0] RF [31:0]; // 32 registers each 32 bits long

assign Data1 = RF[Read1];
assign Data2 = RF[Read2];

always begin
// write the register with new value if Regwrite is
high
@(posedge clock) if (RegWrite) RF[WriteReg] <=
WriteData;
end
endmodule
```

图 A-8-11 用行为级 Verilog 描述的 LEGv8 寄存器文件。该寄存器文件在时钟上升沿进行写操作

A-56

小测验 图 A-8-11 寄存器文件的 Verilog 代码中，正在进行读操作的寄存器对应的输出端口使用连续赋值语句进行赋值。但被写入的寄存器在 `always` 块中被赋值。下面哪一项是其原因？

a. 没有特殊原因，只是为了方便。

- b. 因为 Data1 和 Data2 是输出端口，并且 WriteData 是输入端口。
- c. 因为读操作是一个组合事件，而写操作则是一个时序事件。

A.9 存储元件：SRAM 和 DRAM

寄存器和寄存器文件可以作为基本单元来构造小容量存储器。但大容量存储器由 **SRAM**（静态随机访问存储器）或是 **DRAM**（动态随机访问存储器）来构建。我们首先讨论比较简单的 SRAM，然后介绍 DRAM。

A.9.1 SRAM

SRAM 是很简单的集成电路，由存储阵列构成，通常包含一个访问端口，该端口提供读或写。SRAM 对任一单元的访问时间都是固定的，尽管读操作和写操作的特征不同。根据可寻址单元的数量和每个可寻址单元的位宽，SRAM 芯片有特定的配置。例如，一个 $4\text{M} \times 8$ 的 SRAM，可以提供 4M 的入口，每个入口 8 位宽。因此有 22 条地址线（ $4\text{M} = 2^{22}$ ）、8 位宽的输出数据线和 8 位宽的输入数据线。与 ROM 类似，可寻址单元的数量通常称为高度，每个可寻址单元的位数称为宽度。由于多种技术原因，最新最快的 SRAM 常常使用较窄的配置： $\times 1$ 和 $\times 4$ 。图 A-9-1 显示了一个 $2\text{M} \times 16$ 的 SRAM 的输入和输出信号。

SRAM：一种存储器，其中数据是静态存储的（如触发器），而不是动态存储的（如 DRAM）。SRAM 比 DRAM 快，但是密度较小，每位的价格更高。

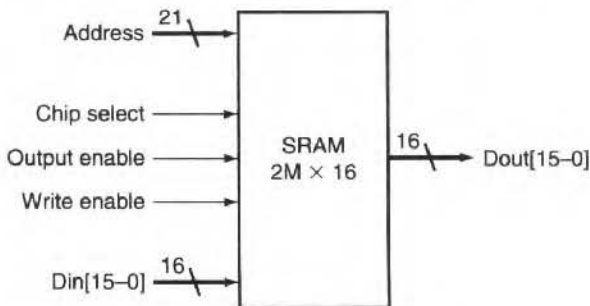


图 A-9-1 一个 $32\text{K} \times 8$ 的 SRAM，包含 21 位地址线（ $32\text{K} = 2^{15}$ ）和 16 位数据输入线，3 条控制线和 16 位数据输出线

A-57

为了启动读写操作，片选信号（Chip select）必须处于有效状态。对于读操作，必须激活活输出使能信号（Output enable），该信号用来控制被地址选中的数据能否驱动到引脚上。输出使能信号允许多个存储器连接到单输出总线上，并且用于决定由哪个存储器来驱动总线。SRAM 读取数据所需的时间通常被定义为从输出使能信号和地址线有效一直到数据输出为止的延迟。2004 年，最快的 CMOS 部件的 SRAM 的读取时间为 $2 \sim 4\text{ns}$ ，但这样的 SRAM 容量较小，数据宽度较窄，更大部件（2004 年时已达到 32M 位数据以上）的 SRAM 的读取时间为 $8 \sim 20\text{ns}$ 。过去 5 年间，消费类产品和数码设备对低功耗 SRAM 的需求极大增长，这些 SRAM 通常具有更低的待机和访问功耗，但速度通常慢了 $5 \sim 10$ 倍。最近，类似同步 DRAM（下一节讨论）的同步 SRAM 也开发出来了。

对于写操作，必须提供待写入的数据和地址，以及写控制信号。当写使能信号（Write enable）和片选信号为真时，数据输入线上的值就被写入地址线指定的存储单元。正如 D 触发器和 D 锁存器，SRAM 的地址线和数据线上的信号也有建立时间和保持时间的要求。同时，写使能信号不是时钟边沿触发的，而是有最小宽度要求的脉冲。写操作完成时间由建立时间、保持时间以及写使能脉冲宽度所共同决定。

大容量 SRAM 不能通过构件寄存器文件的方式实现。寄存器文件中采用的 32-1 多路选择器是切实可行的, 但为 $64K \times 1$ SRAM 采用 $64K-1$ 多路选择器是不切实际的。大容量存储器不采用多路选择器, 而是通过一条共享的输出信号线来完成, 该输出信号线也称为位线, 可被存储阵列的多个存储单元设置。为了满足多个存储单元驱动一条信号线, 需要用到三态缓冲器。三态缓冲器有两个输入, 即数据信号和输出使能信号, 还有一个输出信号, 输出信号有三种状态, 分别为有效、无效或高阻。如果输出使能信号有效, 那么三态缓冲的输出为数据输入信号的值 (无论是否有效)。否则, 其输出为高阻态, 这时将由其他的输出使能有效的三态缓冲器决定共享输出的值。

图 A-9-2 描述了一个由一组三态缓冲器和译码后输入构成的多路选择器。该电路的关键在于, 任意时刻至多允许一个缓冲器的输出使能有效; 否则, 三态缓冲器将会发生输出线竞争现象。在 SRAM 中, 每个存储单元通过使用三态缓冲器, 就能实现存储单元对输出信号线的共享。采用分布式的三态缓冲器比大规模集中式的多路选择器效率更高。三态缓冲器通常被合并到组成 SRAM 基本单元的触发器中。图 A-9-3 描述了一个小容量 4×2 SRAM 的实现, 其中用到了带有使能输入的 D 锁存器来控制三态输出。

图 A-9-3 中没用到多路选择器, 但用到了一个非常大的译码器和大量的字线。例如, 在一个 $4M \times 8$ 的 SRAM 中, 我们需要 $22-4M$ 的译码器, 以及 $4M$ 条字线 (用于各触发器的使能)! 为解决这个问题, 大容量的存储器被组织成矩形阵列, 并且使用了二级译码。图 A-9-4 显示了一个 $4M \times 8$ 型 SRAM 是如何利用二级译码来实现的。我们可以看到, 二级译码对于理解 DRAM 如何运作非常重要。

同步 SRAM (SSRAM) 和同步 DRAM (SDRAM) 近年来都在发展。同步 RAM 的关键优点在于其能将存储阵列或存储行中一系列顺序地址中的数据以突发 (burst) 方式传输。突发传输要定义一个起始地址和突发传输长度。同步 RAM 的速度优势在于, 在突发方式传输时, 无需指定额外的地址位。但在传输突发中的连续位时需要使用时钟。突发传输模式下, 省去指定地址的开销将大大增加数据块的传输速率。正因为这个优点, 同步 SRAM 和同步 DRAM 迅速成为计算机内存系统的首选 RAM。下一节和第 5 章中将更详细地讨论存储系统中同步 DRAM 的使用。

A.9.2 DRAM

在静态 RAM (SRAM) 中, 一个单元中的数据保存在一对反相门电路中, 只要持续供电, 数据就可以一直保持下去。而在动态 RAM (DRAM) 中, 一个单元中的数据是以电荷量的形式被保存在电容中, 通过一个晶体管来访问存储的电荷, 即读取值或者重写存储的电荷。由于动态 RAM 中, 每一位存储位只使用一个晶体管, 因此其密度更高, 单位价格更低。

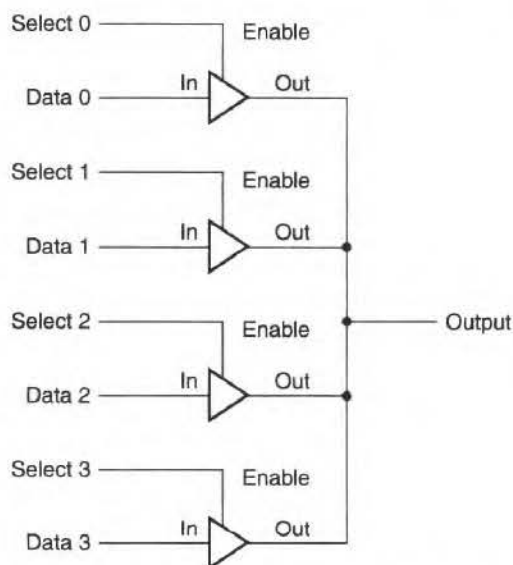


图 A-9-2 用 4 个三态缓冲器实现多路选择器。4 个选择输入信号 (Select) 中只能有一个有效。输出使能无效时, 三态缓冲器输出为高阻态, 允许另一个输出使能有效的三态缓冲驱动共享输出线

A-58

A-59

相比而言, SRAM 每位需要 4 ~ 6 个晶体管。由于 DRAM 将电荷存储在电容中, 因此不能永久保存, 需要不断地刷新。这就是该存储结构称作动态的原因, 与 SRAM 单元的静态存储相反。

为了进行刷新, 我们需要读出该内容并且回写到原单元中。电荷量通常能维持几毫秒, 可能相当于近 100 万个时钟周期。目前, 单芯片存储控制器通常能独立于处理器完成刷新功能。如果 DRAM 的每一位都要单独读出并回写, 那么对于包含几兆字节的大容量存储器, 我们必须持续刷新 DRAM, 而没有时间对数据进行存取。幸好 DRAM 中也采用了二级译码

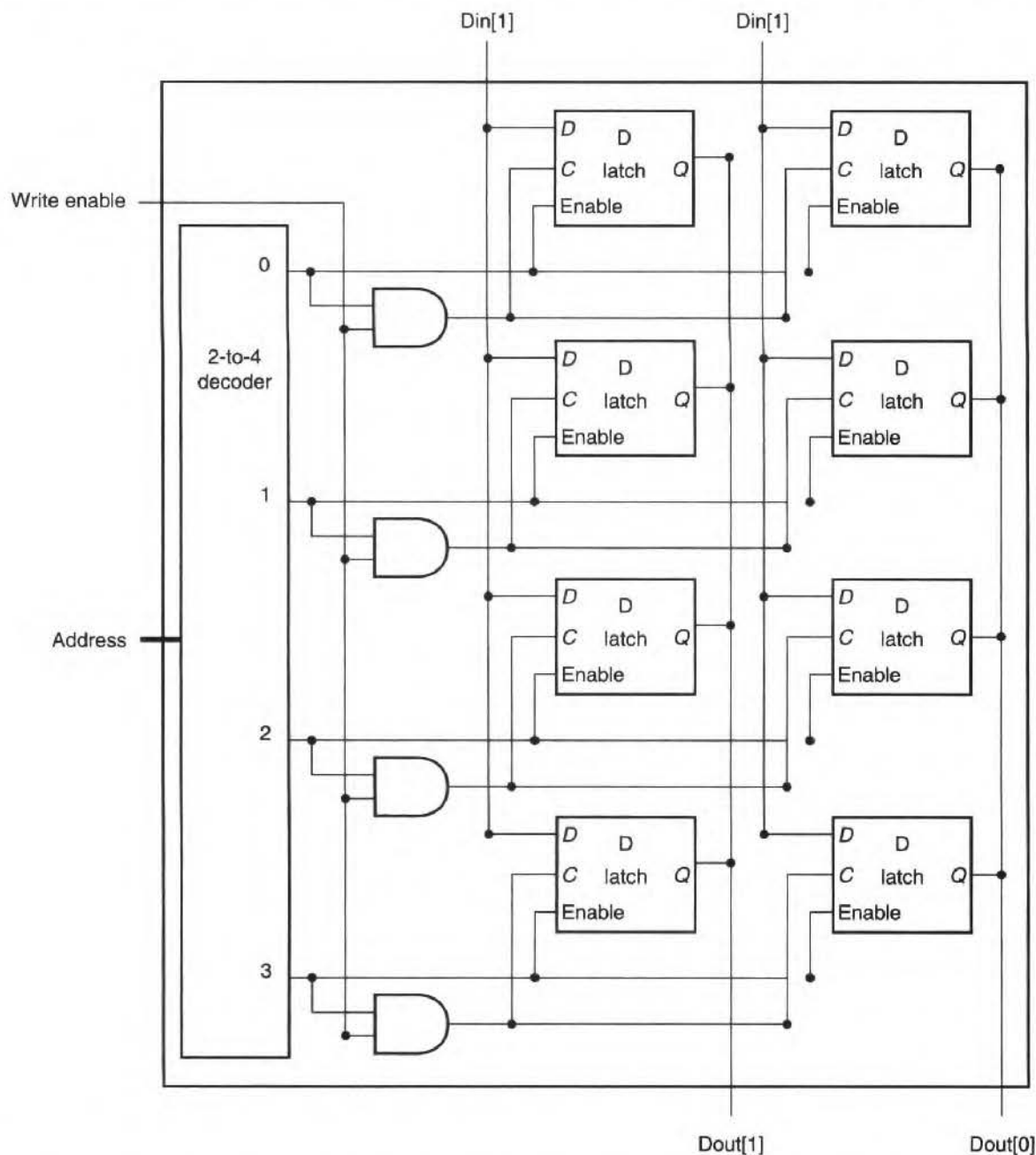


图 A-9-3 4×2 SRAM 的基本结构, 其中译码器用以选择哪一对单元有效。被激活的单元采用三态输出连接到垂直位线 (提供所要求的数据) 上。单元地址信息则通过水平地址线中的某一条 (称为字线) 来传送。为简单起见, 此处省略了输出使能信号和片选信号, 但可以简单地通过与门加入

结构，这就可以在读周期内刷新一整行（一行共享一条字线）。读周期后紧跟一个写周期。通常，刷新工作只占了 DRAM 1% ~ 2% 的活跃周期，剩下 98% ~ 99% 的时间可以用来处理数据的读写。

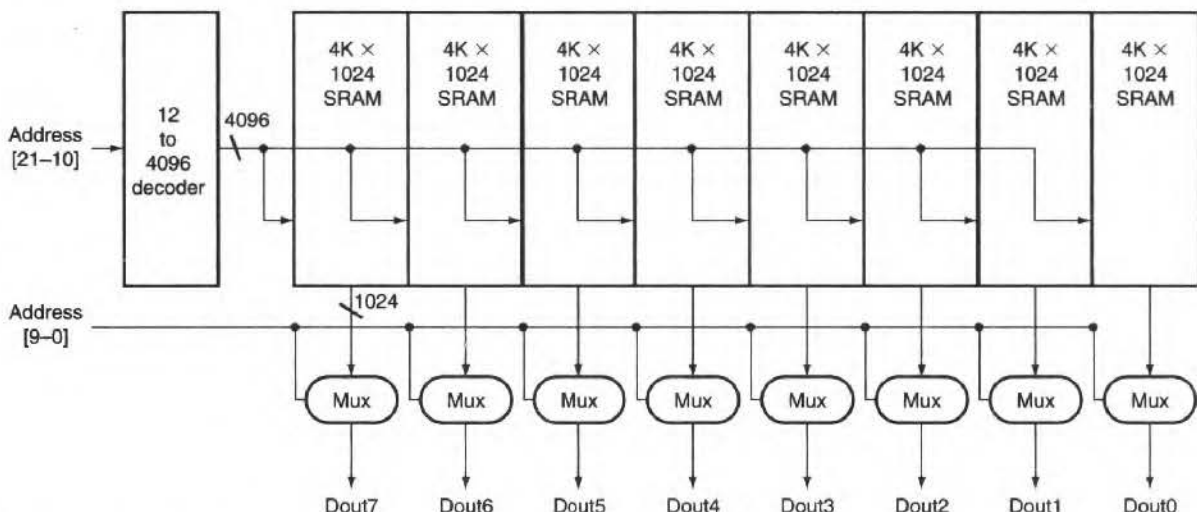


图 A-9-4 4M × 8 SRAM 的典型组织结构（4K × 1024 阵列）。第一个译码器产生 8 个 4K × 1024 阵列的地址，之后的一组多路选择器从每个 1024 位宽的阵列中选出 1 位。该设计远比单级译码器简单，单级译码器需要一个庞大的译码器或者多路选择器。实际上，现在这个大小的 SRAM 可能使用更多数量的模块，并且每一块都会更小

精解 动态 RAM 如何读 / 写单元中存储的信息？单元内的晶体管实际是一个开关，称为传输晶体管，允许存放在电容中的电荷被访问（读或写）。图 A-5-9 显示了单晶体管存储单元的外观。传输晶体管的作用类似于开关：当字线上的信号有效时，开关关闭，将电容连到位线上。如果是写操作，待写的数据就被放到位线上。如果值为 1，则电容被充电；否则电容放电。由于 DRAM 必须能检测到电容中很少的电量，所以读操作略为复杂。通常，在激活字线准备读出数据前，位线先被充电到一半状态。然后，通过激活字线，电容上的电荷可被读出到位线。这导致位线稍微向高电平或低电平方向移动，并且这种变化能通过灵敏放大器（能够检测电平上很小的变化）进行检测。

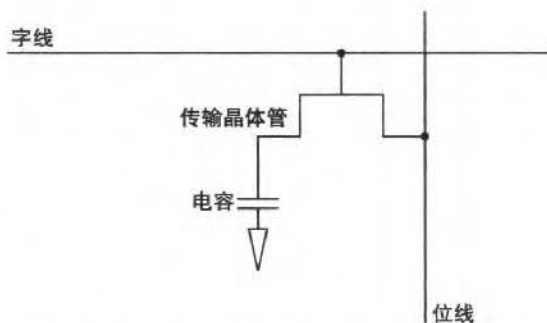


图 A-9-5 单晶体管实现的 DRAM 单元，包含一个用于存储单元内容的电容，以及一个用于读写的晶体管

DRAM 有一个二级译码器，包含一个行访问和一个列访问，如图 A-9-6 所示。其中行访问选中一行，并激活对应的字线。被激活的行中所有列的内容被保存到一组锁存器中。然后，列访问从列锁存器中选取相应的数据。为了节省引脚并进一步减少封装开销，行地址 / 列地址将共享地址线。由一对信号线 RAS（Row Access Strobe，行访问选通脉冲）和 CAS（Column Access Strobe，列访问选通脉冲）来表明是行还是列地址。刷新过程只是简单地将列信息读入列锁存器，然后再写回相同的值。因此，一个周期之内就可以刷新一整行。二级寻址方法再加上中间转换电路，导致 DRAM 的访问时间比 SRAM 的访问时间长很多（5 ~ 10 倍）。2004 年，

典型的 DRAM 访问时间为 45 ~ 65ns, 256Mb 的 DRAM 已量产, 2004 年第一季度第一个 1GB 的 DRAM 样品也生产出来了。单位比特更低的成本使得 DRAM 成为主存的首选, 而更快的访问时间使得 SRAM 成为 cache 的首选。

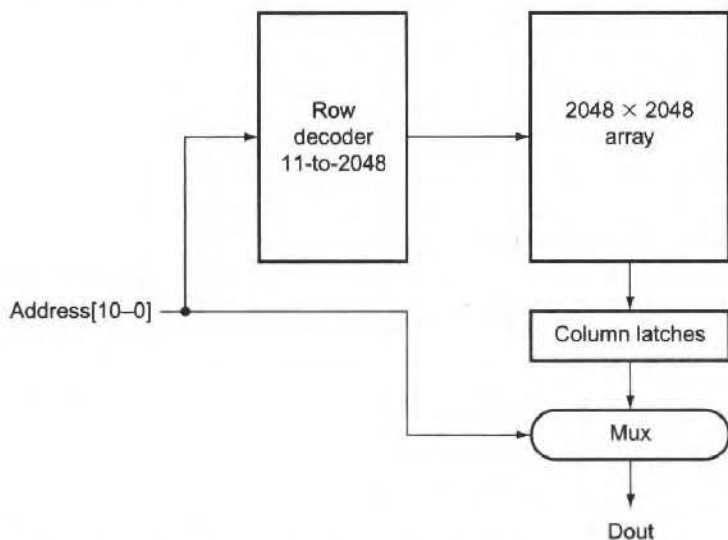


图 A-9-6 用 2048 × 2048 阵列组成 4M × 1 DRAM。行访问采用 11 位选择一行, 然后锁存到 2048 个 1 位锁存器中。多路选择器从 2048 个锁存器中选择输出。RAS 和 CAS 信号则分别控制地址线是送给行译码器还是列多路选择器

你可能会注意到, 64M × 4 的 DRAM 在每次行访问时, 实际能访问 8K 位, 然后在列访问时就丢弃了几乎所有位, 只剩下 4 位。DRAM 设计者早已使用 DRAM 的内部结构, 从 DRAM 中提供更宽的带宽。这通过允许列地址变化而行地址不变实现, 这导致对列锁存器中其他位的访问。为了使这个过程更快更精确, 地址输入被时钟控制, 这样便产生了目前 DRAM 的主要使用形式: 同步 DRAM 或称为 SDRAM。

1999 年以来, SDRAM 成为大多数基于 cache 的主存系统的首选存储芯片。SDRAMS 通过在时钟信号的控制下顺序传送突发传输中的所有位, 提供对行内的一系列位的快速访问。2004 年, DDRRAM (Double Data Rate RAM, 双倍数据传输率 RAM) 是使用最多的 SDRAM 类型。DDRRAM 之所以称为双倍数据传输率, 是因为在外部时钟的上升沿和下降沿都能传输数据。如第 5 章所讨论的, 这些高速传输可用于提高主存储器中可用的带宽, 以满足处理器和高速缓存的需要。

A.9.3 纠错

因为在大容量存储器中可能发生数据损坏的情况, 故而大多数计算机系统都会采用各种校验码来检测可能的数据错误。一种简单并且常用的校验码是奇偶校验码。在奇偶校验码中, 数据中 1 的个数为奇数, 则属于奇校验, 否则属于偶校验。当数据往内存写入时, 校验位也被写入 (奇校验 1, 偶校验 0)。当数据被读出时, 校验位也被读出并校验。如果所存储的数据的奇偶性和读出的校验位不一致时, 说明数据出错。

1 位奇偶校验能检测出最多 1 位错。如果有 2 位错, 那么 1 位校验法就可能无法检测到错误, 因为校验位正好会与两个错误匹配。(实际上, 1 位校验法能测出任何奇数位错。但 3

位错的概率要比 2 位错的概率小得多，所以 1 位校验码通常只用来检测 1 位错。)当然，奇偶校验无法判断数据中哪位出错。

检错码：能够检查数据是否出错，但无法确定错误位置并纠正错误的校验码。

1 位奇偶校验是一种**检错码** (error detection code)。而**纠错码** (Error Correction Code, ECC) 既能检测错误，还能对错误进行纠正。对于大容量主存，许多系统采用的纠错码不仅能检测到 2 位以内出错的情况，并且还能纠正 1 位错。这些校验码使用更多的位来编码数据，例如，主存采用的典型的编码每 128 位数据需要 7 或 8 位校验码。

精解 1 位奇偶校验码是距离为 2 的编码方法，这就是说，对于数据和校验位而言，任何 1 位数字的改变都会被检测出该数据出错。例如，当改变某个数据位时，校验位就出错；反之亦然。当然，如果我们同时改变两位 (2 个数据位，或 1 个数据位和 1 个校验位)，那么校验位同数据依旧匹配，也就无法检测出错误了。因此，在奇偶校验和数据的合法组合间，距离为 2。

为了能检测出多于 1 个的错误或纠正 1 个错误，我们需要距离为 3 的校验码。也就是说，纠错码和数据位的任何组合至少需要有 3 位与其他组合不同。假设存在这样的校验码，并且数据中有 1 位出错。这时，校验码加上数据就有 1 位和合法组合不同，我们就能检测到并纠正；如果有两位出错，我们能检测到错误的发生，但无法纠正。下面参考一个例子，表中为一个 4 位数据项数据字和距离为 3 的纠错码。

数据	校验位	数据	校验位
0000	000	1000	111
0001	011	1001	100
0010	101	1010	010
0011	110	1011	001
0100	110	1100	001
0101	101	1101	010
0110	011	1110	100
0111	000	1111	111

A-65

为了说明校验过程，我们不妨选择一个数据为例。如 0110，其纠错码是 011。该数据发生一位错的情况有 4 种：1110，0010，0100，0111。下面观察具有相同校验码 011 的数据项，即数据 0001 所在的表项。如果纠错译码器收到某个含有 1 位错的数据 (四个可能错误的错误数据之一)，那么需要选择纠正为 0110 或是 0001。这四个错误的数据和正确的 0110 只有 1 位不同，每个相对于 0001 有 2 位不同。因此，该校验码可以很容易地选择纠正为 0110，因为 1 位错的概率高得多。考察 2 位错能被检测出来的情况，注意两位错的所有组合都有不同的编码。重用相同编码，码字中要有 3 位不同。如果纠正 2 位错误，就会得到错误的结果，因为该纠正机制将假设仅产生了 1 位错。如果我们希望纠正 1 位错，并检测 2 位错，那么需要一个距离为 4 的校验码。

虽然我们将数据和校验码区分了出来，但事实上，纠错码把编码和数据看作一个更长的字 (例子中是 7 位)。因此，编码和数据中的错误是被等同对待的。

尽管上例中的 n 位数据需 $n - 1$ 位校验码，但随着数据倍数的增加，校验的位数增长较慢。例如，在距离为 3 的校验码中，64 位数据只需 7 位校验码，128 位数据只需 8 位校验码就能实现。这种校验码叫汉明码，以 R. Hamming 命名，他首先提出了该校验码的编码方法。

A.10 有限状态机

数字逻辑电路可分为组合逻辑电路和时序逻辑电路。时序系统的状态存放在存储器中，其行为不仅依赖于输入信号，同时也与存储器中的内容和系统状态有关。因此，时序系统无法用真值表加以描述，而可以用有限状态机（finite-state machine，也称为状态机）来描述。有限状态机有一组状态量和两个函数，称为输出函数和下一状态函数（next-state function）。状态集合对应于内部存储器中所有可能的值。因此，对于 n 位存储器，有 2^n 个状态量。下一状态函数是一种组合逻辑函数，给定输入和当前状态，就能确定系统的下一状态。输出函数根据当前状态和输入产生一组输出。图 A-10-1 是有限状态机的图示。

有限状态机：一个包含一组输入和输出、一个下一状态函数（将当前状态和输入映射为一个新状态）、一个输出函数（将当前状态和可能的输入映射为有效的输出）的时序逻辑函数。

下一状态函数：一个组合逻辑函数，根据输入和当前状态确定有限状态机的下一状态。

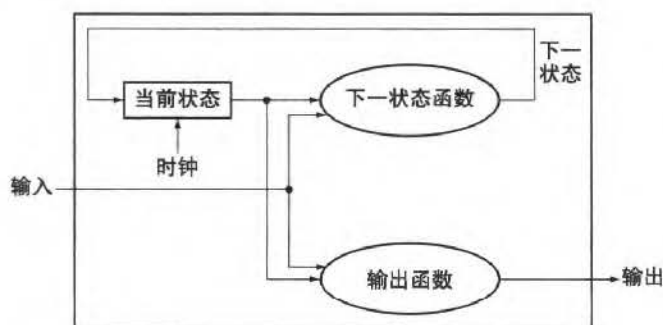


图 A-10-1 状态机由包含状态的内部存储单元和两个组合逻辑函数构成：下一状态函数和输出函数。通常，输出函数被限制为只将当前状态作为输入，这样就不会改变时序机的能力，但会对内部造成影响

本节和第 4 章讨论的状态机都是同步的。也就是说，状态随时钟周期变化，并且每个时钟周期都会计算新状态。因此，状态元件仅在时钟边沿更新。本节和整个第 4 章都使用了该方法，但我们通常不显式地显示出时钟。在第 4 章中，我们使用了状态机来控制处理器的执行以及数据通路的操作。

A-66

为了说明有限状态机如何操作和设计，我们引用简单经典的“交通灯”实例加以说明（第 4、5 章的例子中利用有限状态机来控制处理器的执行过程）。当有限状态机用作控制器时，输出函数通常被限制为仅仅依赖于当前状态。这样的有限状态机称作摩尔机（Moore machine）。在本书中，我们都采用了这种有限状态机。如果输出函数既依赖于当前输入，也依赖于当前状态，这样的状态机称为米利机（Mealy machine）。这两种状态机在功能上是等价的，二者可以互相转化。摩尔机的优点是相对更快，而米利机则结构更小，因为其比摩尔机所需要的状态数更少。第 5 章对其差别已进行了更详细的讨论，并给出了米利机的 Verilog 描述。

假设在东西大道和南北大街相交的十字路口有一个交通灯需要控制。为简单起见，这里只考虑红灯和绿灯（练习题中有加上黄灯的逻辑设计）。我们希望每个方向上灯切换的周期 ≤ 30 秒。因此采用了频率为 0.033Hz 的时钟信号，这样就能保证机器在状态间循环的周期 ≤ 30 秒。其中有两个输出信号：

- NSlite：当该信号有效时，南北方向的交通灯为绿色；反之为红色。
- EWlite：当该信号有效时，东西方向的交通灯为绿色，反之为红色。

此外，还有两个输入：

- NSCar：说明有汽车在探测器处，该探测器放置在南北向道路交通灯前方的路基上。

A-67

- EWcar: 说明有汽车在探测器处, 该探测器放置在东西向道路交通灯前方的路基上。

只有当其他方向有汽车在等待时, 交通灯才会在红绿灯之间切换; 否则, 交通灯的状态保持为绿色, 直到该方向上所有汽车都顺利通过为止。

为了实现这一简单的交通灯, 我们需要两个状态量:

- NSgreen: 南北方向的交通灯为绿色。
- EWgreen: 东西方向的交通灯为绿色。

下面, 我们建立一个下一状态函数表:

	输入		
	NScar	EWcar	下一状态
NSgreen	0	0	NSgreen
NSgreen	0	1	EWgreen
NSgreen	1	0	NSgreen
NSgreen	1	1	EWgreen
EWgreen	0	0	EWgreen
EWgreen	0	1	EWgreen
EWgreen	1	0	NSgreen
EWgreen	1	1	NSgreen

注意, 我们没有在算法中指定当两个方向同时有汽车通行时该怎么办。如果出现这样的情况, 上面的下一状态函数需要进行修改, 以保证一个方向的车流不会堵塞另一方向的交通。

然后, 有限状态机可通过指定输出函数完成。

	输出	
	NSlite	EWlite
NSgreen	1	0
EWgreen	0	1

在考察如何实现这个有限状态机之前, 我们先来看一个有限状态机的图形表示。在图解中, 节点表示状态, 节点中放置的是该状态下有效的一些输出。有向弧用于指出下一状态函数值, 弧上的标记指定输入作为逻辑函数。图 A-10-2 给出了该有限状态机的图形表示。

A-68

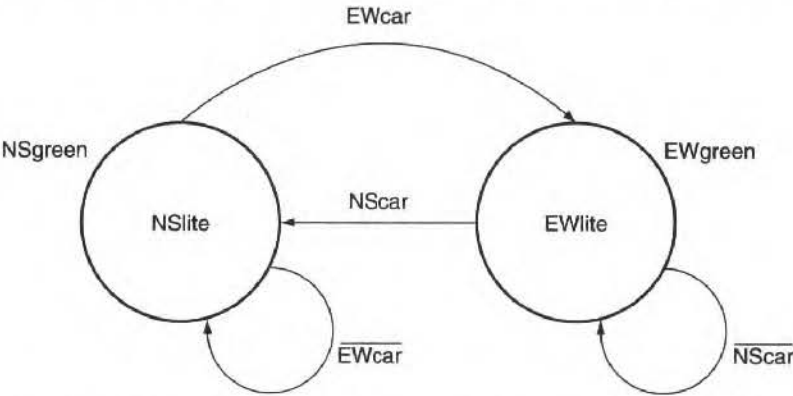


图 A-10-2 两个状态的交通灯控制器的图形表示。其中简化了状态传输的逻辑函数。例如, 状态表中 NSgreen 到 EWgreen 的转换是 $(\overline{\text{NScar}} \cdot \text{EWcar}) + (\text{NScar} \cdot \text{EWcar})$, 与 EWcar 相等

有限状态机可这样实现: 由寄存器保持当前状态, 由组合电路计算出下一状态函数和输出函数。图 A-10-3 描述了一个状态为 4 位 (16 种状态) 的有限状态机的框图。在实现有限状

态机之前，我们先将每个状态编号，该过程称为状态分配。例如，我们将 NSgreen 记为状态 0，EWgreen 记为状态 1。状态寄存器则只有 1 位。下一状态函数可由以下公式计算：

$$\text{NextState} = (\overline{\text{CurrentState}} \cdot \text{EWcar}) + (\text{CurrentState} \cdot \overline{\text{NScar}})$$

其中 CurrentState（当前状态）是状态寄存器的内容（0 或 1）。NextState（下一状态）是下一状态函数的输出，将在时钟周期结束时被写入状态寄存器。输出函数也很简单：

$$\begin{aligned}\text{NSlite} &= \overline{\text{CurrentState}} \\ \text{EWlite} &= \text{CurrentState}\end{aligned}$$

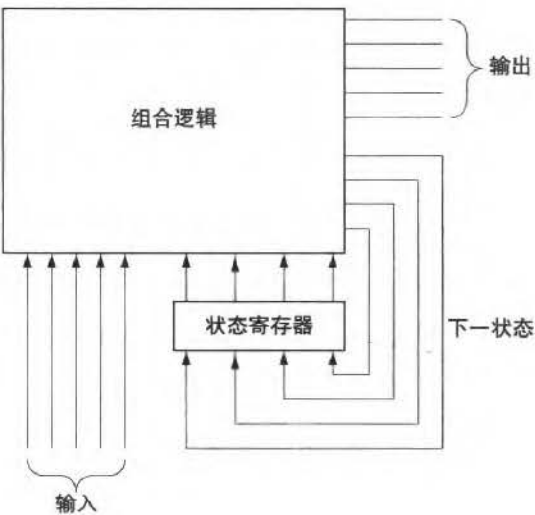


图 A-10-3 有限状态机的实现，包括一个状态寄存器用以保存当前状态，一个组合逻辑用以计算下一状态和输出函数。后两个功能通常采用两个独立的逻辑块分开实现，这样可能需要更少的门电路

组合逻辑通常采用结构化的逻辑电路实现，例如采用 PLA。PLA 可以根据下一状态表和输出函数表自动构建。CAD（Computer-Aided Design，计算机辅助设计）工具可以将图形化或文本化表示的有限状态机自动实现并优化为电路设计。在第 4、5 章中，有限自动机用于控制处理器的执行。附录 C 详细讨论了用 PLA 和 ROM 来实现这些控制。

A-69

为了说明如何使用 Verilog 写出控制逻辑，图 A-10-4 给出了一个用于综合的 Verilog 版

```
module TrafficLite (EWCar, NSCar, EWLite, NSLite, clock);
    input EWCar, NSCar, clock;
    output EWLite, NSLite;
    reg state;
    initial state=0; //set initial state
    //following two assignments set the output, which is based
    //only on the state variable
    assign NSLite = ~ state; //NSLite on if state = 0;
    assign EWLite = state; //EWLite on if state = 1
    always @(posedge clock) // all state updates on a positive
    clock edge
        case (state)
            0: state = EWCar; //change state only if EWCar
            1: state = NSCar; //change state only if NSCar
        endcase
    endmodule
```

图 A-10-4 交通灯控制器的 Verilog 描述

本。注意，对于这个简单的控制功能，米利机没有用处，但第 5 章中为实现控制功能采用的是米利机，比摩尔机控制器所需的状态更少。

A-70

小测验 要满足米利机所需的状态数比摩尔机所需的状态数少这个条件，摩尔机最少的状态数是多少？

- 2，因为 1 个状态的米利机有可能做相同的事情。
- 3，因为可以构造一个简单的摩尔机，跳转到两个不同状态之一，并且在此之后总是返回先前状态。对于这种简单的机器，2 个状态的米利机是可能的。
- 需要至少 4 个状态，才能体现出米利机相对摩尔机的优越性。

A.11 时序方法

本附录以及本书的剩余部分中，我们采用边沿触发的时序方法^①。边沿触发方法比电平触发方法更易于解释和理解。本节将对时序方法进行较为详细的阐述，同时也会介绍有关电平触发的内容。本节末将简单地讨论异步信号和同步信号，这是数字设计中的一个重要问题。

A-71

本节的主要目的是介绍时序方法的主要概念。为了简化，本节做了一些重要的假设；如果你想要进一步了解时序方法，可参阅附录末的参考文献。

我们采用边沿触发方法是因为其有两个优点：易于描述，应用简单。如果我们假设所有时钟都同时到达，那么只要简单地让时钟周期足够长，就能保证在组合逻辑块之间具有边缘触发寄存器的系统可以正确工作，而不会发生竞争现象。当状态元素的内容取决于不同逻辑元素的相对速度时，就会发生竞争。

在边沿触发的电路设计中，时钟周期必须足够长，这样才能满足从一个触发器通过组合逻辑到另一个触发器的路径，满足建立时间的要求。图 A-11-1 描述了采用上升沿触发的系统所需的条件。在这种系统中，时钟周期必须至少达到

$$t_{\text{prop}} + t_{\text{combinational}} + t_{\text{setup}}$$

三个延迟分量的最坏情况。三个延迟分别定义如下：

- t_{prop} 是信号通过触发器传输的时间，有时也称为 clock-to-Q。
- $t_{\text{combinational}}$ 是一个组合逻辑（两端为两个触发器）的最长延时。
- t_{setup} 是时钟上升沿到来前，触发器输入必须有效的的时间。

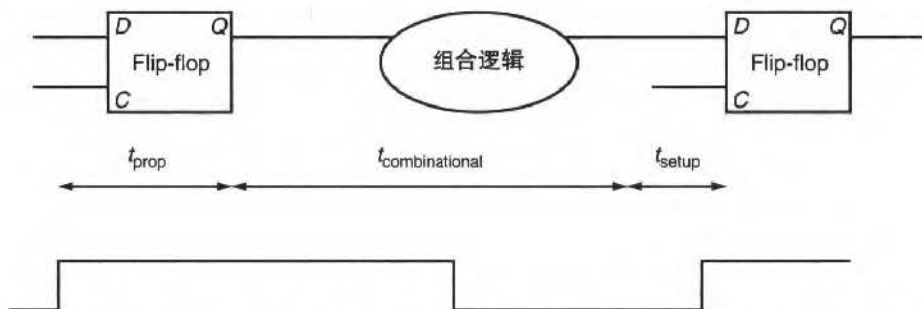


图 A-11-1 在边沿触发的设计中，时钟周期必须足够长，以保证在下一个时钟沿到来之前信号在要求的建立时间内有效。信号从触发器输入端传输到触发器输出端的时间为 t_{prop} ，然后经过 $t_{\text{combinational}}$ 的时间通过组合逻辑，并在下一个时钟沿到来之前 t_{setup} 时刻有效

A-72

① 也称时序策略。——译者注

下面做一个简化假设：保持时间要求已满足。在现代逻辑设计中，保持时间已是几乎不需要考虑的问题。

在边沿触发的设计中还必须考虑的一个复杂问题是**时钟偏移**（clock skew）。时钟偏移是指两个状态单元看到同一个时钟沿时的绝对时间差。时钟偏移产生的原因是时钟信号经常使用两条不同的路径到达两个不同的状态单元，因而延迟略有不同。如果时钟偏移足够大，那么一个状态单元可能变化并导致另一个触发器的输入端在第二个触发器看到时钟沿之前就发生了改变。

时钟偏斜：两个状态单元看到一个时钟沿的绝对时间差。

图 A-11-2 显示了该问题，其中忽略了建立时间和触发器的传输延迟。为避免这种错误，可以增加时钟周期以容忍最大的时钟偏移。因此，时钟周期至少应大于

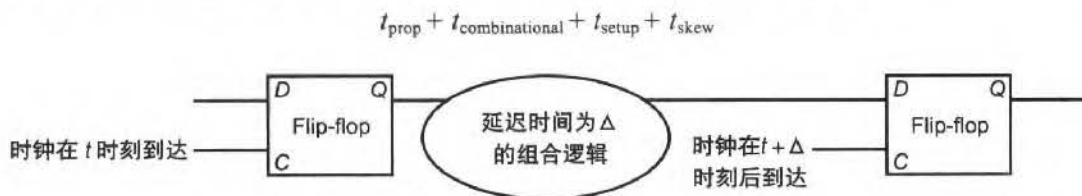


图 A-11-2 时钟偏移如何引发竞争，导致错误的操作。由于两个触发器看到时钟信号的时刻存在差异，存储在第一个触发器中的信号可能会向前竞争传输，并在时钟到达第二个触发器之前改变第二个触发器的输入值

有了这一时钟周期约束，即使两个时钟到达的先后次序颠倒，即第二个时钟早到了 T_{skew} ，电路依旧能正常工作。设计人员为减少时钟偏移，需要仔细布局时钟信号，将到达时间的差异减到最小。另外，聪明的设计师还提供了一些余量，使时钟比最小的时间稍长一点；这允许组件和电源的变化。由于时钟偏移也会影响保持时间约束，最小化偏移量是非常重要的。

边沿触发有两个缺点：需要额外的逻辑电路，有时会更慢。比较 D 触发器和电平敏感锁存器，我们会发现边沿触发的设计需要更多的逻辑。另一种方法是采用**电平敏感时钟**（level-sensitive clocking）。因为状态改变是电平敏感的而不是瞬时的，电平敏感机制要稍微复杂一些，并且需要考虑更多因素以使其正确执行。

电平触发时钟控制：一种时序方法，状态变化发生在时钟为高电平或低电平期间，与边沿触发设计中变化的瞬时发生不同。

A-73

A.11.1 电平敏感时序

电平敏感时序中，状态改变发生在高电平或低电平期间，与边沿触发方法下的瞬时改变不同。由于状态不是瞬时变化的，因此竞争现象很容易产生。为了保证在时钟足够慢的情况下，电平敏感的设计仍能正常工作，设计人员使用了一种双相时钟。双相时钟使用了两个不相重叠的时钟信号（ ϕ_1, ϕ_2 ）。因此任一给定时刻，最多只有一个时钟信号为高电平，如图 A-11-3 所示。我们可以采用双相时钟构建系统，这样的系统包含电平敏感的锁存器，但没有竞争现象，效果与边沿触发电路一样。

设计这种系统的一种简单的方法是交替使用在 ϕ_1 打开的锁存器和在 ϕ_2 打开的锁存器。因为两个时钟不是同时处于有效状态，因此不可能产生竞争。如果组合电路的输入为 ϕ_1 ，那么其输出将被 ϕ_2 时钟锁存。当输入锁存器关闭，并因此输出有效时，其输出值只能在 ϕ_2 的有效信号期间开放，于是能保证输出信号有效。图 A-11-4 显示了一个具有双相时序以及交替锁存器的系统是如何工作的。正如边沿触发的设计中那样，必须注意时钟偏移，尤其是

A-74

在两个时钟相位之间。通过增加两个相位之间非重叠的数量，我们可以减少潜在的误差幅度。因此，如果每个相位足够长，并且相位之间的非重叠部分足够大，那么系统可以保证正确运行。

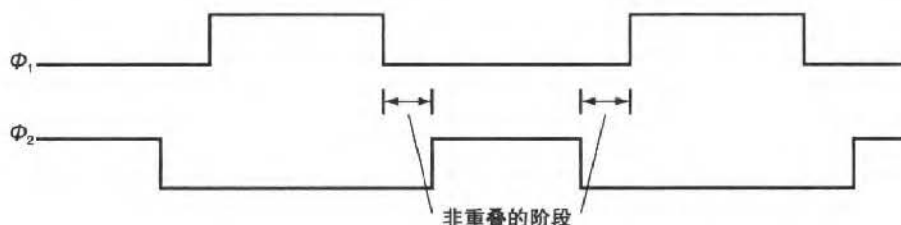


图 A-11-3 双相时钟机制下的每个时钟周期以及非重叠的阶段

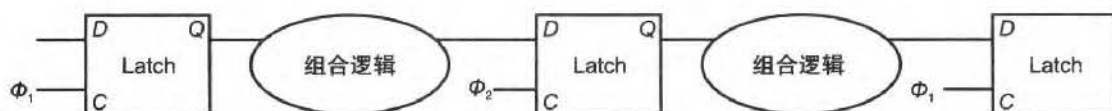


图 A-11-4 在带交替锁存器的双相时序机制下，系统如何在两个时钟相位上运行。锁存器的输出在与输入 C 相反的相位上是稳定的。因此，第一个组合逻辑块在 Φ_2 期间有稳定的输入，其输出则被 Φ_2 锁存。第二个（最右边）组合块以相反的方式工作， Φ_1 期间输入稳定。因此，经过组合块的时延决定了各时钟必须有效的最短时间。非重叠时域的长度则由任意逻辑块的最大时钟偏移和最小时延决定

A.11.2 异步输入和同步器

通过使用单个时钟或双相时钟，如果能够避免时钟偏移问题，那么就能消除竞争现象。不幸的是，整个系统仅使用一个时钟信号并且保持很小的时钟偏移是不切实际的。CPU 可以使用单个时钟，而 I/O 设备可能也会有自己的时钟。异步设备可以通过一系列握手步骤与 CPU 进行通信。为了将异步输入信号转化为同步信号并用于改变系统的状态，我们需要使用同步器。同步器的输入为异步信号和一个时钟，输出信号与输入时钟同步。

我们首先尝试使用边沿触发的 D 触发器实现同步器，其中 D 输入是异步信号，如图 A-11-5 所示。由于我们采用握手协议进行通信，因此在一个时钟或下一个时钟是否检测异步信号的有效状态并不重要，因为该信号会一直保持有效直到被确认。因而，你可能会认为这种简单的设计足以准确地对信号进行采样，除了一个小问题以外。

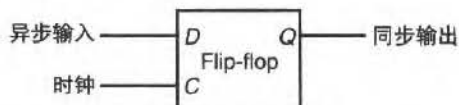


图 A-11-5 由 D 触发器构成的同步器，用于采样异步信号，并将产生和时钟同步的输出信号。该同步器不能正常工作

该问题是一种称为亚稳态（metastability）的情况。假设当时钟边沿到达时，异步信号在高低电平间振荡。显然，难以判断信号是被锁存为高电平还是低电平。这个问题可以克服。不幸的是，情况更糟：如果被采样信号在建立时间和保持时间不稳定，那么触发器可能会进入亚稳态。这种状态下，输出将不具有合法的高电平值或

亚稳态：采样时信号不满足建立时间和保持时间的要求所产生的情况，可能导致采样数据落入高低电平间的不确定区域内。

低电平值，而是介于二者之间。此外，无法保证触发器在有限的时间内退出这种状态（稳定下来）。此时，一些查看触发器输出的逻辑模块可能看到的输出是 0，而其他的可能看到的是 1。这种现象叫作同步器故障（synchronizer failure）。

在纯同步系统中，通过确保满足输出器或锁存器的建立时间和保持时间约束，可以避免同步失败，但是当输入是异步信号时会出现例外。唯一可能的解决方案是，在查看触发器输出信号之前，等待足够长的时间以确保其输出稳定，并且如果进入的话，触发器已退出亚稳态。那究竟该等多长时间呢？触发器处于亚稳态的概率是按指数级衰减的，因此在较短的时间之后，触发器处于亚稳态的概率就非常低了；但永远不会为 0！因此，设计人员需要等待足够长的时间才能使同步失败的概率很小，而这些失败的间隔可能是几年甚至几千年。

对于大多数触发器的设计而言，经过几个建立时间后，同步失败的概率就非常小了。如果时钟频率比亚稳态周期要长（这种情况是可能的），那么就可以用两个 D 触发器来构造一个安全的同步器，如图 A-11-6 所示。有兴趣的读者可进一步阅读参考文献。

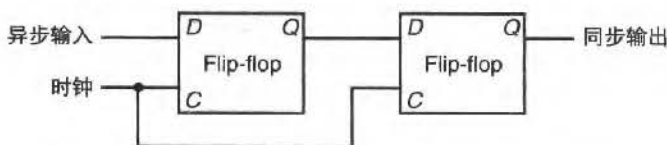


图 A-11-6 如果亚稳态的时间小于时钟周期，该同步器就可以正常工作。尽管第一个触发器的输出可能是亚稳态，但其他任何逻辑单元在第二个时钟之前看不到亚稳态。第二个 D 触发器在第二个时钟采样信号，此时第一个触发器不应再处于亚稳态

同步器故障：触发器进入亚稳态状态，并且有些逻辑模块读到触发器输出为 0，而另外一些模块读到触发器输出为 1。

A-75

小测验 假设设计中有非常大的时钟偏移，比寄存器的传输时间大。那么这样的设计是否总有可能使时钟速度减到足够慢，从而保证逻辑正常工作？

传输时间：触发器的输入传输到触发器的输出所需要的时间。

- a. 可以，如果时钟足够慢，即使时钟偏移非常大，信号也总是能正常传输，整个电路设计能正常运行。
- b. 不可以，因为有可能存在这样一种情况：两个寄存器看到同一个时钟边沿的时间相差过大，以致一个寄存器被触发，其输出被传输出去，并被第二个寄存器在相同的时钟边沿看到。

A-76

A.12 现场可编程器件

对于全定制或半定制芯片，设计者可以利用底层结构的灵活性方便地实现组合或时序逻辑。对于那些不想使用全定制或半定制 IC 的设计者，如何利用高级集成电路实现复杂的逻辑设计呢？全定制和半定制 IC 之外实现时序逻辑和组合逻辑最常用的是现场可编程器件（Field Programmable Device, FPD）。FPD 是一种集成电路，包括的组合逻辑和可能的存储器件可以由使用者配置。

现场可编程器件：一种包含组合逻辑，可能也包含存储器件的集成电路，用户可以对它进行配置。

可编程逻辑器件：一种包含组合逻辑的集成电路，功能可由用户配置。

FPD 主要分为两个阵营：可编程逻辑器件（Programmable Logic

Device, PLD) 是纯粹的组合逻辑; 现场可编程门阵列 (Field Programmable Gate Array, FPGA) 提供组合逻辑和触发器。PLD 由两种形式组成: 简单 PLD (Simple PLD, SPLD), 通常是 PLA 或者可编程阵列逻辑 (Programmable Array Logic, PAL); 复杂 PLD, 包括多于一个的逻辑模块以及模块间的可配置互连线路。当谈到 PLD 中的 PLA 时, 通常是指带有用户可编程与阵列和或阵列的 PLA。PAL 类似于 PLA, 但其或阵列是固定的。

在讨论 FPGA 以前, 先来看看 FPD 是如何配置的。配置的主要问题就是在何处建立或打破连接。门电路和寄存器结构是静态的, 但是连接是可配置的。注意, 通过配置连接, 用户可以决定实现何种逻辑功能。考虑一个可配置的 PLA: 通过决定与阵列和或阵列在何处连接, 用户决定 PLA 实现什么样的逻辑功能。FPD 中的连接可以是永久的, 也可以是可配置的。永久连接涉及在两个连线之间建立或破坏连接。现在的 FPLD 都使用反熔丝 (antifuse) 技术, 允许在编程时建立连接然后再永久固定下来。配置 CMOS FPLD 的另一种方法是使用 SRAM。在上电时, 配置信息下载到 SRAM 中, 这些内容控制开关设定进而决定哪些金属线连接起来。FPD 使用 SRAM 控制的好处在于可以通过修改 SRAM 的内容进行重新配置。基于 SRAM 控制的两个缺点是: 配置信息是易失的, 必须在上电时重新加载; 用于开关的有源晶体管的使用稍微增加了这种连接的电阻。

现场可编程门阵列: 一种可配置的由组合逻辑块和触发器构成的集成电路。

简单可编程逻辑器件: 包含一个可编程的与阵列, 以及一个固定的或阵列。

可编程阵列逻辑: 由一个可编程的与阵列后跟一个可编程的或阵列组成的可编程逻辑电路。

反熔丝: 集成电路中的一种结构, 编程时在两条导线之间形成永久连接。

查找表: 现场可编程器件中单元的名称, 因为这些单元由少量的逻辑和 RAM 组成。

A-77

FPGA 包含逻辑和存储器件, 通常组织为二维阵列结构, 由通道划分为行、列, 用于阵列单元间的全局互连。每个单元是门和触发器的组合, 可以编程执行特定功能。因为它们基本上是小型、可编程 RAM, 因此也被称为查找表 (Lookup Table, LUT)。更新的 FPGA 包括更复杂的构建模块, 例如加法器和用来构建寄存器文件的 RAM 块。一些大型的 FPGA 甚至包含 32 位的 RISC 核。

除了可以对每个单元进行编程以执行特定的功能外, 单元间的互连也是可编程的, 这就使得现在包含上百模块和上千门电路的 FPGA 可以实现复杂的逻辑功能。互连是可定制芯片中最大的挑战, 对于 FPGA 更是如此, 因为单元不代表结构化设计的自然分解单元。许多 FPGA 有 90% 的面积用来实现互连, 只有 10% 是逻辑和存储模块。

正如你不可能不使用 CAD 工具来设计全定制或半定制芯片一样, 你也需要 CAD 工具来设计 FPD。已经开发出针对 FPGA 的逻辑综合工具, 允许从结构或行为级的 Verilog 描述中使用 FPGA 来生成一个系统。

A.13 本章小结

本附录介绍了逻辑设计的一些基础知识。在了解了这些内容之后, 你可以继续阅读第 4、5 章。这两章使用了本附录中讨论的概念和原理。

拓展阅读

关于逻辑设计, 有很多好书, 以下列出了其中一些。

Ciletti, M. D. [2002]. *Advanced Digital Design with the Verilog HDL*, Englewood Cliffs, NJ: Prentice Hall.

A thorough book on logic design using Verilog.

Katz, R. H. [2004]. *Modern Logic Design*, 2nd ed., Reading, MA: Addison-Wesley.
A general text on logic design.

Wakerly, J. F. [2000]. *Digital Design: Principles and Practices*, 3rd ed., Englewood Cliffs, NJ: Prentice Hall.

A general text on logic design.

A-78

A.14 练习题

A.1 [10] <A.2> 除本节中讨论的基本定律之外, 还有两个重要的定律, 称为德·摩根定律:

$$\overline{A+B} = \overline{A} \cdot \overline{B} \quad \text{和} \quad \overline{A \cdot B} = \overline{A} + \overline{B}$$

请使用下面的真值表对上面的德·摩根定律进行证明:

A	B	$\overline{A}$	$\overline{B}$	$\overline{A+B}$	$\overline{A} \cdot \overline{B}$	$\overline{A \cdot B}$	$\overline{A} + \overline{B}$
0	0	1	1	1	1	1	1
0	1	1	0	0	0	1	1
1	0	0	1	0	0	1	1
1	1	0	0	0	0	0	0

A.2 [15] <A.2> 用德·摩根定律和 A.2 节中介绍的公理证明 A.2.2 节例题中 E 的两个表达式是等价的。

A.3 [10] <A.2> 给出一个 n 输入的逻辑函数, 对应的真值表有 2^n 项。

A.4 [10] <A.2> 异或函数具有多种用途(可用于加法器或校验码计算)。对于一个二输入的异或函数, 当且仅当只有一个输入为“真”时输出才为“真”。写出一个二输入异或函数的真值表, 并用与门、或门和反相器(非门)实现该函数。

A.5 [15] <A.2> 通过使用二输入的或非门实现与、或、非三种逻辑功能, 证明利用或非门可以实现任何逻辑功能。

A.6 [15] <A.2> 通过使用二输入的与非门实现与、或、非三种逻辑功能, 证明利用与非门可以实现任何逻辑功能。

A.7 [10] <A.2, A.3> 写出四输入奇校验函数的真值表(奇偶校验参见 A.9.3 节)。

A.8 [10] <A.2, A.3> 用输入端和输出端带有“气泡”的与门和或门实现四输入的奇校验函数。

A.9 [10] <A.2, A.3> 用 PLA 实现四输入的奇校验函数。

A.10 [15] <A.2, A.3> 通过使用多路选择器实现与非门(或者或非门), 证明二输入多路选择器可以实现任何逻辑功能。

A-79

A.11 [5] <4.2, A.2, A.3> 假设 X 包含三位, x_2, x_1, x_0 。分别写出下列 4 个逻辑函数(当且仅当满足下面的条件时逻辑函数为真):

- X 中只有一个 0。
- X 中有偶数个 0。
- 当 X 被当作无符号二进制数时, X 小于 4。
- 当 X 被当作有符号数(二进制补码)时, X 是负数。

A.12 [5] <4.2, A.2, A.3> 用 PLA 实现练习题 A.11 的 4 个逻辑函数。

A.13 [5] <4.2, A.2, A.3> 假设 X 由三位 x_2, x_1, x_0 组成, Y 由三位 y_2, y_1, y_0 组成。写出下列三个逻辑表达式(当且仅当满足下面的条件时逻辑表达式为“真”):

- $X < Y$, X, Y 作为无符号二进制数处理。
- $X < Y$, X, Y 作为有符号(二进制补码)数处理。

- $X = Y$ 。

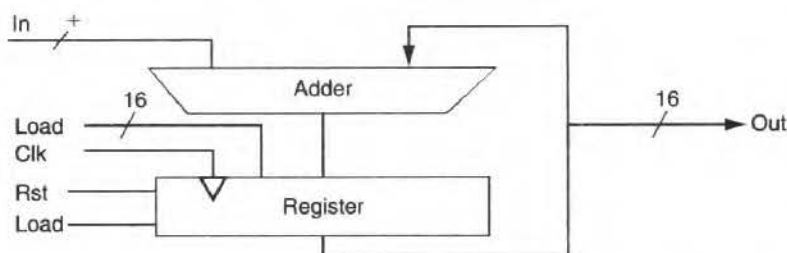
使用分层的方法可以扩展到很多位的情况。说明如何扩展为 6 位的比较。

- A.14** [5] <A.2, A.3> 实现一个开关网络, 具有 2 个输入 (A 和 B), 2 个输出 (C 和 D), 1 一个控制信号 (S)。如果 $S = 1$, 网络为直通模式, 并且 $C = A$, $D = B$; 如果 $S = 0$, 网络为交叉模式, 并且 $C = B$, $D = A$ 。
- A.15** [15] <A.2, A.3> 由 A.2 节中 E 的“积之和”形式推出其“和之积”形式。你需要使用德·摩根定律。
- A.16** [30] <A.2, A.3> 设计一个算法, 该算法能够对任何包含与、或、非逻辑的函数构建其“积之和”形式的表达式。算法应当具有递归性, 并且不应在过程中构建真值表。
- A.17** [5] <A.2, A.3> 写出多路选择器的真值表 (输入为 A 、 B 和 S , 输出为 C), 使用可能的无关项来简化真值表。
- A.18** [5] <A.3> 下面的 Verilog 模块实现了何种功能:

```
module FUNC1 (I0, I1, S, out);
    input I0, I1;
    input S;
    output out;
    out = S? I1: I0;
endmodule

module FUNC2 (out,ctl,clk,reset);
    output [7:0] out;
    input ctl, clk, reset;
    reg [7:0] out;
    always @(posedge clk)
        if (reset) begin
            out <= 8'b0 ;
        end
        else if (ctl) begin
            out <= out + 1;
        end
        else begin
            out <= out - 1;
        end
    end
endmodule
```

- A.19** [5] <A.4>A.8.1 节给出了 D 触发器的 Verilog 代码, 请给出 D 锁存器的 Verilog 代码。
- A.20** [10] <A.3, A.4> 写出 2-4 译码器 (与 / 或编码器) 的 Verilog 模块实现。
- A.21** [10] <A.3, A.4> 根据下面给出的累加器的逻辑图, 写出其 Verilog 模块实现。假定使用上升沿触发寄存器和异步 Rst。



- A.22** [20] <A.3, A.4, A.5> 节介绍了乘法器的基本操作和可能的实现。这一实现的基本单元是一个移位加法单元。给出这个单元的 Verilog 实现, 并说明如何使用这个单元构建一个 32 位的乘法器。
- A.23** [20] <A.3, A.4, A.5> 根据上一题, 实现无符号除法器。

- A.24 [15] <A.5> 仅使用加法器的符号位, ALU 可以支持 `slt` (小于设置)。用这种方法比较 -7_{ten} 和 6_{ten} , 为简单起见, 使用 4 位二进制表示 1001_{two} 和 0110_{two}

$$1001_{\text{two}} - 0110_{\text{two}} = 1001_{\text{two}} + 1010_{\text{two}} = 0011_{\text{two}}$$

这个结果表示 $-7 > 6$, 显然是错误的。因此在判断时必须考虑到溢出。修改图 A-5-10 中的 1 位 ALU 来正确处理 `slt`。为了节省时间可以直接在复印的图上修改。

- A.25 [20] <A.6> 在加法中检查溢出的一个简单方法是看最高有效位的 `CarryIn` 是否和最高有效位的 `CarryOut` 相同。证明这个方法和图 3-2 是一样的。

- A.26 [5] <A.6> 使用新定义重写 A.6.3 节中 16 位加法器的超前进位逻辑公式。首先, 使用加法器独立位 `CarryIn` 信号的名字, 即使用 $c_4, c_8, c_{12}, \dots$, 而不是使用 $C_1, C_2, C_3, \dots$; 此外, $P_{i,j}$ 表示 i 位到 j 位的传输信号, $G_{i,j}$ 表示 i 位到 j 位的生成信号, 例如, 公式

$$C_2 = G_1 + (P_1 \cdot G_0) + (P_1 \cdot P_0 \cdot c_0)$$

可改写成

$$c_8 = G_{7,4} + (P_{7,4} \cdot G_{3,0}) + (P_{7,4} \cdot P_{3,0} \cdot c_0)$$

该定义在建立位数更宽的加法器时非常有用。

- A.27 [15] <A.6> 使用练习题 A.26 的新定义写出 64 位加法器的超前进位逻辑公式, 使用 16 位加法器作为基础模块。并给出类似图 A-6-3 的图。

- A.28 [10] <A.6> 下面计算加法器的相对性能。假定任一公式对应的硬件只包含 AND 或者 OR (例如 A.6.2 节的 p_i 和 g_i 公式), 运行时间为一个时间单位 T 。而由 OR 和若干 AND 构成的公式 (例如 A.6.2 节中的公式 c_1, c_2, c_3 和 c_4) 执行时间需要 $2T$ 。原因是其需要一个 T 来生成 AND 项, 并需要另一个 T 来产生 OR 的结果。分别计算 4 位行波进位加法器和超前进位加法器的运算次数和性能的比。如果公式中的项由其他公式定义, 则增加中间公式带来的相应时延, 并反复迭代直到公式中使用的都是加法器的实际输入为止。画出每个加法器, 标出计算时延, 并标明最坏时延的路径。

- A.29 [15] <A.6> 类似练习题 A.28, 不过这次只计算相对 16 位加法器的相对速度, 这些加法器结构分别是: (1) 行波进位加法器; (2) 4 位一组, 组内超前进位, 组间行波进位; (3) 采用 A.6.2 节方案的超前进位加法器。

- A.30 [15] <A.6> 与练习题 A.28 和 A.29 类似, 本题计算 64 位加法器的相对速度, 加法器的结构分别是: (1) 行波进位加法器; (2) 4 位一组, 组内超前进位, 组间行波进位; (3) 16 位一组, 组内超前进位, 组间行波进位; (4) 采用练习题 A.27 方案的超前进位加法器。

- A.31 [10] <A.6> 我们不仅可以把加法器看成一个把两个数相加然后与进位连接到一起的装置, 也可以令加法器把三个数 (a_i, b_i, c_i) 相加, 并且产生两个输出 (s, c_{i+1})。当进行两个数的加法时, 我们并不能据此做些什么。但是当我们将两个以上的操作数相加时, 就有可能减少进位的开销。该想法是构造两个独立的“和”, 分别叫作 S' (和位) 和 C' (进位位)。在这一过程的末尾, 我们需要用一个普通的加法器把 S' 和 C' 加到一起。这个把进位传输推迟到加法运算最后阶段的技术称为进位保留加法。图 A-14-1 右下角的模块图显示了该加法器的结构, 其中两级进位保留加法器通过一个普通加法器连接在一起。

计算将 4 个 16 位数相加的延迟, 分别采用完全超前进位加法器、带有超前进位加法器 (用来形成最终的累加和) 的进位保留加法器 (时间单位 T 与练习题 A.28 相同)。

- A.32 [20] <A.6> 在计算机当中最有可能同时把多个数相加到一起的情形, 可能出现在试图在一个时钟周期中通过采用多个加法器将多个数相加来加快乘法运算的时候。相比于第 3 章中的乘法算法, 具有多个加法器的进位保留方案可以实现 10 倍以上的乘法运算速度。本题评估一个组合逻辑乘法器计算两个 16 位正数乘法的开销和速度。假设有 16 个中间项 $M_{15}, M_{14}, \dots$,

M_0 ，也称为部分积。这些部分积代表被乘数和乘数每一位 ($m_{15}, m_{14}, \dots, m_0$) 进行“与”运算的结果。我们的想法是用进位保留加法器将 n 个操作数减少到 $2n/3$ 个并行组 (每组 3 个)，反复迭代直至得到两个大数，最后用普通加法器把二者加到一起。

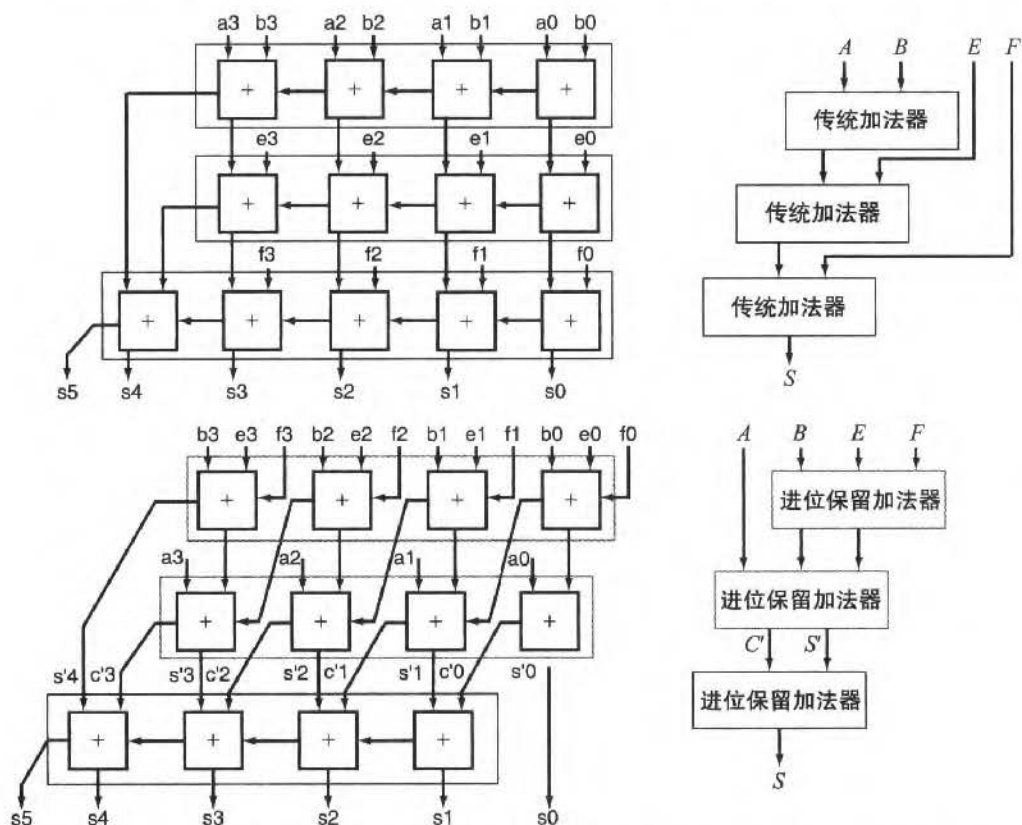


图 A-14-1 4 个 4 位数相加的传统行波进位加法器和进位保留加法器。左边给出了加法器的细节，其中单独信号用小写表示，相应的高层模块见右图，组合信号用大写表示。注意，4 个 n 位数的和需要 $n+2$ 位

A-84

首先，画出将这 16 项相加的 16 位进位保留加法器的结构图，如图 A-14-1 右半部分所示。然后计算这 16 项相加的延迟。再然后计算把这 16 项加到一起的延迟。将该延迟与第 3 章中的迭代乘法方案进行比较，但这里仅假定 16 次迭代过程中使用的是具有完全超前进位的 16 位加法器，该加法器的速度在练习题 A.29 中已经计算过。

- A.33** [10] <A.6> 有时用户想要将一组数加在一起。假设现在要使用 1 位全加器将 4 个 4 位数 (A, B, E, F) 相加，忽略超前进位。你可能想按图 A-14-1 上部的组织形式将这些 1 位加法器连接起来。在传统组织形式之下是一种全新的全加器组织形式。使用这两种组织结构实现 4 个数的加法，来确定你得到了相同的答案。
- A.34** [5] <A.6> 首先，画出将这 16 项相加的 16 位进位保留加法器的组织结构图，如图 A-14-1 所示。假定通过每个 1 位加法器的时间是 $2T$ 。计算分别采用图 A-14-1 上下两个结构将 4 个 4 位数相加所需的时间。
- A.35** [5] <A.8> 很多时候你可能希望给定一个时序图，该时序图包含了对发生在数据输入端 D 和时钟输入端 C (分别如图 A-8-3 和 A-8-6 所示) 的变化的描述，那么 D 锁存器和 D 触发器的输出端波形 (Q) 是不同的。用一两句话描述两个输出端波形之间没有差别的情况 (如输入的性质)。
- A.36** [5] <A.8> 图 A-8-8 描述了 LEGv8 数据通路中寄存器文件的实现。假设需要建立一个新的寄存

器文件，但是只有两个寄存器和一个读端口，并且每个寄存器只有两位数据。重绘图 A-8-8，使图中每根连接线仅与一位数据对应（不像图 A-8-8 中那样，有些连接线为 5 位，有些则为 32 位）。采用 D 触发器重绘图中的寄存器。无需画出 D 触发器或多路选择器的具体实现。

- A.37** [10] <A.10> 有个朋友想让你帮忙设计一个仿安全装置的“电子眼”。该设备由排成一行的三个灯组成，这三个灯分别受输出 Left、Middle 和 Right 控制，即当这三个信号当中的某一个有效时，对应的灯被点亮。每次仅有一个灯被点亮，并且灯光从左到右“移动”，然后再从右到左，这样可以吓跑那些误以为该设备正在监控其行踪的小偷。画出用于控制该“电子眼”的有限状态机图示。注意，“眼睛”的移动速率受时钟速度（不应当过高）的控制，并且根本没有输入信号。
- A.38** [10] <A.10> 为上题中的有限状态机分配状态号，并写出每个输出信号以及下一状态各位的逻辑表达式。
- A.39** [15] <A.2, A.8, A.10> 用 3 个 D 触发器和若干逻辑门构造一个 3 位计数器。计数器的输入包括一个复位信号 reset（将计数器复位至 0），以及一个计数器增加信号 inc。输出为计数器的值。当计数器值为 7 并且继续增加时，计数值回滚到零。
- A.40** [20] <A.10> 格雷码是一个二进制序列，序列中相邻的两个编码最多有一位不同。例如，下面是一个 3 位格雷码序列：000, 001, 011, 010, 110, 111, 101, 100。用三个 D 触发器和一个 PLA 实现 3 位格雷码计数器，具有两个输入信号：复位信号 reset（将计数器设为 000）和增量信号 inc（使计数器进入序列中的下一状态）。注意，该编码序列是循环的，所以序列中 100 的下一个值为 000。
- A.41** [25] <A.10> 我们希望在 A.10 节的交通灯例子中添加一个黄灯。将时钟频率调整为 0.25Hz（时钟周期 4 秒），即黄灯的持续时间。为防止绿灯和红灯循环过快，我们加入了一个 30 秒的计时器。该计时器只有一个输入信号 TimerReset，用于对计时器进行重启；一个输出信号 TimerSignal，表示 30 秒时间已经期满。同时，为了把黄灯包含进去，我们必须重新定义交通信号，每个灯定义两个输出信号 green 和 yellow。如果输出 NSgreen 有效，绿灯被点亮；如果输出 NSyellow 有效，黄灯被点亮。如果两个信号都无效，则红灯被点亮。green 和 yellow 信号不能同时有效，否则美国司机见到之后肯定会感到困惑，即使欧洲司机明白其中的含义！画出上述改进后的控制器对应的有限状态机图示。注意，状态名称不要和输出信号同名。
- A.42** [15] <A.10> 写出练习题 A.41 中交通灯控制器的下一状态表和输出函数表。
- A.43** [15] <A.2, A.10> 为练习题 A.41 的交通灯分配状态编码，并使用练习题 A.42 中的表格写出每个输出信号的逻辑表达式，包括下一状态输出。
- A.44** [15] <A.3, A.10> 用 PLA 实现练习题 A.43 的逻辑表达式。

A-85

小测验答案

A.2 节 否。如果 $A=1, C=1, B=0$ ，那么第一个为真，第二个为假。

A.3 节 C

A.4 节 全部相同。

A.4 节 $A=0, B=1$ 。

A.5 节 2。

A.6 节 1。

A.8 节 c。

A.10 节 b。

A.11 节 b。

A-86

索引

索引中的页码为英文原书页码, 与书中页边标注的页码一致。

1-bit ALU (一位 ALU), A-26-29, 见 Arithmetic logic unit (ALU)
adder (加法器), A-27
CarryOut (进位输出), A-28
for most significant bit (最高有效位), A-33
illustrated (图示), A-29
logic unit for AND/OR (与 / 或逻辑运算单元), A-27
performing AND, OR, and addition (执行与、或以及加法), A-31, A-33
64-bit ALU (64 位 ALU), A-29-37, 见 Arithmetic logic unit (ALU)
defining in Verilog (Verilog 中定义), A-34-37
from 31 copies of 1-bit ALU (复制 31 个 1 位 ALU), A-34
illustrated (图示), A-35
ripple carry adder (行波进位加法器), A-29
with 32 1-bit ALU (采用 32 个 1 位 ALU), A-30

A

AArch32 (ARMv8 32 位架构), 73
AArch64 (ARMv8 64 位架构), 73
Absolute references (绝对引用), 131
Abstractions (抽象)
 hardware/software interface (软 / 硬件接口), 22
 principle (原理), 22
 to simplify design (为简化设计), 11
Acronyms (首字母缩略词), 9
Active matrix (主动矩阵式), 18
ADD (add, 加), 64
ADDI (add Immediate, 立即数加), 64
ADDIS (add Immediate and set flags, 加立即数并设置标志位), 64
Addition (加法), 188-191, 见 Arithmetic

binary (二进制), 188-189
floating-point (浮点), 212-215, 220
operands (操作数), 189
significands (有效位), 211
speed (速度), 191
Address interleaving (地址交叉), 395
Address space (地址空间), 442, 445
 extending (扩展), 493
 flat (平面), 493
 ID (ASID, 地址空间标识), 460
 shared (共享的), 533-534
 single physical (单一物理地址), 533-534
 virtual (虚拟), 460
Address translation (地址转换)
 for ARM cortex-A8 (ARM cortex-A8 芯片), 483
 defined (定义), 443
 fast (快速), 452-454
 for Intel core i7 (英特尔酷睿 i7), 471, 483
 TLB for (快表), 452-454
Addresses (地址)
 base (基址), 69
 byte (字节), 70
 defined (定义), 69
 memory (主存), 79
 virtual (虚拟), 442, 462, 463
Addressing (寻址)
 base (基址), 120
 in branches (分支), 117-120
 displacement (偏移), 120
 immediate (立即数), 120
 LEGv8 modes (LEGv8 模式), 120-121
 PB-relative (PC 相对寻址), 118, 120
 register (寄存器), 120
 x86 modes (x86 模式), 158

- ADDS (add and set flags, 加并设置标志位), 64, 164
- addu (Add Unsigned, 无符号整数加法), 64
- Advanced Encryption Standard (AES, 高级加密标准), 488
- Advanced Vector Extensions (AVX, 高级向量扩展), 232, 240
- Aliasing (别名), 458, 459
- Alignment restriction (对齐限制), 71
- ALU control (ALU 控制), 271-273, 见 Arithmetic logic unit (ALU)
- bits (比特/位), 260
- ALU control block (ALU 控制块), 275
- ALUOp (ALU 操作码), 272
- bits (比特/位), 272, 273
- control signal (控制信号), 275
- Amazon Web Services (AWS, 亚马逊网络服务), 439
- AMD Opteron X4 (Barcelona) (AMD 皓龙处理器 X4 (巴塞罗那)), 559, 560
- AMD64 (AMD 推出的一种 64 位处理器架构), 155, 156, 232
- Amdahl's law (Amdahl 定律), 415, 519
- corollary (结果/推论), 49
- defined (定义), 49
- fallacy (谬误), 572
- and (AND, 与), 64
- AND gates (与门), A-12
- AND operation (与操作), 91, A-6
- andi (And Immediate, 立即数与), 65
- Annual failure rate (AFR, 年失效率), 432
- Versus. MTTF of disks (与磁盘平均无故障时间比较), 433-434
- Antidependence (反依赖), 348
- Antifuse (反熔丝), A-77
- Apple iPad 2 A1395 (苹果平板电脑), 20,
- logic board of (逻辑主板), 20
- processor integrated circuit of (处理器集成电路), 21
- Application binary interface (ABI, 应用程序二进制接口), 22
- Application programming interface (API, 应用程序编程接口)
- Architectural registers (体系结构寄存器), 358
- Arithmetic (算术运算), 186-248
- addition (加法), 188-191
- addition and subtraction (加减法), 188-191
- division (除法), 197-204
- fallacies and pitfalls (谬误与陷阱), 242-245
- floating-point (浮点), 205-230
- historical perspective (历史观点), 248
- multiplication (乘法), 191-197
- parallelism and (并行与), 230-232
- Streaming SIMD Extensions and advanced vector extensions in x86 (x86 中的 SIMD 扩展和高级向量扩展), 232-233
- subtraction (减法), 188-191
- subword parallelism (子字并行), 230-232
- subword parallelism and matrix multiply (子字并行和矩阵乘法), 238-242
- Arithmetic instructions (算术运算指令), 见 Instructions
- logical (逻辑), 263
- operands (操作数), 67-74
- Arithmetic intensity (算术密度), 557
- Arithmetic logic unit (ALU, 算术逻辑运算单元), 见 ALU control, Control units
- 1-bit (1 位), A-26-29
- 64-bit (64 位), A-29-37
- before forwarding (转发之前), 309
- branch datapath (分支数据通路), 266
- hardware (硬件), 190
- memory-reference instruction use (存储器相关指令的使用), 257
- for register values (寄存器值), 264
- R-format operations (R 型操作), 265
- signed-immediate input (有符号立即数输入), 323
- ARM Cortex-A53 (ARM Cortex-A53 芯片), 256, 355-358
- address translation for (地址转换), 483
- caches in, 484
- data cache miss rates for (数据 cache 缺失率), 485
- memory hierarchies of (存储器层次结构), 482
- performance of (性能), 485-487
- specification (规格), 356
- TLB hardware for (快表硬件), 483
- ARM instructions (ARM 指令), 152-154

- 12-bit immediate field (12 位立即数字段), 153
 - condition field (条件字段), 334
 - ARMv7, 62
 - ARMv8, 62, 163-169
 - common features between MIPS and (和 MIPS 之间共同的特点), 152
 - Arrays (数组), 429
 - logic elements (逻辑部件), A-18-19
 - multiple dimension (多维), 226
 - pointers versus (指针), 146-150
 - procedures for setting to zero (置零过程 / 程序), 147
 - ASCII
 - binary numbers versus (二进制数), 111
 - character representation (字符表示), 110
 - defined (定义), 110
 - symbols (符号 / 标志), 113
 - Assemblers (汇编器), 129-131
 - defined (定义), 14
 - function (函数), 129
 - number acceptance (数据接收), 130
 - object file (目标文件), 130
 - Assembly language (汇编语言), 15
 - defined (定义), 14, 129
 - floating-point (浮点), 221
 - illustrated (图示), 15
 - LEGv8, 64, 86
 - programs (程序), 129
 - translating into machine language (翻译为机器语言), 86
 - Asserted signals (无效信号), 262, A-4
 - Associativity (相联度)
 - in caches (在 cache 中), 419
 - degree (度), increasing (增加), 418, 466
 - increasing (增加), 423
 - set (设置), tag size versus (标记位大小), 423
 - Atomic compare and swap (原子比较与交换), 127
 - Atomic exchange (原子交换), 126
 - Atomic fetch-and-increment (原子取后递增), 127
 - Automobiles, computer application in (车载, 计算机应用), 4
 - Average memory access time (AMAT, 平均访存时间), 416
 - calculating (计算), 416
- ## B
- Bandwidth (带宽), 30
 - bisection (二分法), 551
 - external to DRAM (DRAM 外部), 412
 - memory (主存), 394-395, 412
 - network (网络), 549
 - Base addressing (基址寻址), 69, 120
 - Base registers (基址寄存器), 70
 - Basic block (基本块), 96
 - Benchmarks (基准测试程序), 554-556
 - defined (定义), 46
 - multicores (多核), 538-545
 - multiprocessor (多处理器), 554-556
 - NAS parallel (NAS 并行), 556
 - parallel (并行), 555
 - PARSEC suite (PARSEC 程序集), 556
 - SPEC CPU, 46-48
 - SPEC power (SPEC 功耗), 48-49
 - SPECrate (SPEC 分值), 554
 - Stream (流), 564
 - Biased notation (移码表示), 82, 209
 - Big-endian byte order (大端字节次序), 70
 - Binary numbers (二进制数), 83
 - ASCII versus (ASCII 码), 111
 - conversion to decimal numbers (转换为十进制数), 78
 - defined (定义), 75
 - Bisection bandwidth (对分带宽), 551
 - Bit maps (位图),
 - defined (定义), 18, 73
 - goal (目标), 18
 - storing (存储), 18
 - Bits (二进制位)
 - ALUOp (ALU 操作码), 272, 273
 - defined (定义), 14
 - dirty (脏数据), 452
 - guard (保护), 228
 - patterns (模式), 228-229
 - reference (引用), 450
 - rounding (舍入), 228
 - sign (符号), 77

- sticky (粘贴), 228
 - valid (有效), 397
 - Blocking assignment (阻塞赋值), A-24
 - Blocking factor (分块因子), 428
 - Blocks (块)
 - combinational (组合的), A-4
 - defined (定义), 390
 - finding (查找), 467
 - flexible placement (灵活放置), 416-418
 - least recently used (LRU, 最近最少使用), 423
 - locating in cache (位于 cache 内), 421-422
 - miss rate and (失效率), 405
 - multiword mapping addresses to (多字地址映射), 404
 - placement locations (放置位置), 466
 - placement strategies (放置策略), 418
 - replacement selection (替换选择), 423
 - replacement strategies (替换策略), 468
 - spatial locality exploitation (开发空间局部性), 405
 - state (状态), A-4
 - valid data (有效数据), 400
 - Bonding (绑定), 28
 - Boolean algebra (布尔代数), A-6
 - Bounds check shortcut (边界检查的简便方法), 98
 - Branch address (分支地址), 168
 - Branch datapath (分支数据通路)
 - ALU (算术逻辑运算单元), 266
 - operations (操作), 266
 - Branch delay slots (分支延迟槽)
 - Branch instructions (分支指令)
 - pipeline impact (流水线影响), 329
 - Branch not taken (分支未发生)
 - assumption (假定), 328-329
 - defined (定义), 266
 - Branch prediction (分支预测)
 - as control hazard solution (控制冒险解决方案), 295
 - buffers (缓冲区), 331, 333
 - defined (定义), 294
 - dynamic (动态), 295, 331-334
 - static (静态), 345
 - Branch predictors (分支预测器)
 - accuracy (准确性), 333
 - correlation (相关), 333
 - information from (信息来源), 333
 - tournament (竞争), 333
 - Branch register (分支寄存器), 168
 - Branch table (分支表), 169
 - Branch taken (分支发生)
 - cost reduction (成本降低), 330
 - defined (定义), 266
 - Branch target (分支目标)
 - addresses (地址), 266
 - buffers (缓冲区), 333
 - Branches (分支), 见 Conditional branches
 - addressing in (寻址), 117-120
 - compiler creation (编译器创建), 94
 - decision (决策), moving up (上移), 330
 - delayed (延迟), 295, 330-331, 295
 - ending (结束), 96
 - execution in ID stage (在指令译码级执行), 330
 - pipelined (流水线), 330
 - unconditional (无条件), 318
 - Branch-on-zero instruction (为 0 分支指令), 280
 - B-type instruction format (B 型指令格式), 113
 - Bubble Sort (冒泡排序), 145
 - Bubbles (气泡), 326
 - Buses (总线), A-19
 - Bytes (字节),
 - addressing (寻址), 70
 - order (顺序), 70
- ## C
- C language (C 语言)
 - assignment compiling into LEGv8 (赋值语句编译为 LEGv8 指令), 66
 - compiling (编译), 150
 - compiling assignment with registers (使用寄存器编译赋值语句), 68
 - compiling while loops in (编译 while 循环), 95-96
 - sort algorithms (排序算法), 146
 - translation hierarchy (翻译层次), 128
 - translation to LEGv8 assembly language (翻译为 LEGv8 汇编语言), 66
 - variables (变量), 106
 - Cache blocking and matrix multiply (cache 分块和

- 矩阵乘法), 488-491
- Cache coherence (cache 一致性), 477-481
 - coherence (一致性), 477
 - consistency (连贯性 / 一致性), 466
 - enforcement schemes (强制执行), 467-468
 - migration (移植 / 迁移), 479
 - problem (问题), 477, 478, 481
 - protocols (协议), 479
 - replication (复制), 479
 - snooping protocol (监听协议), 479-481
- Cache controllers (cache 控制器), 482
- Cache hits (cache 命中), 458
- Cache misses (cache 缺失)
 - block replacement on (块替换), 468
 - capacity (容量), 470, 471
 - compulsory (强制的), 470
 - conflict (冲突), 470
 - defined (定义), 306
 - direct-mapped cache (直接映射 cache), 418
 - fully associative cache (全相联 cache), 420
 - handling (处理), 406-407
 - memory-stall clock cycles (存储器阻塞时钟周期), 413
 - reducing with flexible block placement (用灵活的块替换以减少), 416-418
 - set-associative cache (组相联 cache), 419
 - steps (步骤), 407
 - in write-through cache (在写直达式 cache 中), 407
- Cache performance (cache 性能), 412-431
 - calculating (计算), 414
 - hit time and (命中时间), 415-416
 - impact on processor performance (对处理器性能的影响), 414
- Cache-aware instructions (cache 感知指令), 496
- Caches, 397-412, 见 Blocks
 - Accessing (访问), 400-403
 - in ARM cortex-A53, (在 ARM cortex-A53 上), 484
 - associativity in (结合性), 419-420
 - bits in (位数), 404
 - bits needed for (需要的位数), 404
 - contents illustration (内容说明), 401
 - defined (定义), 21, 397-398
 - direct-mapped (直接映射), 398, 399, 404, 416
 - empty (空), 400-401
 - FSM for controlling (有限状态机控制), 472
 - fully associative (全相联映射), 417
 - inconsistent (不一致), 407
 - index (索引), 402
 - in Intel Core i7, (英特尔酷睿 i7 处理器), 484
 - Intrinsity FastMATH example (Intrinsity FastMATH 处理器), 409-412
 - locating blocks in (块定位), 421-422
 - locations (位置), 399
 - multilevel (多级), 412, 424
 - nonblocking (非阻塞), 483
 - physically addressed (物理地址), 458, 459
 - physically indexed (物理索引), 458
 - physically tagged (物理标记), 458
 - primary (首要的 / 基本的), 424, 431
 - secondary (次要的), 424, 431
 - set-associative (组相联), 417
 - simulating (模拟), 491
 - size (尺寸), 403
 - split (分离), 411
 - summary (小结), 411-412
 - tag field (标记字段), 402
 - virtual memory and TLB integration (虚拟存储器和快表集成), 457-459
 - virtually addressed (虚拟地址), 458
 - virtually indexed (虚拟索引), 458
 - virtually tagged (虚拟标记), 458
 - write-back (写回), 408, 409, 469
 - write-through (写直达), 407, 409, 469
 - writes (写操作), 407-409
- Callee (被调用者), 101, 103
- Caller (调用者), 101
- Capacity misses (容量缺失), 470
- Carry lookahead (先行进位), A-37-46
 - 4-bit ALUs using (4 位 ALU 使用), A-43
 - adder (加法器), A-38
 - fast, with first level of abstraction (快速, 使用第一级抽象), A-38-40
 - fast, with "infinite" hardware (快速, 使用“无限”硬件), A-38
 - fast, with second level of abstraction (快速, 使用第二级抽象), A-40-45

- plumbing analogy (管道类比), A-41, A-42
- ripple carry speed versus (行波进位速率), A-45
- summary (小结), A-45-46
- Carry save adders (进位保留加法器), 197
- Cause register (原因寄存器)
- Cell phones (移动电话), 7
- Central processor unit (CPU, 中央处理单元), 见 Processors (处理器)
- classic performance equation (经典性能方程), 36-40
- coprocessor0 (协处理器 0), A-33-34
- defined (定义), 19
- execution time (执行时间), 32, 33-34
- performance (性能), 33-35
- system time (系统时间), 32
- time (时间), 413
- time measurements (时间测定), 33-34
- user time (用户时间), 32
- Characters (字符),
 - ASCII representation (ASCII 码表示), 110
 - in Java (在 Java 中), 113
- Chips (芯片), 19, 25, 26
 - manufacturing process (生产过程), 26
- Clear exclusive instruction (CLREX, 清除独占指令), 487
- Clock cycles (时钟周期)
 - defined (定义), 33
 - memory-stall (存储器阻塞), 413
 - number of registers and (寄存器数量), 67
 - worst-case delay and (最坏情况延迟), 283
- Clock cycles per instruction (CPI, 每条指令执行的时钟周期数), 35, 293
 - one level of caching (一级缓存), 424
 - two levels of caching (二级缓存), 424
- Clock rate (时钟频率)
 - defined (定义), 33
 - frequency switched as function of (频率转换函数), 41
 - power and (功率/功耗), 40
- Clocking methodology (时钟策略), 261-263, A-47
 - edge-triggered (边沿触发), 261-263, A-47
 - level-sensitive (电平敏感), A-73, A-74-75
 - for predictability (可预测性), 261
- Clocks (时钟), A-47-49
 - edge (边沿), A-47, A-49
 - in edge-triggered design (边沿触发设计), A-72
 - skew (偏移), A-73
 - specification (说明书/手册), A-56
 - synchronous system (同步系统), A-47-48
- Cloud computing (云计算), 549
 - defined (定义), 7
- Cluster networking (集群网络), 553-554
 - defined (定义), 516, 546
 - isolation (隔离), 547
 - organization (组织), 515
- CMOS (complementary metal oxide semiconductor, 互补金属氧化物半导体), 41
- Coarse-grained multithreading (粗粒度多线程), 530
- Cold-start miss (冷启动失效), 470
- Collision misses (冲突失效), 470
- Column major order (按列顺序), 427
- Combinational blocks (组合块), A-4
- Combinational elements (组合元素), 260
- Combinational logic (组合逻辑), 261, A-3, A-9-20
 - arrays (数组), A-18-19
 - decoders (译码器), A-9
 - defined (定义), A-5
 - don't cares (无关项), A-17-18
 - multiplexors (多路选择器), A-10
 - ROMs (只读存储器), A-14-16
 - two-level (二级), A-11-14
 - Verilog (Verilog 语言), A-23-26
- Commit units (提交单元)
 - buffer (缓冲区), 350
 - defined (定义), 350
 - in update control (更新控制), 355
- Common case fast (加速大概率事件), 11
- Communication (通信), 23-24
 - overhead reducing (开销减少), 44-45
- Compare and branch zero (比较为 0 分支), 330
- Comparisons (比较)
 - constant operands in (常数操作数), 73
 - signed versus unsigned (有符号数与无符号数), 97
- Compilers (编译器), 129
 - branch creation (创建分支), 95
 - defined (定义), 14

- function (函数), 14, 129
- Just In Time (JIT, 及时), 137
- optimization (优化), 146
- speculation (推测), 344-345
- Compiling (编译)
 - C assignment statements (C 赋值语句), 66
 - C language (C 语言), 95, 150
 - floating-point programs (浮点程序), 222-225
 - if-then-else (if-then-else 语句), 94
 - procedures (过程), 102, 104-105
 - recursive procedures (递归过程), 104-105
 - while loops (while 循环), 95-96
- Compulsory misses (强制失效), 470, 471
- Computer architects (计算机架构师), 11-12
 - abstraction to simplify design (通过抽象简化设计), 11
 - common case fast (加速大概率事件), 11
 - dependability via redundancy (通过冗余增加可靠性), 12
 - hierarchy of memories (存储器层次结构), 12
 - Moore's law (摩尔定律), 11
 - parallelism (并行), 12
 - pipelining (流水线), 12
 - prediction (预测), 12
- Computers (计算机)
 - application classes (应用类别), traditional (传统), 5-6
 - applications (应用程序), 4
 - arithmetic for (算术), 186-248
 - component organization (组成结构), 17
 - components (组件), 17, 177
 - design measure (设计尺度), 53
 - desktop (桌面机), 5
 - embedded (嵌入式), 5
 - in information revolution (信息革命), 4
 - instruction representation (指令表示), 82-89
 - Post-PC era (后 PC 时代), 6-7
 - principles (原则 / 原理), 86
 - servers (服务器), 5
- Condition codes/flags (条件码 / 标志), 97
- Conditional branches (条件分支)
 - changing program counter with (更改程序计数器), 333
 - compiling if-then-else into (将 if-then-else 语句编译为), 94
 - defined (定义), 93
 - implementation (实现), 99
 - in loops (循环中), 119
 - PC-relative addressing (PC 相对寻址), 118
- Conditional move instructions (条件移动指令), 334
- Conflict misses (冲突失效), 470
- Constant operands (常数操作数), 73-74
 - frequent occurrence (频发), 73
- Content Addressable Memory (CAM, 按内容访问存储器), 422
- Context switch (上下文切换), 460
- Control (控制)
 - ALU (算术逻辑运算单元), 271-273
 - challenge (挑战), 336
 - finishing (完成), 281
 - forwarding (转发 / 旁路), 320
 - pipelined (流水线), 311-315
- Control functions (控制功能)
 - defining (定义), 276
 - for single-cycle implementation (单周期实现), 281
- Control hazards (控制冒险), 292-295, 328-329
 - branch delay reduction (减少分支延迟), 330
 - branch not taken assumption (假定分支不执行), 328
 - branch prediction as solution (分支预测作为解决方案), 295
 - delayed decision approach (延迟决定方法), 295
 - dynamic branch prediction (动态分支预测), 331
 - pipeline stalls as solution (流水线阻塞作为解决方案), 293
 - pipeline summary (流水线小结), 335-336
 - simplicity (简单), 328
 - solutions (解决方案), 293
 - static multiple-issue processors (静态多发射处理器), 345-346
- Control lines (控制线)
 - asserted (有效), 276
 - in datapath (数据通路中), 275
 - execution/address calculation (执行 / 地址计算), 312
 - final three stages (最后三个阶段), 314

- instruction decode/register file read (指令译码 / 读寄存器文件), 312
 - instruction fetch (取指令), 312
 - memory access (存储器访问), 312
 - setting of (一组), 276
 - values (数值), 312
 - write-back (写回), 312
 - Control signals (控制信号)
 - ALUOp (ALU 操作码), 275
 - defined (定义), 262
 - effect of (影响), 276
 - multi-bit (多位), 276
 - pipelined datapaths with (流水线数据通路), 311-315
 - Control units (控制单元), 259, 见 Arithmetic logic unit
 - illustrated (举例说明), 277
 - main designing (主要部分设计), 273-276
 - output (输出), 271-273
 - Coprocessors (协处理器)
 - defined (定义), 226
 - Core LEGv8 instruction set (核心 LEGv8 指令集), 248
 - abstract view (抽象图), 258
 - implementation (实现), 256-260
 - implementation illustration (实现举例), 259
 - overview (概述), 257-260
 - subset (子集), 256
 - Cores (核)
 - defined (定义), 43
 - number per chip (每片数量), 43
 - Correlation predictor (相关预测器), 333
 - CPU (中央处理单元), 9
 - Critical word first (关键字优先), 406
 - Crossbar networks (交叉开关网络), 551
 - CUDA programming environment (CUDA 编程环境), 539
 - Cyclic redundancy check (循环冗余校验), 437
 - Cylinder (柱面), 396
- D**
- D flip-flops (D 触发器), A-50, A-52
 - D latches (D 锁存器), A-50, A-51
 - Data bits (数据位), 435
 - Data hazards (数据冒险) 289-292, 316-328, 见 Hazards
 - forwarding (转发 / 旁路), 289, 316-328
 - load-use (读后写), 290, 329
 - stalls and (阻塞), 324-328
 - Data race (数据竞争), 125
 - Data selectors (数据选择器), 258
 - Data transfer instructions (数据传输指令), 见 Instructions
 - defined (定义), 68, 69
 - load (装载), 69
 - offset (偏移量), 70
 - store (存储), 71-72
 - Datacenters (数据中心), 7
 - Data-level parallelism (数据级并行), 524
 - Datapath elements (数据通路部件)
 - defined (定义), 263
 - sharing (共享), 268
 - Datapaths (数据通路)
 - branch (分支), 266
 - building (构建 / 搭建), 263-271
 - control unit (控制单元), 277
 - defined (定义), 19
 - design (设计), 263
 - exception handling (异常处理), 339
 - for fetching instructions (取指令), 265
 - for hazard resolution via forwarding (通过旁路解决冒险), 323
 - for LEGv8 architecture (LEGv8 体系结构), 269
 - for memory instructions (访存指令), 267
 - in operation for branch-on-zero instruction (为 0 分支指令执行), 279
 - in operation for R-type instruction (R 型指令执行), 277, 278
 - operation of (……的操作), 276-280
 - pipelined (流水的), 297-315
 - for R-type instructions (R 型指令), 278, 276-277
 - single creating (单一创建), 267
 - single-cycle (单周期), 296
 - static two-issue (静态双发射), 347
 - Deasserted signals (无效信号), 262, A-4
 - Decimal numbers (十进制数)
 - binary number conversion to (二进制数转换), 78

- defined (定义), 75
- Decision-making instructions (决策指令), 93-99
- Decoders (译码器), A-9
 - two-level (二级), A-64
- Decoding machine language (机器语言译码), 121-125
- Defect (缺陷 / 瑕疵), 26
- Delayed branches (延迟分支), 295, 见 Branches
 - as control hazard solution (作为控制冒险的解决方案), 295
 - for five-stage pipelines (五级流水), 323-324
 - reducing (减少), 330
- Delayed decision (延迟决策), 295
- DeMorgan's theorems (德·摩根定理), A-11
- Denormalized numbers (非规格化数), 230
- Dependability via redundancy (通过冗余提高可靠性), 12
- Dependable memory hierarchy (可信存储器层次结构), 432-437
 - failure defining (失效定义), 432
- Dependences (依赖 / 相关)
 - between pipeline registers (流水线寄存器之间), 319
 - between pipeline registers and ALU inputs (流水线寄存器和 ALU 输入之间), 319
 - bubble insertion and (气泡插入), 326
 - detection (检测), 318
 - name (名字), 348
 - sequence (时序), 316
- Design (设计)
 - compromises and (折中), 85
 - datapath (数据通路), 263
 - digital (数字), 366
 - logic (逻辑), 260-263
 - main control unit (主控制单元), 273-276
 - memory hierarchy challenges (存储器层次结构的挑战), 472
 - pipelining instruction sets (流水化指令集), 288
- Desktop computers defined (桌面计算机定义), 5
- DGEMM (Double precision General Matrix Multiply, 双精度通用矩阵乘法), 238, 363, 365, 427, 553
 - cache blocked version of (cache 分块版本), 429
 - optimized C version of (优化的 C 版本), 241, 363, 489
 - performance (性能), 365, 430
- Dicing (切割), 27
- Dies (管芯), 26, 26-27
- Digital design pipeline (数字设计流水线), 366
- Direct-mapped caches (直接映射 cache), 421,
 - 见 Caches address portions
 - choice of (……的选择), 422
 - defined (定义), 398, 416
 - illustrated (举例说明), 399
 - memory block location (存储器块定位), 417
 - misses (缺失), 419
 - single comparator (一位比较器), 421
 - total number of bits (总位数), 404
- Dirty bit (脏位), 452
- Disk memory (磁盘存储器), 395-397
- Displacement addressing (偏移寻址), 120
- Divide algorithm (除法算法), 200
- Dividend (被除数), 198
- Division (除法), 197-203
 - algorithm (算法), 199
 - dividend (被除数), 198
 - divisor (除数), 198
- Divisor (除数), 198
- divu (Divide Unsigned, 无符号数除法), 见 Arithmetic
 - faster (加速), 202-203
 - floating-point (浮点), 220
 - hardware (硬件), 198-201
 - hardware improved version (硬件改进版), 201
 - in LEGv8 (在 LEGv8 指令集中), 203
 - operands (操作数), 198
 - quotient (商), 198
 - remainder (余数), 198
 - signed (有符号数), 201-202
 - SRT (一种除法算法), 203
- Don't cares (无关项), A-17-18
 - example (实例), A-17-18
 - term (术语), 273
- Double data rate (DDR, 双倍数据速率), 393
- Double Data Rate (DDR) SDRAM (双倍数据速率 SDRAM), 393-394, A-64
- Double precision (双精度), 见 Single precision
 - defined (定义), 207

representation (表示), 206-207

Double words (双字), 66, 158

Dual inline memory module (DIMM, 双列直插存储器模块), 395

Dynamic branch prediction (动态分支预测), 331-334, 见 Control hazards

branch prediction buffer (分支预测缓冲), 331

loops and (循环), 333

Dynamic hardware predictors (动态硬件预测器), 295

Dynamic multiplD-issue processors (动态多发射处理器), 343, 349-352, 见 Multiple issue

pipeline scheduling (流水线调度), 350-352

superscalar (超标量), 349

Dynamic pipeline scheduling (动态流水线调度), 350-352

commit unit (提交单元), 350

concept (概念), 350

hardware-based speculation (基于硬件的推断), 352

primary units (主要单元), 351

reorder buffer (重排序缓冲区), 355

reservation station (保留站), 350

Dynamic random access memory (DRAM, 动态随机访问存储器), 392, 393-395, A-62-64

bandwidth external to (外部带宽), 412

cost (代价), 23

defined (定义), 19, A-62

Double Data Rate (DDR, 双倍数据速率), 393-394

growth of capacity (容量的增长), 25

internal organization of (内部组织), 394

pass transistor (传输晶体管), A-62

single-transistor (单晶体管), A-63

size (尺寸), 412

speed (速率), 23

synchronous (SDRAM, 同步 SDRAM), 393-394, A-59, A-64

two-level decoder (两级译码器), A-64

Dynamically linked libraries (DLL, 动态链接库), 134-136

Defined (定义), 134

lazy procedure linkage version (惰性过程链接

版本), 135

E

Early restart (提前重启), 406

EdgD-triggered clocking methodology (边沿触发时钟策略), 261, 262, A-47, A-72

advantage (优势), A-48

clocks (时钟), A-72

drawbacks (弊端), A-73

illustrated (举例说明), A-49

rising edge/falling edge (上升沿 / 下降沿), A-47

Electrically erasable programmable read-only memory (EEPROM, 电可擦除可编程只读存储器), 395

Elements (元素 / 部件)

combinational (组合的), 260

datapath (数据通路), 263, 268

memory (存储器), A-49-57

state (状态), 260, 262, 264, A-47, A-49

Embedded computers (嵌入式计算机), 5

application requirements (应用需求), 6

design (设计), 5

Encoding (编码)

LEGv8 instruction (LEGv8 指令), 86, 122

ROM logic function (ROM 逻辑函数), A-15

x86 instruction (x86 指令), 161-162

Error correction (纠错), A-64-66

Error detection (错误检测), A-65

Error detection code (错误检测码), 434

Ethernet (以太网), 23

EX stage (执行级)

load instructions (load 指令), 303

overflow exception detection (溢出异常检测), 338, 341

store instructions (store 指令), 305

Exabyte (艾字节), 6

Exception enable (异常使能), 461

Exception link register (ELR, 异常链接寄存器), 337, 459, 461

address capture (地址捕获), 340

defined (定义), 338

in restart determination (重启确定), 337

Exception Program Counters (EPC, 异常程序计数器), 326

- address capture (地址捕获), 331
- copying (复制), 181
- defined (定义), 181, 327
- in restart determination (重启决策), 326-327
- transferring (转移), 182

Exception Syndrome Register (ESR, 异常特征寄存器), 337, 461

Exceptions (异常), 336-342

- association (关联), 342
- datapath with controls for handling (带有控制处理的数据通路), 339
- defined (定义), 207, 336
- detecting (检测), 336
- event types and (事件类型), 336
- imprecise (不精确的), 342
- interrupts versus (与中断相对), 336
- in LEGv8 architecture (在 LEGv8 体系结构中), 337-338
- overflow (溢出), 339
- pipelined computer example (流水线计算机实例), 339
- in pipelined implementation (在流水线实现中), 338-342
- precise (精确的), 342
- reasons for (原因), 337-338
- result due to overflow in add instruction (由于加法指令溢出导致的结果), 341
- saving/restoring stage on (保存/恢复级), 462

Executable files (可执行文件)

- defined (定义), 131

Execute or address calculation stage (执行或地址计算级), 303

Execute/address calculation (执行或地址计算)

- control line (控制线), 312
- load instruction (load 指令), 303
- store instruction (store 指令), 303

Execution time (执行时间)

- as valid performance measure (作为有效的性能指标), 51
- CPU (中央处理单元), 32, 33-34
- pipelining and (流水线技术), 297

Exponents (指数), 206

Extended-register instructions (寄存器扩展指令), 164

F

Failures synchronizer (失效/故障同步器), A-76

Fallacies (谬误), 227, 见 Pitfalls add immediate unsigned

Amdahl's law (Amdahl 定律), 572

arithmetic (算术运算), 242-245

assembly language for performance (汇编语言性能), 169

commercial binary compatibility importance (商用计算机二进制兼容的重要性), 170

defined (定义), 49

low utilization uses little power (低利用率低能耗), 50

peak performance (峰值性能), 572

pipelining (流水线技术), 366

powerful instructions mean higher performance (强大的指令意味着更高的性能), 169

right shift (右移), 242

False sharing (假共享), 480

Fast carry (快速进位)

- with "infinite" hardware ("无限的" 硬件), A-38

- with first level of abstraction (第一级抽象), A-38-40

- with second level of abstraction (第二级抽象), A-40-45

Fault avoidance (故障避免), 433

Fault forecasting (故障预测), 433

Fault tolerance (容错), 433

Fermi architecture (Fermi 体系结构), 539, 568

Field programmable devices (FPD, 现场可编程器件), A-77

Field programmable gate arrays (FPGA, 现场可编程门阵列), A-77

Fields (字段)

- defined (定义), 84

- LEGv8 (LEGv8), 84-86

- names (名字), 84

Fine-grained multithreading (细粒度多线程), 530

Finite-state machines (FSM, 有限状态机), 472-

- 477, A-67-71
- controllers (控制器), 475
- for simple cache controller(简单 cache 控制器), 476-477
- implementation (实现), 474, A-69
- Mealy (米利机), 475
- Moore (摩尔机), 475
- next-state function (下一状态函数), 474, A-66
- output function (输出函数), A-66, A-68
- state assignment (状态分配), A-69
- state register implementation(状态寄存器实现), A-70
- style of (类型), 475
- synchronous (同步), A-66
- traffic light example (交通灯实例), A-67-69
- Flash memory (闪存), 395
 - characteristics (特征), 23
 - defined (定义), 23
- Flip-flops (触发器)
 - D flip-flops (D 触发器), A-50, A-52
 - defined (定义), A-50
- Floating point (浮点), 205-230, 232
 - assembly language (汇编语言), 221
 - binary to decimal conversion (二进制向十进制转换), 211
 - branch (分支), 220
 - challenges (挑战), 246
 - division (除法), 220
 - form (形式), 206
 - fused multiply add (混合乘加), 228
 - guard digits (保护位), 226-227
 - IEEE 754 standard (IEEE 754 标准), 207-211
 - intermediate calculations (中间结果计算), 226
 - machine language (机器语言), 212
 - LEGv8 instruction frequency for (LEGv8 指令频率), 226
 - LEGv8 instructions (LEGv8 指令), 220-226
 - machine language (机器语言), 221
 - operands (操作数), 221
 - overflow (溢出), 206
 - packed format (压缩格式), 232
 - precision (精度), 243
 - procedure with two-dimensional matrices (带有二维矩阵的过程), 223-225
- programs compiling (程序编译), 222-225
- registers (寄存器), 226
- representation (表示 / 表达), 206-211
- rounding (舍入), 226
- sign and magnitude (符号与幅值), 206
- SSE2 architecture (SSE2 体系结构), 232-233
- subtraction (减法), 220
- underflow (下溢), 206
- units (单位), 227
- in x86 (在 x86 中), 233
- Floating-point addition (浮点数加法), 212-215
 - arithmetic unit block diagram (算术运算单元框图), 216
 - binary (二进制), 213
 - illustrated (举例说明), 214
 - instructions (指令), 220
 - steps (步骤), 212
- Floating-point multiplication (浮点乘法), 215-219
 - binary (二进制), 219
 - illustrated (举例说明), 218
 - instructions (指令), 220
 - significants (有效位), 215
 - steps (步骤), 215-217
- Flushing instructions (清空指令), 329, 330
 - exceptions and (异常), 340
- Forwarding (转发旁路), 316-328
 - ALU before (算术逻辑运算单元), 321
 - control (控制), 320
 - datapath for hazard resolution (用于解决冒险的数据通路), 323
 - defined (定义), 289
 - functioning (功能), 317
 - graphical representation (图解表示), 290
 - multiple results and (多个返回值), 292
 - multiplexors (多路选择器), 322
 - pipeline registers before (流水线寄存器), 321
 - with two instructions (带有两条指令), 289-290
- Fractions (分数), 206, 207
- Frame buffer (帧缓冲), 18
- Frame pointers (帧指针), 106
- Fully associative caches (全相联 cache), 468-469, 见 Caches block replacement strategies

choice of (选择), 422
 defined (定义), 417
 memory block location (存储器块定位), 417
 misses (缺失), 420
 Fully connected networks (全互连网络), 551
 Fused-multiply-add (FMA) operation (混合乘加操作), 228

G

Galois/Counter Mode (GCM) encryption (伽罗瓦 / 计数器模式加密), 488
 Gates (门), A-3, A-8
 AND (与), A-12
 delays (延迟), A-45-46
 NAND (与非), A-8
 NOR (或非), A-8, A-49
 Gather-scatter (聚集 - 分散), 527, 568
 General-purpose registers (通用寄存器), 154
 Generate (生成)
 defined (定义), A-39
 example (实例), A-44
 super (超级), A-40
 Gigabyte (G 字节), 6
 Global miss rates (全局缺失率), 430
 Global pointers (全局指针), 106
 GPU computing (GPU 计算), 见 Graphics processing units (GPU)
 Graphics displays (图像显示)
 computer hardware support (计算机硬件支持), 18
 LCD (液晶显示屏), 18
 Graphics processing units (GPU, 图形处理单元), 538-543, 见 GPU computing
 as accelerators (加速器), 538
 defined (定义), 46, 522
 memory (存储器), 538
 multilevel caches and (多级 cache), 538
 parallelism (并行性), 539
 perspective (观点), 543-545
 Gresham's Law (Gresham 法则), 248
 Grid computing (网格计算), 549
 GTX (GTX 公司), 280, 564-569
 Guard digits (保护位)
 defined (定义), 226

rounding with (舍入), 227

H

Halfwords (半字), 114
 Hamming, Richard, 434
 Hamming distance (汉明距离), 434
 Hamming Error Correction Code (ECC, 汉明 (检错) 纠错码), 434-435
 calculating (计算), 434-435
 Hard disks (硬盘)
 access times (访问时间), 23
 defined (定义), 23
 Hardware (硬件)
 as hierarchical layer (作为分层的层次), 13
 language of (……的语言), 14-16
 operations (操作), 63-67
 supporting procedures in (支持过程), 100-110
 synthesis (综合), A-21
 virtualizable (可虚拟化), 440
 Hardware description languages (硬件描述语言), 见 Verilog
 defined (定义), A-20
 using (使用), A-20-26
 VHDL (VHDL 硬件描述语言), A-20-21
 Hardware multithreading (硬件多线程), 530-533
 coarse-grained (粗粒度), 530
 options (选择), 531
 simultaneous (同时), 531
 Hardware-based speculation (基于硬件的推测), 352
 Hazard detection units (冒险检测单元), 324
 functions (函数), 324
 pipeline connections for (流水线连接), 327
 Hazards (冒险), 292-293, 328-336, 见 Pipelining control
 data (数据), 289, 316-328
 forwarding and (旁路), 323
 structural (结构的), 288-289, 305
 Heap (堆)
 allocating space on (在……上分配空间), 107-110
 defined (定义), 107
 Hexadecimal numbers (十六进制数), 83

- binary number conversion to (将二进制数转换为……), 83, 84
 - Hierarchy of memories (存储器层次结构), 12
 - High-level languages (高级语言), 14-16
 - benefits (收益), 16
 - importance (重要性), 16
 - Hit rate (命中率), 390
 - Hit time (命中时间)
 - cache performance and (cache 性能), 415-416
 - defined (定义), 390
 - Hit under miss (缺失下命中), 483
 - Hold time (保持时间), A-53
 - Human genome project (人类基因组计划), 4
- I
- I/O benchmarks (输入 / 输出基准测试程序), 见 Benchmarks
 - ID stage (译码级)
 - branch execution in (分支执行), 330, 331
 - load instructions (load 指令), 303
 - store instruction in (store 指令), 302
 - IEEE 754 floating-point standard (IEEE 754 浮点标准), 207-211, 208, 见 Floating point
 - rounding modes (舍入模式), 227
 - If statements (条件语句), 118
 - I-format (I 型), 87
 - If-then-else (If-then-else 语句), 94
 - Immediate addressing (立即数寻址), 120
 - Immediate instructions (立即数指令), 73
 - Imprecise interrupts (不精确中断), 342
 - Index-out-of-bounds check (索引越界检查), 98
 - In-order commit (按序提交), 351
 - Input devices (输入设备), 16
 - Inputs (输入), 273
 - Instruction count (指令计数), 36, 38
 - Instruction decode/register file read stage (指令译码 / 读寄存器文件级)
 - control line (控制线), 311-312
 - load instruction (load 指令), 300
 - store instruction (store 指令), 305
 - Instruction execution illustrations (指令执行图解)
 - clock cycle (时钟周期), 9
 - Instruction fetch stage (取指令级)
 - control line (控制线), 312
 - load instruction (load 指令), 300
 - store instruction (store 指令), 305
 - Instruction formats (指令格式), 161
 - ARMv7, 151
 - defined (定义), 83
 - I-type (I 型), 85
 - LEGv8, 151
 - MIPS, 151
 - R-type (R 型), 85, 273
 - x86, 161
 - Instruction latency (指令延迟), 367
 - Instruction mix (指令混合比), 39
 - Instruction set architecture (指令集体系结构)
 - ARM, 152-154
 - branch address calculation (分支地址计算), 266
 - defined (定义), 22, 52
 - history (历史), 173-174
 - maintaining (维护), 52
 - protection and (保护), 441
 - thread (线程), B-31-34
 - virtual machine support (虚拟机支持), 440-441
 - Instruction sets (指令集)
 - ARMv8, 171
 - design for pipelining (针对流水线设计), 228
 - LEGv8, 247
 - MIPS-32 (MIPS-32 指令集), 151
 - x86 growth (x86 增长), 170
 - Instruction-level parallelism (ILP, 指令级并行), 365, 见 Parallelism
 - defined (定义), 43, 344
 - exploitation increasing (开发增长), 354
 - and matrix multiply (矩阵乘法), 363-365
 - Instructions (指令), 60-174, 见 Arithmetic instructions, MIPS, Operands
 - add immediate (加立即数), 73
 - addition (加法), 190
 - arithmetic-logical (算术 - 逻辑), 263
 - ARM, 152-154
 - assembly (汇编), 66
 - cache-aware (cache 感知), 496
 - conditional branch (条件分支), 93, 94
 - conditional move (条件移动), 334

- core (核), 246
- data transfer (数据传输), 68
- decision-making (决策制定), 93-99
- defined (定义), 14, 62
- as electronic signals (电子信号), 82
- encoding (编码), 86
- exception and interrupt (异常和中断), A-80
- exclusive OR (异或), A-57
- fetching (读取), 265
- fields (字段), 83
- floating-point (x86)(浮点型), 232, 233
- floating-point (浮点), 220-221
- flushing (清空), 329, 330, 340
- immediate (立即数), 73
- introduction to (引言), 62-63
- left-to-right flow (从左向右流动), 298
- load, 69
- logical operations (逻辑运算), 90-93
- memory-reference (存储器引用/访问), 257
- multiplication (乘法), 197
- nop (空操作), 325-326
- performance (性能), 35-36
- pipeline sequence (流水顺序), 325
- remainder (余数), A-55
- representation in computer (在计算机中表示), 82-89
- restartable (可重启的), 462
- R-type (R 型), 263, 268
- store (存储), 72
- store exclusive register (STXR, 存储专用寄存器), 126
- subtraction (减法), 190
- vector (向量), 524
- as words (以字的形式), 62
- x86, 154-159
- Instructions per clock cycle (IPC, 每时钟周期执行的指令数), 343
- Integrated circuits (IC, 集成电路), 19, 见 specific chips
 - cost (成本/开销), 27
 - defined (定义), 25
 - manufacturing process (制造/生产过程), 26
 - very largD-scale (VLSI, 超大规模集成电路), 25
- Intel Core i7 (英特尔酷睿 i7), 46-49, 256, 517, 564-569
 - address translation for (地址转换), 483
 - architectural registers (体系结构级寄存器), 358
 - caches in (……中的 cache), 484
 - memory hierarchies of (存储器层次结构), 482-487
 - microarchitecture (微体系结构), 358
 - performance of (……的性能), 485-486
 - SPEC CPU benchmark (SPEC CPU 基准测试程序), 46-48
 - SPEC power benchmark (SPEC 功耗基准测试程序), 48-49
 - TLB hardware for (TLB 硬件), 483
- Intel Core i7 920 (英特尔酷睿 i7 920), 358-360
 - microarchitecture (微体系结构), 358
- Intel Core i7 (英特尔酷睿 i7), 960
 - benchmarking and rooflines of (基准测试程序和屋顶线), 564-569
- Intel Core i7 Pipelines (英特尔酷睿 i7 流水线), 354, 358-360
 - memory components (存储组件), 359
 - performance (性能), 361-362
 - program performance (程序性能), 362
 - specification (规格说明), 356
- Intel x86 microprocessors (英特尔 x86 微处理器)
 - clock rate and power for (时钟频率和功耗), 40
- Interleaving (交叉), 412
- Interrupt enable (中断使能), 461
- Interrupts (中断)
 - defined (定义), 207, 336
 - event types and (事件类型), 336
 - exceptions versus (异常与……), 336
 - imprecise (不精确), 342
 - precise (精确), 342
 - vectored (向量), 337
- Intrinsity FastMATH processor (Intrinsity FastMATH 处理器), 409-412
 - caches, 410
 - data miss rates (数据缺失率), 411, 421
 - read processing (读处理), 456
 - TLB (快表), 454-457

write-through processing (写直达处理), 456

Inverted page tables (反向页表), 451

Issue packets (发射包), 345

J

Java

bytecode (字节码), 136

characters in (字符), 113-115

goals (目标), 136

interpreting (解释), 136, 150

programs starting (程序开始), 136-137

sort algorithms (排序算法), 146

strings in (字符串), 113-115

translation hierarchy (翻译层次), 136

Java Virtual Machine (JVM, Java 虚拟机), 150

Jump instructions (跳转指令), 254

branch instruction versus (分支指令与……), 270

control and datapath for (控制和数据通路), 271

implementing (实现), 270

instruction format (指令格式), 270

Just In Time (JIT) compilers (及时编译器), 137, 576

K

Karnaugh maps (卡诺图), A-18

Kernel mode (核心态), 459

Kilobyte (千字节), 6

L

LAPACK (线性代数程序包), 243

Latches (锁存器)

D latch (D 锁存器), A-50, A-51

defined (定义), A-50

Latency (延迟)

instruction (指令), 367

pipeline (流水线), 297

use (使用), 346

LDUR (load register, load 寄存器指令), 64

LDURB (load byte, load 字节指令), 64

LDURH (load half, load 半字指令), 64

LDURSW (load signed word, load 无符号字指令), 64

LDXR (load exclusive register, load 专用寄存器

指令), 64, 122

Leaf procedures (叶过程), 见 Procedures

defined (定义), 104

example (实例), 113

Least recently used (LRU, 最近最少使用)

as block replacement strategy (作为块替换策略), 468-469

defined (定义), 423

pages (页), 448

Least significant bits (最低有效位)

defined (定义), 75

Left-to-right instruction flow (从左向右指令流), 298-299

LEGv8, 62, 64, 86

architecture (体系结构), 204

arithmetic core (算术运算核心), 246

arithmetic instructions (算术运算指令), 63

assembly instruction, mapping (汇编指令映射), 82-83

compiling C assignment statements into (将 C 赋值语句编译为……), 66

compiling complex C assignment into (将复杂 C 赋值语句编译为……), 66

control registers (控制寄存器), 461

divide in (……中的除法), 203

exceptions in (……中的异常), 337-338

fields (字段), 84-85

floating-point instructions (浮点指令), 220-221

instruction classes (指令分类), 173

instruction encoding (指令编码), 86, 122

instruction formats (指令格式), 124, 151

instruction set (指令集), 62, 171, 246, 247, 256-260,

machine language (机器语言), 88

memory addresses (存储器地址), 71

memory allocation for program and data (程序和数据的存储空间分配), 108

multiply in (……中的乘法), 197

operands (操作数), 64

Pseudo (伪码), 246

register conventions (寄存器), 109

static multiple issue with (静态多发射), 345-347

- Level-sensitive clocking (电平敏感时钟), A-73,
A-74-75
defined (定义), A-73
two-phase (二相), A-74
- Linkers (链接器), 131-134
defined (定义), 131
executable files (可执行文件), 131
steps (步骤), 131
using (使用), 131-134
- Linking object files (目标文件链接), 132-134
- Linpack (线性测试程序包), 554
- Liquid crystal displays (LCD, 液晶显示器), 18
- Load balancing (负载均衡), 521-522
- Load byte (加载字节), 167
- Load halfword (加载半字), 167
- Load instructions (load 指令), 见 Store instructions
base register (基址寄存器), 274
compiling with (用……编译), 71-72
datapath in operation for (……操作的数据通路), 279
defined (定义), 69
EX stage (执行级), 303
halfword unsigned (无符号半字), 114
ID stage (译码级), 302
IF stage (取指级), 302
load byte unsigned (加载字节无符号数), 79
load half (加载半字), 114
MEM stage (访存级), 304
move wide with keep (剩余位保持的宽立即数移位指令), 115
move wide with zeros (剩余位置零的宽立即数移位指令), 115
pipelined datapath in (……中的流水线数据通路), 307
signed (有符号), 79
unit for implementing (执行/实现单元), 267
unsigned (无符号), 79
WB stage (写回级), 304
- Load register (加载寄存器), 69, 72
- Loaders (装载器), 134
- Load-use data hazard (读后写数据冒险), 290, 329
- Load-use stalls (读后写阻塞), 329
- Local area networks (LAN, 局域网), 24, 见 Networks
- Local miss rates (局部缺失率), 430
- Locality (局部性),
principle (原则/原理), 388
spatial (空间的), 388, 391
temporal (时间的), 388, 391
- Lock synchronization (同步锁), 125
- Locks (锁), 534
- Logic (逻辑)
combinational (组合的), 262, A-5, A-9-20
components (组件/部件), 261
equations (方程式), A-7
minimization (最小化), A-18
programmable array (PAL, 可编程阵列), A-77
sequential (时序), A-5, A-55-57
two-level (两级), A-11-14
- Logical operations (逻辑操作), 90-93
AND (与), 91
ARM, 154
EOR (异或), 92
NOT (非), 91
shifts (移位), 90
- Lookup tables (LUT, 查找表), A-78
- Loop unrolling (循环展开),
for multiple-issue pipelines (多发射流水线), 348
register renaming and (寄存器重命名), 348
- Loops (循环), 95-96
conditional branches in (条件分支), 118
for (for 语句), 147
prediction and (预测), 333-334
test (测试), 147, 148
while compiling (while 语句编译), 95-96
- ## M
- Machine code (机器代码), 83
- Machine instructions (机器指令), 83
- Machine language (机器语言), 15
branch offset in (分支偏移), 119
decoding (译码), 121-124
defined (定义), 14, 83
floating-point (浮点), 221
illustrated (举例说明), 15

- LEGV8, 88
- SRAM (静态随机存储器), 21
- translating MIPS assembly language into (将 MIPS 汇编语言转换为……), 86
- Main memory (主存), 442, 见 Memory
 - defined (定义), 23
 - page tables (页表), 451
 - physical addresses (物理地址), 442
- Matrix multiply (矩阵乘法), 238-242, 569-571
- Mealy machine (米利有限状态机), 475, A-67, A-70, A-71
- Mean time to failure (MTTF, 平均无故障时间), 432
 - improving (改进), 433
 - versus AFR of disks (磁盘年失效率), 433-434
- Megabyte (兆字节), 6
- Memory (存储器)
 - addresses (地址), 79
 - affinity (关联), 562
 - bandwidth (带宽), 394-395, 411
 - cache, 21, 397-412, 412-431
 - CAM (计算机辅助制造), 422
 - defined (定义), 19
 - DRAM (动态随机存取存储器), 19, 393-394, A-62-64
 - flash (闪存), 23
 - GPU (图形处理单元), 538
 - instructions datapath for (指令数据通路), 267
 - main (主要), 23
 - nonvolatile (非易失), 22
 - operands (操作数), 68-69
 - read-only (ROM, 只读存储器), A-14-16
 - SDRAM (同步动态随机存取存储器), 393-394
 - secondary (二级/辅助的), 23
 - SRAM (静态随机存取存储器), A-57-61
 - stalls (阻塞), 414
 - technologies for building (制造技术), 24-28
 - virtual (虚拟), 441-465
 - volatile (易失), 22
- Memory access stage (访存阶段)
 - control line (控制线), 313
 - load instruction (load 指令), 303
 - store instruction (store 指令), 303
- Memory bandwidth (存储器带宽), 565, 573
- Memory consistency model (存储一致性模型), 481
- Memory elements (存储单元), A-49-57
 - clocked (时钟控制), A-50
 - D flip-flop (D 触发器), A-50, A-52
 - D latch (D 锁存器), A-51
 - DRAMs (动态随机存取存储器), A-62-66
 - flip-flop (触发器), A-50
 - hold time (保持时间), A-53
 - latch (锁存器), A-50
 - setup time (建立时间), A-52, A-53
 - SRAMs (静态随机存储器), A-57-61
 - unclocked (无时钟的), A-50
- Memory hierarchies (存储器层次结构), 559
 - of ARM cortex-A8 (ARM cortex-A8 芯片), 482-487
 - block (or line)(块(或行)), 390
 - cache performance (cache 性能), 412-431
 - caches, 397-431
 - common framework (一般框架), 465-472
 - defined (定义), 389
 - design challenges (设计挑战), 472
 - exploiting (开发), 386-513
 - of Intel core i7 (英特尔酷睿 i7 处理器), 482-487
 - level pairs (层对), 390
 - multiple levels (多层次), 389
 - overall operation of (全部操作), 457-458
 - parallelism and (并行), 477-481
 - pitfalls (陷阱), 491-495
 - program execution time and (程序执行时间), 431
 - quantitative design parameters (定量设计参数), 466
 - redundant arrays and inexpensive disks (廉价冗余磁盘阵列), 481
 - reliance on (利用/依赖), 390
 - structure (结构), 389
 - structure diagram (结构图), 392
 - variance (变化), 431
 - virtual memory (虚拟存储器), 441-465
- Memory rank (存储块), 395
- Memory technologies (存储技术), 392-397
 - disk memory (磁盘存储器), 395-397

- DRAM technology (动态随机存储器技术), 392, 393-395
- flash memory (闪存), 395
- SRAM technology (静态随机存储器技术), 392, 393
- Memory-stall clock cycles(存储器阻塞时钟周期), 413
- Message passing (消息传递)
 - defined (定义), 543
 - multiprocessors (多处理器), 543-548
- Metastability (亚稳态), A-75
- Microarchitectures (微体系结构), 358
 - Intel Core i7 920 (英特尔酷睿 i7 920), 358
- Microprocessors (微处理器)
 - design shift (设计转换), 517
 - multicore (多核), 8, 43, 517
- Migration (迁移), 479
- Million instructions per second (MIPS, 每秒百万条指令), 51
- Minterms (最小项)
 - defined (定义), A-12
- MIPS and ARMv8 (MIPS 和 ARMv8)
 - common features between (共同特征), 152
- MIPS-32 instruction set (MIPS-32 指令集), 151
- MIPS-64 instructions (MIPS-64 指令集), 151
- Miss penalty (失效 / 缺失代价)
 - defined (定义), 390
 - determination (决策), 405-406
 - multilevel caches reducing (多级 cache 减少), 424
- Miss rates (缺失率)
 - block size versus (块大小), 406
 - data cache (数据 cache), 467
 - defined (定义), 390
 - global (全局), 430
 - improvement (改进), 405-406
 - Intrinsity FastMATH processor (Intrinsity FastMATH 处理器), 411
 - local (本地), 430
 - miss sources (缺失原因), 471
 - split cache (分离 cache), 411
- Miss under miss (缺失下的缺失), 483
- MMX (MultiMedia eXtension, 多媒体扩展), 232
- Moore machines (摩尔机), 475, A-67, A-70, A-71
- Moore's law (摩尔定律), 11, 379, 538
- Most significant bit (最高有效位),
 - 1-bit ALU for (1 位 ALU), A-33
 - defined (定义), 75
- move (移动), 144
- MOVK (move wide with keep, 剩余位保持宽立即数移位指令), 64, 115
- MOVZ (move wide with zero, 剩余位置零宽立即数移位指令), 64, 115
- Multicore (多核), 533-537
- Multicore multiprocessors (多核多处理器), 8, 43
 - defined (定义), 8, 517
- Multilevel caches (多级 cache), 见 Caches
 - complications (并发), 430
 - defined (定义), 412, 430
 - miss penalty reducing (减少缺失代价), 424
 - performance of (性能), 424
 - summary (小结), 431-432
- Multimedia extensions (多媒体扩展)
 - vector versus (向量), 525-526
- Multiple dimension arrays (多维数组), 226
- Multiple instruction multiple data (MIMD, 多指令多数据), 574
 - defined (定义), 523, 524
- Multiple instruction single data (MISD, 多指令单数据), 523
- Multiple issue (多发射), 343-350
 - code scheduling (代码调度), 347-348
 - dynamic (动态), 343, 349-350
 - issue packets (发射包), 345
 - loop unrolling and (循环展开), 348
 - processors (处理器), 343, 344
 - static (静态), 343, 345-349
 - throughput and (吞吐率), 353
- Multiple processors (多个处理器), 569-571
- Multiple-clock-cycle pipeline diagrams (多时钟周期流水线图), 308
 - five instructions (五条指令), 309
 - illustrated (说明), 309
- Multiplexors (多路选择器), A-10
 - controls (控制), 473
 - in datapath (数据通路), 275

- defined (定义), 258
- forwarding control values (转发/旁路控制值), 322
- selector control (选择控制), 271
- two-input (双输入), A-10
- Multiplicand (被乘数), 192
- Multiplication (乘法), 191-197, 见 Arithmetic
 - fast hardware (快速硬件), 196
 - faster (更快), 196-197
 - first algorithm (第一种算法), 194
 - floating-point (浮点), 215-217
 - hardware (硬件), 192-194
 - instructions (指令), 197
 - in MIPS (在 MIPS 中), 197
 - multiplicand (被乘数), 197
 - multiplier (乘数), 197
 - operands (操作数), 197
 - product (乘积), 197
 - sequential version (顺序版本), 192-194
 - signed (有符号), 196
- Multiplier (乘数), 192
- Multiply algorithm (乘法算法), 195
- Multiprocessors (多处理器)
 - benchmarks (基准测试程序), 554-556
 - defined (定义), 516
 - historical perspective (历史观点), 577
 - message-passing (消息传递), 529-534
 - organization (组织), 515, 545
 - for performance (性能), 573
 - shared memory (共享存储器), 517, 533-537
 - software (软件), 517
 - UMA (一致性存储器访问), 534
- Multistage networks (多级网络), 551
- Multithreading (多线程)
 - coarse-grained (粗粒度), 530
 - defined (定义), 522
 - fine-grained (细粒度), 530
 - hardware (硬件), 530-533
 - simultaneous (SMT, 同时多线程), 531-533
- Mutual exclusion (互斥), 125
- N
- Name dependence (名字依赖), 348
- NAND gates (与非门), A-8
- NAS (NASA Advanced Supercomputing, NASA 先进超级计算), 556
- Negation shortcut (求相反数的简便方法), 79
- Nested procedures (嵌套过程), 104-105
 - compiling recursive procedure showing (递归过程编译), 104-105
- Network topologies (网络拓扑结构), 550-553
 - implementing (实现), 552
 - multistage (多级), 553
- Networks (网络), 23-24
 - advantages (优势), 23
 - bandwidth (带宽), 549
 - crossbar (交叉开关), 551
 - fully connected (全互连), 551
 - local area (LAN, 局域网), 24
 - multistage (多级), 551
 - wide area (WAN, 广域网), 24
- Newton's iteration (牛顿迭代), 226
- Next-state function (下一状态函数), 474, A-66
 - defined (定义), 474
- No write allocation (不写分配), 408
- Nonblocking assignment (非阻塞分配), A-24
- Nonblocking caches (非阻塞 cache), 355, 483
- Nonuniform memory access (NUMA, 非一致性存储器访问), 534
- Nonvolatile memory (非易失存储器), 22
- Nops (空操作), 326
- NOR gates (或非门), A-8
 - cross-coupled (交叉耦合), A-49
 - D latch implemented with (D 锁存器实现), A-51
- NOT operation (非操作), 91, A-6
- NOT-A-Number (NaN, 非数), 235, 236
- Numbers (数)
 - binary (二进制), 75
 - computer versus real-world (计算机与现实世界), 229
 - decimal (十进制), 75, 78
 - denormalized (非规格化), 230
 - hexadecimal (十六进制), 84
 - signed (有符号), 75-82
 - unsigned (无符号), 75-82
- NVIDIA GPU architecture (NVIDIA GPU 体系结

构), 539-541
 NVIDIA GTX, 280, 565, 566
 NVIDIA Tesla GPU, 564-569

O

Object files (目标文件), 132
 debugging information (调式信息), 131
 header (头部), 130
 linking (链接), 132-134
 relocation information (重定位信息), 130
 static data segment (静态数据段), 130
 symbol table (符号表), 130
 text segment (代码段), 130
 Object-oriented languages (面向对象语言), 见 Java
 brief history
 defined (定义), 150
 One's complement (反码), 82, A-29
 Opcodes (操作码)
 control line setting and (控制线设置), 276
 defined (定义), 84, 274
 OpenMP (Open MultiProcessing, 开放多处理,
 536, 556
 Operands (操作数), 67-72, 见 Instructions
 32-bit immediate (32 位立即数), 115-116
 adding (加), 189
 arithmetic instructions (算术运算指令), 67
 compiling assignment when in memory (编译在
 存储器中的赋值语句), 69
 constant (常量), 73-74
 division (除法), 197
 floating-point (浮点), 221
 LEGv8, 64
 memory (存储器), 68-69
 multiplication (乘), 191
 Operating systems (操作系统)
 defined (定义), 13
 encapsulation (封装), 22
 Operations (操作)
 atomic, implementing (原子实现), 126
 hardware (硬件), 63-67
 logical (逻辑), 90-93
 x86 integer (x86 整数), 157, 158
 Optimization (优化)

 compiler (编译器), 146
 manual (手册), 150
 OR operation (或操作), 91, A-6
 Out-of-order execution (乱序执行)
 defined (定义), 351
 performance complexity (性能复杂性), 430
 processors (处理器), 355
 Output devices (输出设备), 16
 Overflow (溢出)
 defined (定义), 76, 206
 detection (检测), 190
 exceptions (异常), 339
 floating-point (浮点), 207
 occurrence (事件 / 发生), 77
 saturation and (饱和), 191
 subtraction (减), 189

P

Packed floating-point format (打包浮点格式), 232
 Page faults (缺页故障), 448, 见 Virtual memory
 for data access (数据访问), 463
 defined (定义), 442
 handling (处理), 443, 461-464
 virtual address causing (虚拟地址导致), 457,
 458
 Page tables (页表), 468
 defined (定义), 446
 illustrated (举例说明), 449
 indexing (索引), 446
 inverted (反向), 451
 levels (级), 451
 main memory (主存储器), 451
 register (寄存器), 446
 storage reduction techniques (存储减少技术),
 451
 updating (更新), 446
 VMM (虚拟机), 463
 Pages (页), 见 Virtual memory
 defined (定义), 442
 dirty (脏), 452
 finding (查找), 446-447
 LRU (最近最少使用), 448
 offset (偏移), 443

- physical number (物理页号), 443
- placing (放置), 432-434
- Size (尺寸), 444
- virtual number (虚拟页号), 443
- Parallel execution (并行执行), 125
- Parallel processing programs (并行处理器程序), 518-523
 - creation difficulty (创建难度), 518-523
 - defined (定义), 516
 - for message passing (消息传递), 533
 - for shared address space (共享地址空间), 533-534
 - use of (使用), 573
- Parallel software (并行软件), 517
- Parallelism (并行性), 12, 43, 342-355
 - and computers arithmetic (计算机算术), 230-232
 - data-level (数据级), 246, 524
 - GPU and (图形处理单元), 538
 - instruction-level (指令级), 43, 342, 354
 - memory hierarchies and (存储器层次结构), 477-481
 - multicore and (多核), 533
 - multiple issue (多发射) 342-349
 - multithreading and (多线程), 531
 - performance benefits (性能收益), 44
 - process-level (进程级), 516
 - redundant arrays and inexpensive disks (廉价冗余磁盘阵列), 481
 - task-level (任务级), 516
- Paravirtualization (泛虚拟化), 495
- Parity (奇偶)
 - bits (位), 435
 - code (码), 434, A-64
- PARSEC (Princeton Application Repository for Shared Memory Computers, 普林斯顿共享内存计算机应用程序库), 556
- Pass transistor (传输晶体管), A-62
- PCI-Express (PCIe, 一种总线和接口标准), 553
- PC-relative addressing (PC 相对寻址), 118, 120
- Peak floating-point performance (峰值浮点性能), 558
- Pentium bug morality play (Pentium 缺陷闹剧), 244
- Performance (性能), 28-40
 - assessing (评估), 28
 - classic CPU equation (经典的 CPU 方程), 36-40
 - components (组件), 38
 - CPU (中央处理单元), 33-35
 - defining (定义), 29-32
 - equation using (方程使用), 36
 - improving (改进), 34-35
 - instructions (指令), 35-36
 - measuring (测量), 33-35
 - program (程序), 39-40
 - ratio (比率), 31
 - relative (相对的), 31-32
 - response time (响应时间), 30-31
 - throughput (吞吐率), 30-31
 - time measurement (时间测量), 32
- Personal computers (PC, 个人计算机), 7
 - defined (定义), 5
- Personal mobile device (PMD, 个人移动设备)
 - define (定义), 7
- Petabyte (千兆字节), 6
- Physical addresses (物理地址), 442
 - mapping to (映射), 442-443
 - space (空间), 533, 535
- Physically addressed caches (物理编址 cache), 458
- Pipeline registers (流水线寄存器)
 - before forwarding (转发前), 320
 - dependence (依赖), 319
 - forwarding unit selection (转发单元选择), 323
- Pipeline stall (流水线阻塞), 291
 - avoiding with code reordering (通过代码重排避免), 291
 - data hazards and (数据冒险), 324-328
 - insertion (插入), 326
 - load-use (读后写), 329
 - as solution to control hazards (作为控制冒险的解决方案), 293
- Pipelined branches (流水线分支), 331
- Pipelined control (流水线控制), 311-315, 见 Control
 - control lines (控制线), 311, 312
 - overview illustration (概述说明), 327

- specifying (指定), 312
- Pipelined datapaths (流水线数据通路), 297-315
 - with connected control signals (连接的控制信号), 315
 - with control signals (控制信号), 311-315
 - corrected (纠正), 307
 - illustrated (举例说明), 300
 - in load instruction stages (load 指令级), 307
- Pipelined dependencies (流水线依赖), 307
- Pipelines (流水线)
 - branch instruction impact (分支指令影响), 329
 - execute and address calculation stage (执行和地址计算级), 301, 303
 - five-stage (五级), 285, 301, 309
 - graphic representation (图示), 290, 307-311
 - instruction decode and register file read stage (指令译码和读寄存器文件级), 300, 303
 - instruction fetch stage (取指令级), 301, 303
 - instruction sequence (指令序列), 325
 - latency (延迟), 297
 - memory access stage (存储器访问级), 301, 303
 - multiple-clock-cycle diagrams (多时钟周期图), 308
 - performance bottlenecks (性能瓶颈), 353
 - single-clock-cycle diagrams (单时钟周期图), 308
 - stages (级), 285
 - static two-issue (静态双发射), 345
 - write-back stage (写回级), 301, 305
- Pipelining (流水线), 12, 283-297
 - advanced (先进/高级), 354-355
 - benefits (收益), 283
 - control hazards (控制冒险), 292-293
 - data hazards (数据冒险), 289
 - exceptions and (异常), 338-342
 - execution time and (执行时间), 297
 - fallacies (谬误), 366-367
 - hazards (冒险), 288
 - instruction set design for (指令集设计), 288
 - laundry analogy (洗衣类比), 284
 - overview (概述), 283-297
 - paradox (悖论), 285
 - performance improvement (性能提高), 288
 - pitfall (陷阱), 366-367
 - simultaneous executing instructions (同时执行指令), 297
 - speed-up formula (加速比公式), 285
 - structural hazards (结构冒险), 288, 305
 - summary (小结), 296
 - throughput and (吞吐率), 297
- pitfalls (陷阱), 见 Fallacies
 - address space extension (地址空间扩展), 493
 - arithmetic (算术), 242-245
 - associativity (相联度), 492
 - defined (定义), 49
 - ignoring memory system behavior (忽略存储器系统行为), 491
 - memory hierarchies (存储器层次结构), 491-495
 - out-of-order processor evaluation (乱序处理器评估), 492
 - performance equation subset (性能方程子集), 50-51
 - pipelining (流水线), 366-367
 - pointer to automatic variables (指向自动变量的指针), 171
 - sequential word addresses (连续字地址), 171
 - simulating cache (模拟 cache), 491
 - software development with multiprocessors (多处理器软件发展), 570
 - VMM implementation (VMM 实现), 495
- Pixels (像素), 18
- Pointers (指针),
 - arrays versus (数组), 146-150
 - frame (帧), 106
 - global (全局), 106
 - incrementing (递增), 148
 - stack (栈), 101, 105
- Pop (弹出), 101
- Power (功率/功耗)
 - clock rate and (时钟频率), 40
 - critical nature of (关键特性), 53
 - efficiency (效率), 354-355
 - relative (相对), 41
- Precise interrupts (精确中断), 342
- Prediction (预测), 12
 - 2-bit scheme (2 位方法), 333

- accuracy (准确的), 333
- dynamic branch (动态分支), 331-333
- loops and (循环), 333-334
- steady-state (稳态), 333
- Prefetching (预取), 496, 560
- Procedure calls (过程调用)
 - preservation across (在……期间保持不变), 106
- Procedures (过程), 100-110
 - compiling (编译), 102
 - compiling showing nested procedure linking (编译显示嵌套过程链接), 102-104
 - execution steps (执行步骤), 100
 - frames (帧), 106
 - leaf (叶), 104
 - nested (嵌套), 104-106
 - recursive (递归), 108
 - for setting arrays to zero (设置数组为 0), 147
 - sort (排序), 140-145
 - strcpy (字符串复制 (函数)), 112-113
 - string copy (字符串复制), 112-113
 - swap (交换), 138
- Process identifiers (进程标识符), 460
- Process-level parallelism (进程级并行), 516
- Processors (处理器), 254-368
 - as cores (作为核心), 43
 - control (控制), 19
 - datapath (数据通路), 19
 - defined (定义), 17, 19
 - dynamic multiple-issue (动态多发射), 343
 - multiple-issue (多发射), 343
 - out-of-order execution (乱序执行), 355, 430
 - performance growth (性能提高), 44
 - speculation (推测), 344-345
 - static multiple-issue (静态多发射), 343, 345-349
 - superscalar (超标量), 349, 531-532
 - technologies for building (制造技术), 24-28
 - two-issue (双发射), 346
 - vector (向量), 523-524
 - VLIW (超长指令字), 345
- Product (乘积), 192
- Product of sums (和之积), A-11
- Program counters (PC, 程序计数器), 263
 - changing with conditional branch (条件分支转换), 334
 - defined (定义), 101, 263
 - exception (异常), 459, 461
 - incrementing (递增), 263, 265
 - instruction updates (指令更新), 300
- Program performance (程序性能)
 - elements affecting (元素影响), 39
 - understanding (理解), 9
- Programmable array logic (PAL, 可编程阵列逻辑), A-77
- Programmable logic array (PLA, 可编程逻辑阵列),
 - component dots illustration (组件点说明), A-16
 - defined (定义), A-12
 - example (示例), A-13-14
 - illustrated (说明), A-13
 - ROMs and (只读存储器), A-15-16
 - truth table implementation (真值表实现), A-13
- Programmable logic device (PLD, 可编程逻辑器件), A-77
- Programmable ROM (PROM, 可编程 ROM), A-14
- Programming languages (编程语言), 见 specific languages
 - object-oriented (面向对象), 150
 - variables (变量), 67
- Programs (程序)
 - assembly language (汇编语言), 129
 - Java starting (Java 开始), 136-137
 - parallel processing (并行处理), 516-523
 - starting (开始), 128-137
 - translating (翻译), 128-137
- Propagate (传输)
 - defined (定义), A-39
 - example (示例), A-44
 - super (超级), A-40
- Protection (保护)
 - defined (定义), 442
 - implementing (实现), 459-460
 - VMs for (虚拟机), 438
- Pseudo MIPS (伪 MIPS)
 - defined (定义), 246

instruction set (指令集), 248

Pseudoinstructions (伪指令)

defined (定义), 129

summary (小结), 130

Pthreads (POSIX thread, POSIX 线程 (标准)), 556

Push (压栈)

defined (定义), 101

using (使用), 104

Q

Quad word (四字), 158

Quicksort (快速排序), 425, 426

Quotient (商), 198

R

Race (竞争), A-72

Radix sort (基数排序), 425, 426

RAID, 见 Redundant arrays of inexpensive disks

RAM (随机访问存储器), 9

Raster refresh buffer (光栅刷新缓冲区), 18

Ray casting (RC, 光线投射 / 射线造型法), 568

Read-only memories (ROM, 只读存储器), A-14-16

logic function encoding (逻辑函数编码), A-15

PLAs and (可编程逻辑阵列), A-15-16

programmable (PROM, 可编程 ROM), A-14

Read-stall cycles (读阻塞周期), 413

Read-write head (读写头), 395

Receive message routine (接收消息例程), 545

Recursive procedures (递归过程), 108, 见 Procedures

clone invocation (克隆调用), 104

Reduction (归约), 535

Reference bit (引用位), 450

References (引用)

absolute (绝对), 126

Register (寄存器), 31, 74, 102, 175

Register addressing (寄存器寻址), 120

Register files (寄存器文件), A-49, A-53-55

defined (定义), 264, A-49, A-53

in behavioral Verilog (Verilog 行为), A-56

single (单个), 269

two read ports implementation (两个读端口的

实现), A-54

with two read ports/one write port (两个读端口 / 一个写端口), A-54

write port implementation (写端口实现), A-55

Registers (寄存器), 156, 157-158

architectural (结构), 336-342

base (基), 69

clock cycle time and (时钟周期), 67

compiling C assignment with (编译 C 赋值语句), 68

defined (定义), 67

destination (目的), 85, 274

floating-point (浮点), 226

left half (左半边), 301

mapping (映射), 82

LEGV8 conventions (LEGV8 约定), 108

number specification (数据手册 / 数据说明书), 264

page table (页表), 446

pipeline (流水线), 319, 321, 323

primitives (基本元素), 67

renaming (重命名), 348

right half (右半边), 301

spilling (溢出), 72

Status (状态), 337

temporary (暂时), 68, 102

variables (变量), 67

Relative performance (相对性能), 31-32

Relative power (相对功率 / 功耗), 41

Reliability (可靠性), 432

Remainder (余数)

defined (定义), 198

Reorder buffers (重排序缓冲区), 355

Replication (复制), 479

Requested word first (请求字优先), 406

Request-level parallelism (请求级并行), 548

Reservation stations (保留站)

buffering operands in (在……中缓冲操作数), 350

defined (定义), 350

Response time (响应时间), 30-31

Restartable instructions (重启指令), 462

Return address (返回地址), 100

Return from exception (ERET, 从异常中返回), 459

R-format (R 型), 274

ALU operations (ALU 操作), 265

defined (定义), 86

Ripple carry (行波进位)

adder (加法器), A-29

carry lookahead speed versus (先行进位速度), A-45-46

Roofline model (Roofline 模型), 558-559, 560, 561

with ceilings (天花板), 561

computational roofline (可计算的屋顶线), 559

illustrated (举例说明), 557

Opteron generations (皓龙处理器时代), 558

with overlapping areas shaded (重叠部分阴影表示), 563

peak floating-point performance (峰值浮点性能), 562

peak memory performance (峰值存储性能), 562

with two kernels (两个核心程序), 563

Rotational delay (旋转延迟), 见 Rotational latency

Rotational latency (旋转延迟), 397

Rounding (舍入), 226

accurate (精确), 226

bits (位), 228

with guard digits (带保护位), 227

IEEE 754 modes (IEEE 754 格式), 227

Row-major order (行优先顺序), 225, 427

R-type instructions (R 型指令), 264

datapath for (数据通路), 276

datapath in operation for (……操作的数据通路), 278

S

Saturation (饱和), 191

SCALAPAK (可扩展线性代数库), 244

Scaling (缩放 / 扩展)

strong (强比例缩放), 521

weak (弱比例缩放), 521

Scientific notation (科学记数法)

adding numbers in (加入数), 213

defined (定义), 205

for reals (对于实数), 205

Search engines (搜索引擎), 4

Secondary memory (二级 / 辅助存储器), 23

Sectors (扇区), 395

Seek (寻找), 396

Segmentation (分段), 445

Selector values (选择信号值), A-10

Semiconductors (半导体), 25

Send message routine (发送消息例程), 545

Sensitivity list (敏感列表), A-24

Sequential logic (时序逻辑), A-5

Servers (服务器), 见 Desktop and server RISC

cost and capability (成本和功能), 5

Service accomplishment (服务完成), 432

Service interruption (服务中断), 432

Set instructions (设置指令), 97

Set-associative caches (组相连 cache), 417, 见

Caches

address portions (地址部分), 421

block replacement strategies (块替换策略), 468

choice of (选择), 467

four-way (四路), 418, 421

memory-block location (存储器块位置), 417

misses (缺失), 419-420

n-way (n 路), 417

two-way (2 路), 418

Setup time (建立时间), A-52, A-53

Shared memory (共享存储器 / 内存), 见 Memory

Shared memory multiprocessors (SMP, 共享内存多处理器), 531-535

defined (定义), 517, 531

single physical address space (单一物理地址空间), 531

synchronization (同步), 534

Shift amount (移位数量), 84

Shift instructions (移位指令), 90

Sign and magnitude (符号和幅值), 206

Sign bit (符号位), 78

Sign extension (符号扩展), 266

defined (定义), 78

shortcut (简便方法), 80

Signals (信号)

asserted (有效), 262, A-4

- control (控制), 262, 274-275
- deasserted (无效), 262, A-4
- Signed division (有符号数除法), 201-202
- Signed multiplication (有符号数乘法), 196
- Signed numbers (有符号数), 75-82
 - sign and magnitude (符号和幅值), 77
 - treating as unsigned (作为无符号数处理), 98
- Significands (有效位), 207
 - addition (额外的), 212
 - multiplication (乘), 215
- Silicon (硅), 25
 - as key hardware technology (作为关键硬件技术), 53
 - crystal ingot (晶锭), 26
 - defined (定义), 26
 - wafers (晶圆), 26
- Silicon crystal ingot (硅晶锭), 26
- SIMD (Single Instruction Multiple Data, 单指令多数据), 522, 574
 - vector architecture (向量结构), 524-525
 - in x86 (在 x86 中), 524
- Simple programmable logic devices (SPLD, 简单可编程逻辑器件), A-77
- Simplicity (简化), 171
- Simultaneous multithreading (SMT, 同时多线程), 531-533
 - support (支持), 531
 - thread-level parallelism (线程级并行), 531
 - unused issue slots (未使用的发射槽), 531
- Single error correcting/Double error correcting (SEC/DEC, 单纠错/双纠错), 434-436
- Single instruction single data (SISD, 单指令单数据), 523
- Single precision (单精度), 见 Double precision
 - binary representation (二进制表示), 210
 - defined (定义), 207
- Single-clock-cycle pipeline diagrams (单时钟周期流水图), 308
 - illustrated (说明), 310
- Single-cycle datapaths (单周期数据通路), 见 Datapaths
 - illustrated (说明), 298
 - instruction execution (指令执行), 299
- Single-cycle implementation control function for (单周期实现控制功能), 281
 - defined (定义), 281
 - nonpipelined execution versus pipelined execution (非流水执行和流水执行), 287
 - non-use of (未使用), 284
 - penalty (代价), 283
 - pipelined performance versus (流水线性能), 285
- Smart phones (智能手机), 7
- Snooping protocol (监听协议), 479-481
- Software optimization (软件优化)
 - via blocking (通过阻塞), 427-432
- Sort algorithms (排序算法), 146
- Software (软件)
 - layers (层), 13
 - multiprocessor (多处理器), 516
 - parallel (并行), 517
 - as service (服务), 7, 547, 574
 - system terms (系统), 13
- Sort procedure (排序过程), 140-144, 见 Procedures
 - code for body (程序体代码), 140-142
 - full procedure (全过程), 143-144
 - passing parameters in (传递参数), 143
 - preserving registers in (将寄存器保存在), 143
 - procedure call (过程调用), 143
 - register allocation for (寄存器分配), 140
- Space allocation (空间分配)
 - on heap (堆上), 107-110
 - on stack (栈上), 106
- Spatial locality (空间局部性), 388
 - large block exploitation of (大块开发), 405
 - tendency (趋势), 392
- SPEC
 - CPU benchmark (CPU 基准测试程序), 46-48
 - power benchmark (功耗基准测试程序), 48-49
 - SPECrate (SPEC 分值), 554
 - SPECratio (SPEC 分值), 47
- Speculation (推测), 344-345
 - hardware-based (基于硬件的), 352
 - implementation (实现), 344
 - performance and (性能), 344
 - problems (问题), 344
 - recovery mechanism (恢复机制), 344

- Speed-up challenge (加速挑战), 518
 - balancing load (负载均衡), 518-519
 - bigger problem (更大的问题), 520-521
- Spilling registers (溢出寄存器), 72, 101
- Split algorithm (分割算法), 568
- Split caches (分离 cache), 411
- Stack pointers (栈指针)
 - adjustment (调整), 104
 - defined (定义), 101
 - values (值), 104
- Stacks (栈)
 - allocating space on (分配栈空间), 106
 - for arguments (参数), 145
 - defined (定义), 101
 - pop (弹栈), 101
 - push (压栈), 101, 104
- Stalls (阻塞), 291
 - as solution to control hazard (作为控制冒险的解决方案), 292
 - avoiding with code reordering (通过代码重排以避免), 291
 - data hazards and (数据冒险), 324-328
 - insertion into pipeline (插入流水线), 326
 - load-use (读后写), 329
 - memory (存储器), 414
 - write-back scheme (回写方案), 413
 - write buffer (写缓冲区), 413
- State (状态)
 - in 2-bit prediction scheme (2 位预测方案), 333
 - assignment (分配), A-69
 - exception saving/restoring (异常保存 / 恢复), 462
 - logic components (逻辑部件), 261
 - specification of (规范), 446
- State elements (状态元素)
 - clock and (时钟), 262
 - combinational logic and (组合逻辑), 262
 - defined (定义), 260, A-47
 - inputs (输入), 261
 - in storing/accessing instructions (存储 / 访问指令), 264
 - register file (寄存器文件), A-49
- Static branch prediction (静态分支预测), 345
- Static data (静态数据)
 - segment (段), 107
- Static multiple-issue processors (静态多发射处理器), 343, 345-349, 见 Multiple issue
 - control hazards and (控制冒险), 345
 - instruction sets (指令集), 345
 - with LEGv8 ISA (LEGv8 指令集体系结构), 345-348
- Static random access memories (SRAM, 静态随机访问存储器), 392, 393, A-57-61
 - array organization (数组 / 阵列组织), A-61
 - basic structure (基本结构), A-60
 - defined (定义), 21, A-57
 - fixed access time (固定访问时间), A-57
 - large (大), A-58
 - read/write initiation (读 / 写启动), A-58
 - synchronous (SSRAM, 同步 SRAM), A-59
 - three-state buffers (三态缓冲器), A-58, A-59
- Static variables (静态变量), 106
- Steady-state prediction (稳态预测), 333
- Sticky bits (粘贴位), 228
- Store buffers (存储缓冲区), 355
- Store instructions (store 指令), 见 Load instructions
 - base register (基址寄存器), 274
 - compiling with (用……编译), 71
 - conditional (有条件的), 126
 - defined (定义), 71
 - EX stage (执行级), 305
 - ID stage (译码级), 302
 - IF stage (取指级), 302
 - instruction dependency (指令依赖), 323
 - MEM stage (访存级), 304
 - unit for implementing (实现单元), 267
 - WB stage (写回级), 304
- Store register (存储寄存器 / 写寄存器), 72
- Stored program concept (存储程序概念), 63
 - as computer principle (计算机原理), 88
 - illustrated (说明), 89
 - principles (原理), 171
- Strcpy procedure (字符串复制过程), 112-113, 见 Procedures
 - as leaf procedure (作为叶子过程), 113
 - pointers (指针), 113
- Stream benchmark (流基准测试程序), 564

Streaming SIMD Extension 2 (SSE2, 单指令多数据流扩展)
 floating-point architecture (浮点体系结构), 232
 Streaming SIMD Extensions (SSE) and advanced vector extensions in x86 (x86 中的单指令多数据流扩展和高级向量扩展), 232-233
 Strings (字符串)
 defined (定义), 111
 in Java (在 Java 中), 113-115
 representation (表示), 111
 Strip mining (条带开发), 526
 Strong scaling (强比例), 521
 Structural hazards (结构冒险), 288-289, 305
 STUR (store register, 存储寄存器), 64
 STURB (store byte, 存储字节), 64
 STURH (store half, 存储半字), 64
 STURW (store word, 存储字), 64
 STXR (store exclusive register, 存储专用寄存器), 64, 126
 SUB (Subtract, 减法), 64
 SUBI (subtract immediate, 减立即数), 64
 SUBIS (subtract immediate and set flags, 减立即数并设置标志位), 64
 SUBS (减并设置标志位), 64
 Subnormals (亚规格化), 230
 Subtraction (减法), 188-191, 见 Arithmetic
 binary (二进制), 188-189
 floating-point (浮点), 220
 instructions (指令), A-56-57
 negative number (负数), 190
 overflow (溢出), 190
 Subword parallelism (子字并行), 230-232, 365
 and matrix multiply (矩阵乘法), 238-242
 Sum of products (积之和), A-11, A-12
 Supercomputers (超级计算机)
 defined (定义), 5
 Superscalars (超标量)
 defined (定义), 349
 dynamic pipeline scheduling (动态流水线调度), 349
 multithreading options (多线程选项), 516
 Swap procedure (交换过程), 138, 见 Procedures
 body code (程序体代码), 138
 full (满), 139-140, 143-145

 register allocation (寄存器分配), 138
 Swap space (交换空间), 448
 Symbol tables (符号表), 130
 Synchronization (同步), 125-127, 568
 defined (定义), 534
 lock (锁), 125
 overhead reducing (开销减少), 44-45
 unlock (解锁), 125
 Synchronizers (同步器)
 defined (定义), A-75
 failure (故障/失效), A-76
 from D flip-flop (D 触发器), A-75
 Synchronous DRAM (SRAM, 同步 DRAM), 393-394, A-59, A-64
 Synchronous SRAM (SSRAM, 同步 SRAM), A-59
 Synchronous system (同步系统), A-47
 System calls (系统调用)
 defined (定义), 459
 Systems software (系统软件), 13

T

Tablets (表), 7
 Tags (标记)
 defined (定义), 398
 in locating block (定位块), 421
 page tables and (页表), 448
 size of (尺寸), 423
 Tail call (尾调用), 109
 Task identifiers (任务标识符), 460
 Task-level parallelism (任务级并行), 516
 Tebibyte (TiB, 太字节), 5
 Temporal locality (时间局部性), 388
 tendency (趋势), 392
 Temporary registers (临时寄存器), 68, 102
 Terabyte (TB, 太字节), 6
 defined (定义), 5
 Thrashing (抖动), 464
 Thread blocks (线程块), 542
 Three Cs model (3C 模型), 459-461
 ThreD-state buffers (三态缓冲器), A-58, A-59
 Throughput (吞吐量)
 defined (定义), 30-31
 multiple issue and (多发射), 342

- pipelining and (流水线), 286, 342
 - Timing (时序)
 - asynchronous inputs (异步输入), A-75-76
 - level-sensitive (电平敏感), A-74-75
 - methodologies (方法), A-71-76
 - two-phase (两相), A-74
 - TLB misses (TLB 缺失), 453, 见 Translation-look aside buffer
 - handling (处理), 461-465
 - occurrence (事件 / 发生), 461
 - problem (问题), 464
 - Touchscreen (触摸屏), 19
 - Tournament branch predictors (竞争分支预测器), 334
 - Tracks (磁道), 395-396
 - Transfer time (传输时间), 397
 - Transistors (晶体管), 25
 - Translation Table Base Register (TTBR, 转换页表基址寄存器), 449
 - Translation-Lookaside buffer (TLB, 快表), 452-454, 见 TLB misses
 - associativities (相关性), 454
 - illustrated (说明), 453
 - integration (集成), 457
 - Intrinsity FastMATH, 454-457
 - typical values (典型值), 454
 - Truth tables (真值表), A-5
 - for control bits (控制位), 272
 - defined (定义), 272
 - example (例子), A-5
 - PLA implementation (PLA 实现), A-13
 - Two's complement representation (补码表示), 77-78
 - advantage (优势), 78
 - negation shortcut (求反简便方法), 79-80
 - rule (规则), 81
 - sign extension shortcut (符号扩展简便方法), 80-81
 - Two-level logic (两级逻辑), A-11-14
 - Two-phase clocking (两相时钟), A-74
- ## U
- Unconditional branches (无条件分支), 94
 - Underflow (下溢), 206
 - Unicode (统一码)
 - alphabets (字母表), 113
 - defined (定义), 113
 - example alphabets (示例字母), 114
 - Uniform memory access (UMA, 一致性存储器访问), 534
 - multiprocessors (多处理器), 534
 - Units (单元)
 - commit (提交确认), 350, 355
 - control (控制), 259, 271-273
 - defined (定义), 227
 - floating point (浮点), 227
 - hazard detection (冒险检测), 324, 327-328
 - for load/store implementation (load/store 实现), 267
 - Unlock synchronization (同步解锁), 125
 - Unscaled signed immediate off set (不可扩展立即数偏移), 166
 - Unsigned numbers (无符号数), 75-82
 - Use latency (使用延迟)
 - defined (定义), 346
 - one-instruction (一条指令), 346
- ## V
- Vacuum tubes (真空管), 25
 - Valid bit (有效位), 400
 - Variables (变量)
 - C language (C 语言), 106
 - programming language (编程语言), 67
 - register (寄存器), 67
 - static (静态), 106
 - storage class (存储类别), 106
 - type (类型), 106
 - Vector lanes (向量通道), 526
 - Vector processors (向量处理器), 523-524, 见 Processors
 - conventional code comparison (传统代码比较), 525-526
 - instructions (指令), 524
 - multimedia extensions and (多媒体扩展), 524-525
 - scalar versus (标量), 526-527
 - Vectored interrupts (向量中断), 337
 - Verilog (一种硬件描述语言)

behavioral definition of MIPS ALU (MIPS ALU 的行为级定义), A-25

behavioral specification (行为级定义), A-21

blocking assignment (阻塞赋值), A-24

combinational logic (组合逻辑), A-21-22

datatypes (数据类型), A-21-22

defined (定义), A-20

LEGV8 ALU definition in (LEGV8 ALU 定义), A-34-37

modules (模块), A-23

nonblocking assignment (非阻塞赋值), A-24

operators (运算符), A-22

program structure (程序结构), A-23

reg, A-21-22

sensitivity list (敏感列表), A-24

sequential logic specification (时序逻辑定义), A-55-57

structural specification (结构定义), A-21

wire (线型), A-21-22

Very large-scale integrated (VLSI) circuits (超大规模集成电路), 25

Very Long Instruction Word (VLIW, 超长指令字)

defined (定义), 345

processors (处理器), 345

VHDL (一种硬件描述语言), A-20-21

Virtual addresses (虚拟地址)

causing page faults (导致缺页), 462

defined (定义), 442

mapping from (从……映射), 442-443

size (尺寸), 444

Virtual machine monitors (VMM, 虚拟机监视器)

defined (定义), 438

implementing (实现), 494

laissez-faire attitude (放任的态度), 494

page tables (页表), 463

in performance improvement (在提高性能方面), 441

requirements (要求), 440

Virtual machines (VM, 虚拟机), 438-441

benefits (收益), 438

illusion (描述), 463

instruction set architecture support (指令集体系结构支持), 441

performance improvement (性能提高), 441

for protection improvement (防护的改进), 438

Virtual memory (虚拟存储器), 441-465, 见 Pages

address translation (地址转换), 443, 452-454

integration (集成), 457-459

for large virtual addresses (大虚拟地址), 450-451

mechanism (机制), 464

motivations (动机), 427-442

page faults (缺页), 442, 448

protection implementation (保护实现), 459-460

segmentation (分段), 445

summary (小结), 463

virtualization of (虚拟化), 463

writes (写), 452

Virtualizable hardware (虚拟化硬件), 440

Virtually addressed caches (虚拟地址 cache), 458

Volatile memory (易失性存储器), 22

W

Wafers (晶圆), 26

defects (瑕疵), 26

dies (管芯), 26-27

yield (产量), 27

Warehouse Scale Computers (WSC, 仓储式计算机), 7, 545-550, 574

Weak scaling (弱比例缩放), 21

Wear levelling (损耗均衡), 395

While loops (while 循环), 95

Wide area networks (WAN, 广域网), 24, 见 Networks

Words (字)

accessing (访问), 69

defined (定义), 67

double (双(字)), 158

load, 69, 71

quad (四(字)), 158

store, 71

Working set (工作集), 464

World Wide Web (万维网), 4

Worst-case delay (最坏情况下的延迟), 283

Write buffers (写缓冲)

defined (定义), 408

stalls (阻塞), 413

write-back cache (写回式 cache), 409

Write invalidate protocols (写无效协议), 479

Write serialization (写串行化), 479

Write-back caches (写回式 cache), 见 Caches

advantages (优点), 469

complexity (复杂性), 409

defined (定义), 408, 469

stalls (阻塞), 413

write buffers (写缓冲), 409

Write-back stage (写回级)

control line (控制线), 313

load instruction (load 指令), 303

store instruction (store 指令), 305

Writes (写)

complications (困难), 408

expense (开销), 464

handling (处理), 407-409

memory hierarchy handling of (存储器层次结构的处理), 469-470

schemes (方案), 408

virtual memory (虚拟存储器), 451

write-back cache (写回式 cache), 408, 409

write-through cache (写直达 cache), 408, 409

Write-stall cycles (写阻塞周期), 414

Write-through caches (写直达 cache), 见 Caches

advantages (优点), 469

defined (定义), 407, 469

tag mismatch (标记不匹配), 408

X

x86, 154-162

Advanced Vector Extensions in (高级向量扩展), 232

conclusion (结论), 162

data addressing modes (数据寻址方式), 157-158

evolution (评估), 154-157

first address specifier encoding (首地址符编码), 162

instruction encoding (指令编码), 161-162

instruction formats (指令格式), 161

instruction set growth (指令集增长), 170

instruction types (指令类型), 160

integer operations (整数运算), 158-160

registers (寄存器), 157

SIMD in (单指令流多数据流), 522

Streaming SIMD Extensions in (流处理 SIMD 扩展), 232-233

typical instructions/functions (典型指令/功能), 161

typical operations (典型操作), 161

XMM (AMD64 中的一种寄存器), 232

Y

Yahoo! Cloud Serving Benchmark (YCSB, 雅虎云服务基准测试程序), 556

Yield (收益), 27

YMM (Intel 将寄存器宽度再次扩充的一种寄存器), 232

Z

Zettabyte (泽字节), 6

推荐阅读



计算机体系结构：量化研究方法（英文版·第5版）

作者：John L. Hennessy 等 ISBN: 978-7-111-36458-0 定价：138.00元

“本书之所以会成为不朽的经典之作，是因为每次再版都不仅仅是一次更新补充，而是全面修订，针对这个激动人心且快速变化的领域，给予我们最及时的信息和权威的解读。即便对于我这样已从业二十多年的人来说，再次阅读本书时，依旧自觉学无止境，感佩于两位卓越大师的渊博学识和深厚功底。”

—— Luiz Andre Barroso, Google公司

“本书是一部经典，犹如美酒，历久而弥醇。只有Hennessy和Patterson才可能接触到谷歌、亚马逊、微软等云计算与互联网规模的应用提供商的内部人士，对这一重要领域，业内的介绍材料无出其右。”

—— James Hamilton, Amazon Web服务

“《计算机体系结构：量化研究方法（第5版）》继续发扬传统，为学习计算机体系结构的学生提供了当前计算平台的最新信息，使他们能够洞悉体系结构，便于设计未来系统。这一版的亮点在于大幅修订了数据级并行那一章，用传统的体系结构术语清晰地解读了GPU体系结构。”

—— Krste Asanovi, 加州大学伯克利分校

“本书已经成为一本经典教科书了，这一版突出介绍了各种显式并行技术（数据、线程、请求）的兴起，各用整整一章来描述。数据并行一章尤为夺目：通过向量SIMD、指令级SIMD和GPU的对比，避开每种体系结构的专用术语，揭示了这些体系结构之间的相似与区别。”

—— Kunle Olukotun, 斯坦福大学

“这一版新增加了一章非常丰富的内容，用来讨论向量、SIMD和GPU体系结构中的数据级并行技术。它解释了应用于大众市场的GPU内部的关键体系结构概念，给出这些概念与传统术语的对应关系，并与向量和SIMD体系结构进行了对比。这一内容非常及时，与业内转向GPU并行计算的潮流相适应。《计算机体系结构：量化研究方法》继续独领风骚，全面地介绍了体系结构方面的重大新进展！”

—— John Nickolls, NVIDIA

推荐阅读



深入理解计算机系统（原书第3版）

作者：[美] 兰德尔 E. 布莱恩特 等 译者：龚奕利 等 书号：978-7-111-54493-7 定价：139.00元

理解计算机系统首选书目，10余万程序员的选择

卡内基-梅隆大学、北京大学、清华大学、上海交通大学等国内外众多知名高校选用指定教材

从程序员视角全面剖析的实现细节，使读者深刻理解程序的行为，将所有计算机系统的相关知识融会贯通
新版本全面基于X86-64位处理器

基于该教材的北大“计算机系统导论”课程实施已有五年，得到了学生的广泛赞誉，学生们通过这门课程的学习建立了完整的计算机系统的知识体系和整体知识框架，养成了良好的编程习惯并获得了编写高性能、可移植和健壮的程序的能力，奠定了后续学习操作系统、编译、计算机体系结构等专业课程的基础。北大的教学实践表明，这是一本值得推荐采用的好教材。本书第3版采用最新x86-64架构来贯穿各部分知识。我相信，该书的出版将有助于国内计算机系统教学的进一步改进，为培养从事系统级创新的计算机人才奠定很好的基础。

——梅宏 中国科学院院士/发展中国家科学院院士

以低年级开设“深入理解计算机系统”课程为基础，我先后在复旦大学和上海交通大学软件学院主导了激进的教学改革……现在我课题组的青年教师全部是首批经历此教学改革的学生。本科的扎实基础为他们从事系统软件的研究打下了良好的基础……师资力量的补充又为推进更加激进的教学改革创造了条件。

——臧斌宇 上海交通大学软件学院院长

推荐阅读



计算机系统：系统架构与操作系统的高度集成

作者：Umakishore Ramachandran 译者：陈文光
ISBN: 978-7-111-50636-2 定价：99.00元



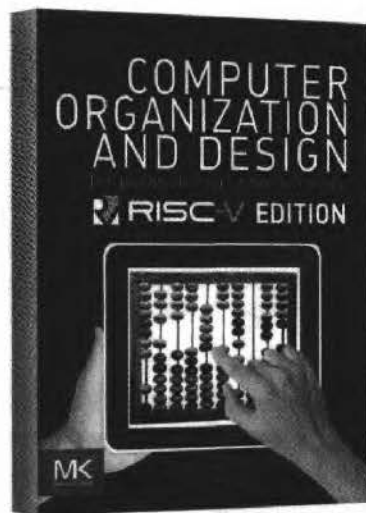
计算机系统：核心概念及软硬件实现（原书第4版）

作者：J. Stanley Warford 译者：龚奕利
ISBN: 978-7-111-50783-3 定价：79.00元



数字逻辑设计与计算机组成

作者：Nikrouz Faroughi 译者：戴志涛
ISBN: 978-7-111-57061-5 定价：89.00元



计算机组成与设计：硬件/软件接口（原书第5版·RISC-V版）

作者：David A. Patterson, John L. Hennessy 译者：易江芳
预计2019年9月出版

计算机组成与设计 硬件/软件接口 原书第5版·ARM版

Computer Organization and Design The Hardware/Software Interface ARM Edition

很高兴看到本书推出了ARM版，这本书曾启发了一代又一代的计算机科学家和工程师。ARM版不仅能够帮助世界各地的读者学习并掌握计算机组成与设计的理论知识，同时能够带领大家体验当今移动计算中使用最广泛的处理器体系结构。

—— Khaled Benkrid, ARM Ltd.

教材的选择往往是一个令人沮丧的妥协过程——教学方法的适用度、知识点的覆盖范围、文辞的流畅性、内容的严谨度、成本的高低等都需要考虑。本书之所以是难得一见的好书，正是因为它能满足各个方面的要求，不再需要任何妥协。这不仅是一部关于计算机组成的教科书，也是所有计算机科学教科书的典范。

—— Michael Goldweber, Xavier University

无论是对于80后、90后还是00后，这都是一本应该珍藏在书架上（或iPad中）的计算机体系结构教材。这本书既古老又新颖，不仅介绍了那些伟大的原理——摩尔定律、抽象、加速大概率事件、可靠性、存储器层次结构、并行和流水线，而且使用现代设计对这些伟大原理进行了说明。

—— Mark D. Hill, University of Wisconsin-Madison

本书不仅仅会讲解计算机体系结构，而且为读者准备了迎接新的变化与挑战的“锦囊”。目前，半导体工艺技术按比例缩小的困难使得所有系统功率受限，而移动系统和大数据处理的性能需求却仍在不断增长。在计算技术这一新领域，必须进行软硬件协同设计，并且系统级体系结构优化与部件级优化一样重要。

—— Christos Kozyrakis, Stanford University

Patterson和Hennessy讨论了不断变化的计算机硬件体系结构中的重要议题，强调硬件和软件模块在不同抽象层次上的交互。书中涵盖各种硬件和软件机制，I/O和并行的概念贯穿其中，全景式呈现了后PC时代的计算机体系结构。无论是平板电脑硬件工程师还是云计算软件架构师，如果你正对能源效率和并行化问题一筹莫展，那么本书必将成为不二之选。

—— Jae C. Oh, Syracuse University

本书译自原版 *Computer Organization and Design: The Hardware/Software Interface, ARM Edition* 并由 Elsevier 授权出版

ELSEVIER



投稿热线: (010) 88379604
客服热线: (010) 88378991 88361066
购书热线: (010) 68326294 88379649 68995259

华章网站: www.hzbook.com
网上购书: www.china-pub.com
数字阅读: www.hzmedia.com.cn

上架指导: 计算机/计算机组成

ISBN 978-7-111-60894-3



9 787111 608943 >

定价: 139.00元